Základy numerické matematiky

přepsáno z poznámek doc. Ivety Hnětynkové a doc. Václava Kučery

Jan Koška

25. prosince 2025

Obsah

1 Základní principy a pojmy **13**
1.1 Od reálného problému k řešení . . . 13
1.2 Korektnost a podmíněnost úlohy . . . 13
1.3 Aritmetika s plovoucí čárkou . . . 14
1.3.1 Nebezpečné operace . . . 15
1.4 Stabilita algoritmů . . . 16
1.4.1 Gaussova eliminace a LU rozklad . . . 17
1.5 Analýza chyb . . . 17
1.6 Přehled témat kurzu . . . 17

2 Řešení soustav lineárních algebraických rovnic **18**
2.1 Přímé metody . . . 18
2.2 Iterační metody pro řešení soustav . . . 19
2.2.1 Jacobiho metoda . . . 20
2.2.2 Gauss-Seidelova metoda . . . 21
2.3 Konvergence stacionárních metod . . . 22
2.4 Konvergence pro speciální matice . . . 24
2.5 Iterační zpřesnění . . . 26

3 Ortogonální transformace **27**
3.1 Citlivost řešení lineárních algebraických rovnic na chyby v pravé straně . . . 27
3.2 Givensovy rotace v $\mathbf{R}$ . . . 28
3.3 Householderovy reflexe v $\mathbf{R}^n$ . . . 30
3.4 Zpětná stabilita . . . 33
3.5 QR rozklad . . . 33
3.6 Gram-Schmidtova ortogonalizace . . . 36

4 Řešení lineárních aproximačních úloh **40**
4.1 Problém nejmenších čtverců . . . 40
4.1.1 Řešení LS s minimální normou . . . 42
4.2 Úplný problém nejmenších čtverců . . . 43
4.3 Metody řešení LS problému v případě triviálního jádra . . . 45
4.3.1 Řešení pomocí QR rozkladu . . . 45
4.3.2 Řešení pomocí normálních rovnic . . . 46

4.3.3 Rozšířená matice . . . 47
4.4 Metody řešení LS problému v případě netriviálního jádra . . . 47
4.4.1 Řešení pomocí SVD . . . 47
4.4.2 Rank-Revealing QR . . . 48

5 Částečný problém vlastních čísel **50**
5.1 Mocninná metoda . . . 50
5.1.1 Aproximace vlastního čísla . . . 51
5.1.2 Inverzní mocninná metoda . . . 52
5.2 Krylovovy prostory . . . 53
5.2.1 Výpočet báze Krylovova prostoru . . . 55
5.2.2 Lanczosova tridiagonalizace . . . 57
5.3 Ritzova čísla . . . 59
5.3.1 Shrnutí a speciální případy . . . 60

6 Úplný problém vlastních čísel **62**
6.1 Schurova věta . . . 62
6.2 QR algoritmus . . . 65
6.2.1 Implementační aspekty . . . 69
6.3 QR algoritmus pro SVD . . . 69

7 Kořeny nelineárních algebraických rovnic **72**
7.1 Metoda bisekce . . . 72
7.2 Metoda regula falsi . . . 73
7.3 Newtonova metoda . . . 73
7.3.1 Rychlost konvergence . . . 75
7.4 Modifikace Newtonovy metody . . . 77
7.4.1 Newton s diferencemi . . . 77
7.4.2 Metoda sečen . . . 77

8 Iterace funkcí **79**
8.1 Metoda prosté iterace . . . 79
8.2 Newtonova metoda v $\mathbf{R}^N$ . . . 81

9 Optimalizace **83**
9.1 Minimalizace v $\mathbf{R}$ - Metoda zlatého řezu . . . 83
9.2 Newtonova metoda pro minimalizaci . . . 85
9.3 Optimalizace v $\mathbf{R}^N$ . . . 86
9.3.1 Metody spádových směrů . . . 86
9.4 Newtonova metoda v $\mathbf{R}^N$ . . . 89
9.4.1 Obecný algoritmus a modifikace . . . 91

10 Ortogonální polynomy **92**
10.1 Legendreovy polynomy . . . 94
10.2 Čebyševovy polynomy . . . 94

11 Interpolace funkcí **97**
11.1 Lagrangeova interpolace . . . 97
11.2 Konvergence interpolace . . . 101

12 Spline funkce **104**
12.1 Lineární interpolační spliny . . . 104
12.2 Kubické interpolační spliny . . . 105
12.2.1 Okrajové podmínky a konstrukce . . . 106
12.2.2 Konvergence . . . 108

13 Kvadratura **110**
13.1 Newton-Cotesovy vzorce . . . 111
13.2 Chyba kvadraturních vzorců . . . 113
13.3 Gaussova kvadratura . . . 114
13.4 Složené kvadraturní vzorce . . . 117
13.5 Metoda polovičního kroku . . . 119

14 Jednokrokové metody řešení ODR **121**
14.1 Odhad chyby a konvergence . . . 124
14.2 Absolutní stabilita . . . 127
14.3 Konstrukce metod . . . 128
14.4 Metoda polovičního kroku . . . 131

15 Vícekrokové metody **133**
15.1 Konstrukce . . . 135
15.2 Lokální diskretizační chyba a řád metody . . . 136
15.3 Analýza řádu a konzistence . . . 136
15.4 Stabilita . . . 139

Seznam vět a definic

1.1 Definice (Korektnost úlohy (Hadamard)) . . . 13
1.2 Definice (Podmíněnost úlohy) . . . 13
1.3 Definice (Zpětná stabilita algoritmu) . . . 16
1.4 Definice (LU rozklad) . . . 16
1.1 Věta (O stabilitě LU rozkladu) . . . 17
2.1 Definice (Řídkost matice) . . . 19
2.1 Věta (Konvergence stacionárních metod) . . . 22
2.2 Definice (Spektrální poloměr) . . . 23
2.2 Lemma (Vztah normy a spektrálního poloměru) . . . 23
2.3 Věta (Oldenburgova) . . . 23
2.4 Věta . . . 24
2.3 Definice (Geršgorinův kruh) . . . 24
2.5 Věta (Geršgorinova) . . . 24
2.4 Definice (Ostře diagonálně dominantní matice (ODD)) . . . 25
2.6 Věta (Konvergence ODD) . . . 25
3.1 Věta (O podmíněnosti řešení soustavy) . . . 27
3.1 Definice (Elementární Givensova rotace) . . . 28
3.2 Definice (Givensova rotace v $\mathbf{R}^n$) . . . 28
3.2 Lemma (Vlastnosti Givensovy rotace) . . . 29
3.3 Definice (Nadrovina zrcadlení) . . . 30
3.4 Definice (Householderova reflexe v $\mathbf{R}^n$) . . . 31
3.3 Lemma (Vlastnosti Householderovy reflexe) . . . 31
3.4 Věta (Zpětná stabilita unitárních transformací) . . . 32
3.5 Definice (QR rozklad) . . . 33
3.5 Lemma (Rozklad ortogonálního projektoru) . . . 37
3.6 Věta (Zpětná stabilita QR rozkladů) . . . 39
4.1 Definice (Problém nejmenších čtverců (LS)) . . . 40
4.1 Věta (LS řešení s minimální normou) . . . 41
4.2 Definice (Problém úplných nejmenších čtverců (TLS)) . . . 42
4.3 Definice (Frobeniova norma matice) . . . 43
4.2 Lemma (Vlastnosti QR rozkladu) . . . 45
4.3 Věta (Řešení LS pomocí rozšířené matice) . . . 47
4.4 Definice (Pseudoinverze matice (Moore-Penroseova)) . . . 48
5.1 Definice (Rayleighův podíl) . . . 51
5.1 Věta (Konvergence mocninné metody) . . . 51
5.2 Definice (Krylovův prostor) . . . 53
5.3 Definice (Stupeň vektoru) . . . 53
5.2 Věta (Vlastnosti stupně vektoru) . . . 54
5.3 Lemma (Korektnost Arnoldiho procesu) . . . 55
5.4 Lemma (Vztahy v Arnoldiho procesu) . . . 56
5.5 Věta (Arnoldiho proces pro Hermitovskou matici) . . . 57
5.6 Věta (Vztahy v Lanczosově procesu) . . . 58
5.4 Definice (Ritzův pár) . . . 59
5.7 Lemma (Vlastnosti Ritzova rezidua) . . . 59
5.8 Lemma (Vlastnosti Ritzova páru v Lanczosově metodě) . . . 61
6.1 Lemma (Podobnostní transformace a spektrum) . . . 62

6.2 Věta (Schurova) . . . 62
6.1 Definice (Schurův rozklad) . . . 63
6.3 Věta (Unitární diagonalizovatelnost normálních matic) . . . 63
6.4 Věta (Hustota diagonalizovatelných matic) . . . 63
6.5 Věta (Abel & Galois & Ruffini) . . . 64
6.6 Lemma (Vlastnosti QR iterací) . . . 66
6.7 Věta (Konvergence QR algoritmu) . . . 67
6.8 Lemma (Mocninná metoda v QR iteracích) . . . 68
6.9 Lemma (Konvergence prvního sloupce) . . . 68
7.1 Věta (Existence řešení) . . . 72
7.1 Definice (Newtonova metoda) . . . 73
7.2 Definice (Řád konvergence) . . . 74
7.2 Věta (Lokální kvadratická konvergence Newtonovy metody) . . . 75
7.3 Věta (Li-Yorke) . . . 77
8.1 Definice (Kontrakce) . . . 80
8.1 Věta (Banachova věta o kontrakci) . . . 80
8.2 Lemma (Lineární konvergence prosté iterace) . . . 81
9.1 Definice (Lokální a globální minimum) . . . 83
9.2 Definice (Newtonova metoda pro minimalizaci) . . . 85
9.1 Věta (Podmínky pro extrém v $\mathbf{R}^N$) . . . 86
9.3 Definice (Metoda spádového směru) . . . 86
9.4 Definice (Spádový směr) . . . 87
9.2 Lemma (Charakterizace spádového směru) . . . 87
9.3 Lemma (Směr největšího spádu) . . . 87
9.5 Definice (Metoda největšího spádu) . . . 88
9.4 Lemma (Zig-zag efekt) . . . 88
9.5 Věta (Konvergence metody největšího spádu) . . . 89
9.6 Věta (Konvergence pro konvexní funkce) . . . 89
9.6 Definice (Newtonova metoda pro minimalizaci) . . . 90
9.7 Lemma (O spádovém směru) . . . 90
10.1 Definice (Skalární součin s vahou) . . . 92
10.2 Definice (Vektorový prostor polynomů) . . . 92
10.3 Definice (Ortogonální polynomy) . . . 92
10.1 Věta (3-členná rekurence) . . . 93
10.2 Věta (Kořeny ortogonálních polynomů) . . . 94
10.4 Definice (Legendreovy polynomy) . . . 94
10.5 Definice (Čebyševovy polynomy) . . . 94
10.3 Věta (Rekurence pro Čebyševovy polynomy) . . . 95
10.4 Věta (Ortogonalita Čebyševových polynomů) . . . 95
10.5 Věta (Monické Čebyševovy polynomy) . . . 95
10.6 Věta (Min-Max vlastnost) . . . 95
11.1 Definice (Lagrangeova interpolace) . . . 97
11.2 Definice (Lagrangeovy bázové polynomy) . . . 98
11.1 Lemma (Vlastnost bázových polynomů) . . . 98
11.2 Věta (Existence a jednoznačnost interpolačního polynomu) . . . 99
11.3 Věta (Odhad chyby (Cauchy)) . . . 99
11.4 Věta (Analýza chyby u Rungeho jevu) . . . 101
11.3 Definice (Čebyševova interpolace) . . . 102

11.5 Věta (Konvergence pro Čebyševovy uzly) . . . 102
11.6 Věta (Faberova věta (o divergenci)) . . . 103
12.1 Definice (Lineární interpolační spline) . . . 104
12.1 Věta (Odhad chyby lineárního splinu) . . . 104
12.2 Definice (Kubický interpolační spline) . . . 105
12.3 Definice (Momenty splinu) . . . 106
12.2 Lemma (Soustava lineárních rovnic pro momenty M_i) . . . 106
12.3 Věta (Odhad chyby kubického splinu pro $f \in \mathcal{C}^4$) . . . 108
12.4 Věta (Konvergence pro spojité funkce) . . . 108
13.1 Definice (Obecný kvadraturní vzorec) . . . 110
13.2 Definice (Kvadraturní vzorec určený uzly Lagrangeovy interpolace) . . . 111
13.1 Věta (Simpsonovo pravidlo je přesné pro $\mathcal{P}_3$) . . . 112
13.3 Definice (Algebraický řád kvadratury) . . . 112
13.2 Věta (Řád Newton-Cotesových vzorců) . . . 113
13.3 Věta (Odhad chyby obecné kvadratury) . . . 113
13.4 Definice (Gaussova kvadratura) . . . 114
13.4 Věta (Gaussova kvadratura a ortogonální polynomy) . . . 115
13.5 Věta (Odhad chyby Gaussovy kvadratury) . . . 115
13.6 Věta (Chyba složeného lichoběžníkového pravidla) . . . 117
13.7 Věta (Odhad chyby složené kvadratury metodou poloviční kroku) . . . 119
14.1 Definice (Eulerova metoda) . . . 121
14.2 Definice (Heunova metoda) . . . 123
14.3 Definice (Jednokroková metoda (JKM)) . . . 123
14.4 Definice (Konzistence metody) . . . 123
14.5 Definice (Lokální diskretizační chyba) . . . 124
14.6 Definice (Řád metody) . . . 124
14.1 Lemma (Konzistence jednokrokové metody) . . . 124
14.7 Definice (Globální chyba) . . . 125
14.2 Věta (Odhad chyby jednokrokové metody) . . . 125
14.3 Věta (Vztah konzistence a konvergence) . . . 126
14.8 Definice (A-stabilita) . . . 127
14.9 Definice (Dvoustupňová Runge-Kuttova metoda) . . . 129
14.10 Definice (Obecná explicitní Runge-Kutta metoda) . . . 130
14.4 Věta (Odhad chyby řešení ODR metodou poloviční kroku) . . . 131
15.1 Definice (k-kroková metoda) . . . 133
15.1 Věta (Konvergence k řešení implicitní k-krokové metody) . . . 134
15.2 Definice (Lokální diskretizační chyba) . . . 136
15.3 Definice (Řád vícekrokové metody) . . . 136
15.2 Věta (Podmínky řádu VKM) . . . 136
15.4 Definice (Konzistentní VKM) . . . 137
15.3 Lemma (Podmínka konzistence) . . . 137
15.5 Definice (Charakteristický polynom) . . . 139
15.4 Lemma (Kořen charakteristického polynomu) . . . 140
15.5 Věta (Obecné řešení pro jednoduché kořeny) . . . 140
15.6 Věta (Podmínka stability VKM) . . . 141
15.7 Lemma (Řešení pro vícenásobné kořeny) . . . 141
15.6 Definice (Stabilita VKM) . . . 142
15.8 Věta (Dahlquist) . . . 142

Seznam definic

1.1 Definice (Korektnost úlohy (Hadamard)) . . . 13
1.2 Definice (Podmíněnost úlohy) . . . 13
1.3 Definice (Zpětná stabilita algoritmu) . . . 16
1.4 Definice (LU rozklad) . . . 16
2.1 Definice (Řídkost matice) . . . 19
2.2 Definice (Spektrální poloměr) . . . 23
2.3 Definice (Geršgorinův kruh) . . . 24
2.4 Definice (Ostře diagonálně dominantní matice (ODD)) . . . 25
3.1 Definice (Elementární Givensova rotace) . . . 28
3.2 Definice (Givensova rotace v $\mathbf{R}^n$) . . . 28
3.3 Definice (Nadrovina zrcadlení) . . . 30
3.4 Definice (Householderova reflexe v $\mathbf{R}^n$) . . . 31
3.5 Definice (QR rozklad) . . . 33
4.1 Definice (Problém nejmenších čtverců (LS)) . . . 40
4.2 Definice (Problém úplných nejmenších čtverců (TLS)) . . . 42
4.3 Definice (Frobeniova norma matice) . . . 43
4.4 Definice (Pseudoinverze matice (Moore-Penroseova)) . . . 48
5.1 Definice (Rayleighův podíl) . . . 51
5.2 Definice (Krylovův prostor) . . . 53
5.3 Definice (Stupeň vektoru) . . . 53
5.4 Definice (Ritzův pár) . . . 59
6.1 Definice (Schurův rozklad) . . . 63
7.1 Definice (Newtonova metoda) . . . 73
7.2 Definice (Řád konvergence) . . . 74
8.1 Definice (Kontrakce) . . . 80
9.1 Definice (Lokální a globální minimum) . . . 83
9.2 Definice (Newtonova metoda pro minimalizaci) . . . 85
9.3 Definice (Metoda spádového směru) . . . 86
9.4 Definice (Spádový směr) . . . 87
9.5 Definice (Metoda největšího spádu) . . . 88
9.6 Definice (Newtonova metoda pro minimalizaci) . . . 90
10.1 Definice (Skalární součin s vahou) . . . 92
10.2 Definice (Vektorový prostor polynomů) . . . 92
10.3 Definice (Ortogonální polynomy) . . . 92
10.4 Definice (Legendreovy polynomy) . . . 94
10.5 Definice (Čebyševovy polynomy) . . . 94
11.1 Definice (Lagrangeova interpolace) . . . 97
11.2 Definice (Lagrangeovy bázové polynomy) . . . 98
11.3 Definice (Čebyševova interpolace) . . . 102
12.1 Definice (Lineární interpolační spline) . . . 104
12.2 Definice (Kubický interpolační spline) . . . 105
12.3 Definice (Momenty splinu) . . . 106
13.1 Definice (Obecný kvadraturní vzorec) . . . 110
13.2 Definice (Kvadraturní vzorec určený uzly Lagrangeovy interpolace) . . . 111
13.3 Definice (Algebraický řád kvadratury) . . . 112
13.4 Definice (Gaussova kvadratura) . . . 114

14.1 Definice (Eulerova metoda) . . . 121
14.2 Definice (Heunova metoda) . . . 123
14.3 Definice (Jednokroková metoda (JKM)) . . . 123
14.4 Definice (Konzistence metody) . . . 123
14.5 Definice (Lokální diskretizační chyba) . . . 124
14.6 Definice (Řád metody) . . . 124
14.7 Definice (Globální chyba) . . . 125
14.8 Definice (A-stabilita) . . . 127
14.9 Definice (Dvoustupňová Runge-Kuttova metoda) . . . 129
14.10 Definice (Obecná explicitní Runge-Kutta metoda) . . . 130
15.1 Definice (k-kroková metoda) . . . 133
15.2 Definice (Lokální diskretizační chyba) . . . 136
15.3 Definice (Řád vícekrokové metody) . . . 136
15.4 Definice (Konzistentní VKM) . . . 137
15.5 Definice (Charakteristický polynom) . . . 139
15.6 Definice (Stabilita VKM) . . . 142

Seznam vět

1.1 Věta (O stabilitě LU rozkladu) . . . 17
2.1 Věta (Konvergence stacionárních metod) . . . 22
2.2 Lemma (Vztah normy a spektrálního poloměru) . . . 23
2.3 Věta (Oldenburgova) . . . 23
2.4 Věta . . . 24
2.5 Věta (Geršgorinova) . . . 24
2.6 Věta (Konvergence ODD) . . . 25
3.1 Věta (O podmíněnosti řešení soustavy) . . . 27
3.2 Lemma (Vlastnosti Givensovy rotace) . . . 29
3.3 Lemma (Vlastnosti Householderovy reflexe) . . . 31
3.4 Věta (Zpětná stabilita unitárních transformací) . . . 32
3.5 Lemma (Rozklad ortogonálního projektoru) . . . 37
3.6 Věta (Zpětná stabilita QR rozkladů) . . . 39
4.1 Věta (LS řešení s minimální normou) . . . 41
4.2 Lemma (Vlastnosti QR rozkladu) . . . 45
4.3 Věta (Řešení LS pomocí rozšířené matice) . . . 47
5.1 Věta (Konvergence mocninné metody) . . . 51
5.2 Věta (Vlastnosti stupně vektoru) . . . 54
5.3 Lemma (Korektnost Arnoldiho procesu) . . . 55
5.4 Lemma (Vztahy v Arnoldiho procesu) . . . 56
5.5 Věta (Arnoldiho proces pro Hermitovskou matici) . . . 57
5.6 Věta (Vztahy v Lanczosově procesu) . . . 58
5.7 Lemma (Vlastnosti Ritzova rezidua) . . . 59
5.8 Lemma (Vlastnosti Ritzova páru v Lanczosově metodě) . . . 61
6.1 Lemma (Podobnostní transformace a spektrum) . . . 62
6.2 Věta (Schurova) . . . 62
6.3 Věta (Unitární diagonalizovatelnost normálních matic) . . . 63
6.4 Věta (Hustota diagonalizovatelných matic) . . . 63
6.5 Věta (Abel & Galois & Ruffini) . . . 64

6.6 Lemma (Vlastnosti QR iterací) . . . 66
6.7 Věta (Konvergence QR algoritmu) . . . 67
6.8 Lemma (Mocninná metoda v QR iteracích) . . . 68
6.9 Lemma (Konvergence prvního sloupce) . . . 68
7.1 Věta (Existence řešení) . . . 72
7.2 Věta (Lokální kvadratická konvergence Newtonovy metody) . . . 75
7.3 Věta (Li-Yorke) . . . 77
8.1 Věta (Banachova věta o kontrakci) . . . 80
8.2 Lemma (Lineární konvergence prosté iterace) . . . 81
9.1 Věta (Podmínky pro extrém v $\mathbf{R}^N$) . . . 86
9.2 Lemma (Charakterizace spádového směru) . . . 87
9.3 Lemma (Směr největšího spádu) . . . 87
9.4 Lemma (Zig-zag efekt) . . . 88
9.5 Věta (Konvergence metody největšího spádu) . . . 89
9.6 Věta (Konvergence pro konvexní funkce) . . . 89
9.7 Lemma (O spádovém směru) . . . 90
10.1 Věta (3-členná rekurence) . . . 93
10.2 Věta (Kořeny ortogonálních polynomů) . . . 94
10.3 Věta (Rekurence pro Čebyševovy polynomy) . . . 95
10.4 Věta (Ortogonalita Čebyševových polynomů) . . . 95
10.5 Věta (Monické Čebyševovy polynomy) . . . 95
10.6 Věta (Min-Max vlastnost) . . . 95
11.1 Lemma (Vlastnost bázových polynomů) . . . 98
11.2 Věta (Existence a jednoznačnost interpolačního polynomu) . . . 99
11.3 Věta (Odhad chyby (Cauchy)) . . . 99
11.4 Věta (Analýza chyby u Rungeho jevu) . . . 101
11.5 Věta (Konvergence pro Čebyševovy uzly) . . . 102
11.6 Věta (Faberova věta (o divergenci)) . . . 103
12.1 Věta (Odhad chyby lineárního splinu) . . . 104
12.2 Lemma (Soustava lineárních rovnic pro momenty M_i) . . . 106
12.3 Věta (Odhad chyby kubického splinu pro $f \in \mathcal{C}^4$) . . . 108
12.4 Věta (Konvergence pro spojité funkce) . . . 108
13.1 Věta (Simpsonovo pravidlo je přesné pro $\mathcal{P}_3$) . . . 112
13.2 Věta (Řád Newton-Cotesových vzorců) . . . 113
13.3 Věta (Odhad chyby obecné kvadratury) . . . 113
13.4 Věta (Gaussova kvadratura a ortogonální polynomy) . . . 115
13.5 Věta (Odhad chyby Gaussovy kvadratury) . . . 115
13.6 Věta (Chyba složeného lichoběžníkového pravidla) . . . 117
13.7 Věta (Odhad chyby složené kvadratury metodou polovičního kroku) . . . 119
14.1 Lemma (Konzistence jednokrokové metody) . . . 124
14.2 Věta (Odhad chyby jednokrokové metody) . . . 125
14.3 Věta (Vztah konzistence a konvergence) . . . 126
14.4 Věta (Odhad chyby řešení ODR metodou polovičního kroku) . . . 131
15.1 Věta (Konvergence k řešení implicitní k-krokové metody) . . . 134
15.2 Věta (Podmínky řádu VKM) . . . 136
15.3 Lemma (Podmínka konzistence) . . . 137
15.4 Lemma (Kořen charakteristického polynomu) . . . 140
15.5 Věta (Obecné řešení pro jednoduché kořeny) . . . 140

15.6 Věta (Podmínka stability VKM) 141
15.7 Lemma (Řešení pro vícenásobné kořeny) 141
15.8 Věta (Dahlquist) 142

Seznam algoritmů

1 Jacobiho metoda . . . 20
2 Gauss-Seidelova metoda . . . 21
3 Iterační zpřesnění . . . 26
4 CGS (Classical Gram-Schmidt) . . . 38
5 MGS (Modified Gram-Schmidt) . . . 38
6 Mocninná metoda . . . 51
7 Arnoldiho algoritmus (MGS varianta) . . . 56
8 Lanczosův algoritmus (krátké rekurence) . . . 58
9 Arnoldiho metoda aproximace vlastních párů . . . 60
10 QR algoritmus . . . 67
11 Metoda bisekce . . . 72
12 Zúžení intervalu obsahující minimum . . . 84
13 Konstrukce ortogonální báze (Gram-Schmidt) . . . 93

Seznam obrázků

1 Růst chyby při výpočtu nestabilní rekurence v aritmetice s plovoucí čárkou 15
2 Rotace vektoru v rovině . . . 28
3 Householderova reflexe podle nadroviny $\mathcal{H}(q)$ dané normálovým vektorem $q \in \mathbf{R}^n$ . . . 30
4 Tvary matic při QR rozkladu pro různé dimenze . . . 34
5 Postup QR rozkladu pomocí ortogonálních transformací . . . 35
6 Projekce na jednorozměrný podprostor . . . 37
7 Projekce na ortogonální doplněk . . . 37
8 Srovnání CGS a MGS . . . 38
9 Geometrická interpretace problému nejmenších čtverců . . . 40
10 Projekce pravé strany na obraz matice . . . 41
11 Množina řešení LS v případě neúplné sloupcové hodnosti . . . 42
12 Srovnání LS a TLS regrese . . . 44
13 Schéma QR rozkladu . . . 45
14 Motivace mocninné metody . . . 50
15 Vnoření Krylovových prostorů . . . 53
16 Konvergence Ritzových čísel k vlastním číslům matice A . . . 60
17 Vztahy mezi třídami matic a jejich rozklady . . . 64
18 Deflace v QR algoritmu . . . 70
19 Metoda půlení intervalu. . . . 73
20 Metoda Regula falsi. . . . 74
21 Newtonova metoda . . . 74
22 Metoda sečen. . . . 78
23 Divergence ($|g'| > 1$) . . . 79
24 Monotónní konvergence ($0 < g' < 1$) . . . 79
25 Oscilující konvergence ($-1 < g' < 0$) . . . 79
26 Pavučinové diagramy (Cobweb plots) . . . 79
27 Iterace funkce $g(x) = \ln(2x + 1)$ . . . 80
28 Princip kontrakce a konvergence k pevnému bodu . . . 81
29 Metoda zlatého řezu pro unimodální funkci . . . 84

30 Opakované využití dělícího bodu při metodě zlatého řezu 85
31 Newtonova metoda pro minimalizaci 86
32 Iterační metoda postupující přes vrstevnice k minimu. 87
33 Jeden krok metody největšího spádu. 88
34 Zig-zag efekt metody největšího spádu u špatně podmíněné úlohy (úzké eliptické vrstevnice). Po sobě jdoucí kroky jsou na sebe kolmé. 89
35 Srovnání Newtonovy metody a metody největšího spádu na nekvadratické funkci (Rosenbrockova funkce). Newtonova metoda (modře) konverguje k minimu v několika málo krocích, zatímco gradientní metoda (červeně) osciluje a konverguje pomalu. 90
36 Legendreovy polynomy 95
37 Čebyševovy polynomy (ortogonální na $[-1,1]$ s vahou $w(x)=\frac{1}{\sqrt{1-x^2}}$) . . . 96
38 Proložení bodů grafem polynomu 97
39 Lineární interpolace 98
40 Lagrangeovy bázové polynomy pro $n=1$ 98
41 Ilustrace Rolleovy věty pro chybu interpolace 101
42 Rungeho jev: funkce $f(x)$ a interpolační polynom $L_{10}(x)$ na rovnoměrném dělení 101
43 Kořeny Čebyševova polynomu pro $n=9$ 102
44 Lineární interpolační spline 104
45 Kubický spline 108
46 Obdélníkové pravidlo 110
47 Lichoběžníkové pravidlo 112
48 Simpsonovo pravidlo 112
49 Legendreův polynom $\mathcal{L}_2(x)$ a jeho kořeny, které tvoří uzly Gaussovy kvadratury pro $n=1$. 115
50 Složené lichoběžníkové pravidlo: aproximace funkce lomenou čarou. 118
51 Složené obdélníkové pravidlo. 118
52 Složené Simpsonovo pravidlo: aproximace pomocí parabol na dvojičkách intervalů. 119
53 Směrové pole a integrální křivka 121
54 Princip Eulerovy metody 122
55 Lokální diskretizační chyba vs. globální chyba 125
56 Stabilita Eulerovy metody 128
57 Metoda polovičního kroku – zjemnění sítě 131
58 Konvergence implicitní VKM 134
59 Geometrický význam Adams-Bashforthovy metody: extrapolace f pomocí polynomu $L_{k-1}(x)$ a integrace pod ním. 135
60 Ukázka nestability metody $y_{n+2}-4y_{n+1}+3y_n=0$ s počáteční poruchou $y_1=\varepsilon$ (logaritmická škála). 141

1 Základní principy a pojmy

1.1 Od reálného problému k řešení

Numerické řešení reálného problému obvykle probíhá v cyklu, který zahrnuje několik fází, přičemž každá z nich vnáší do výsledku určité chyby:

1. **Reálný problém** se převede na **matematický model**. Model je zjednodušený popis problému jazykem matematiky (rovnice, tabulky), vstupují do něj měřená data, která jsou často nepřesná. Zde vznikají **chyby modelu** (chyby v datech, zjednodušení).

2. Pro matematický model se hledá **výpočetní metoda**. Jedná se o postup vedoucí k získání přibližného řešení. Zde vznikají **chyby aproximace** (diskretizace spojitého problému, uříznutí řad).

3. Metoda se implementuje – **algoritmizace**. Počítačová aritmetika není přesná (FPA – Floating Point Arithmetic), což vnáší **zaokrouhlovací chyby**.

Celý proces uzavírá **verifikace**, kdy se přibližné řešení porovnává s realitou.

Numerická analýza se zabývá zkoumáním těchto fází:

- U matematického modelu zkoumáme jeho **korektnost** a **citlivost** na chyby v datech.
- U výpočetní metody zkoumáme její **stabilitu** a **aproximační vlastnosti**.

1.2 Korektnost a podmíněnost úlohy

Zabývejme se otázkou, jak se změní řešení úlohy při malé změně (perturbaci) vstupních dat.

Příklad 1.1 (Citlivost lineární soustavy)**.** Uvažujme dvě podobné soustavy lineárních rovnic:

$$\begin{aligned} 2x + 6y &= 8 \\ 2x + 6.00001y &= 8.00001 \end{aligned} \quad \Longrightarrow \quad x = 1, y = 1$$

$$\begin{aligned} 2x + 6y &= 8 \\ 2x + 5.99999y &= 8.00002 \end{aligned} \quad \Longrightarrow \quad x = 10, y = -2$$

Malá změna v koeficientech způsobila obrovskou změnu v řešení.

Zavedeme značení: d jsou vstupní data (např. A, b), U je úloha (např. $Ax = b$), $U(d)$ je přesné řešení pro data d.

DEFINICE 1.1: *Korektnost úlohy (Hadamard)*

Řekneme, že úloha U je **korektní** (well-posed), pokud:

1. Pro každé d z množiny přípustných dat D existuje řešení $U(d)$.
2. Řešení $U(d)$ je jednoznačné.
3. Řešení $U(d)$ závisí spojitě na datech úlohy.

Jinak je úloha **nekorektní** (ill-posed).

DEFINICE 1.2: *Podmíněnost úlohy*

Řekneme, že korektní úloha U je **dobře podmíněná**, pokud malá relativní změna vstupních dat $|\Delta d|/|d|$ vyvolá malou relativní změnu $|\Delta U(d)|/|U(d)|$ řešení úlohy. Jinak je úloha **špatně podmíněná**.

Pro soustavy lineárních rovnic $Ax = b$ s regulární maticí A platí odhad relativní chyby:

$$\frac{\|\Delta x\|}{\|x\|} \leq \|A\|\|A^{-1}\|\frac{\|\Delta b\|}{\|b\|} = \varkappa(A)\frac{\|\Delta b\|}{\|b\|},$$

kde $\varkappa(A) = \|A\|\|A^{-1}\|$ je **číslo podmíněnosti** matice A.

Poznámka 1.1 (Značení čísla podmíněnosti)**.** Podmíněnost úlohy značíme $\varkappa(A)$, čteme „podmíněnost matice A“ nebo „kappa A“. Písmena κ a $\varkappa$ jsou stejné.

- Je-li $\varkappa(A)$ malé, soustava je dobře podmíněná.
- Je-li $\varkappa(A)$ velké, soustava může být velmi citlivá na chyby.

1.3 Aritmetika s plovoucí čárkou

Reálná čísla jsou v počítači reprezentována v tzv. **floating point numbers** (dle standardu IEEE 754). Číslo je uloženo jako trojice (znaménko, exponent, mantisa):

$$x = (-1)^s \cdot m \cdot 2^e.$$

Přesnost je dána počtem bitů pro uložení každého čísla.

- **Single** (32 bit): ≈ 7 desítkových cifer, $\epsilon_{\text{mach}} \approx 10^{-8}$.
- **Double** (64 bit): ≈ 16 desítkových cifer, $\epsilon_{\text{mach}} \approx 10^{-16}$.

Protože množina reprezentovatelných čísel je konečná, vznikají zaokrouhlovací chyby. Relativní přesnost uložení čísel se zachovává, ale absolutní chyba roste s velikostí čísla.

Pro operaci násobení čísel v FPA platí (podobně pro další základní operace):

$$\text{fl}(x \cdot y) = (x \cdot y)(1 + \delta), \quad |\delta| < C\epsilon_{\text{mach}},$$

kde ϵ_{mach} je **strojová přesnost**. V FPA neplatí přesně některé algebraické zákony (např. asociativita, distributivita). Komutativita se však paradoxně zachovává, ale neasociativita nám brání v tom měnit pořadí operací, jak se nám zachce.

Příklad 1.2 (Komutativní & neasociativní operace)**.** Python výraz `10 ** 16 + float(1) + (-10 ** 16)` se ozávorkuje jako $(10^{16} + 1) + (-10^{16})$, a z komutativity je ekvivalentní výrazu $(1 + 10^{16}) + (-10^{16})$. Protože sčítání není asociativní, výraz nelze dál upravit na $(10^{16} - 10^{16}) + 1 = 1$. Vyhodnocením výrazu v Pythonu uvidíme, že výsledek je 0.

1.3.1 Nebezpečné operace

V důsledku zaokrouhlování je kritickou operací v FPA **sčítání dvou čísel velmi rozdílného řádu**. Druhou kritickou operací je **odčítání dvou blízkých čísel** (cancellation), kdy může docházet k tzv. rušení platných cifer (catastrophic cancellation).

Příklad 1.3 (Katastrofická ztráta přesnosti)**.** Výpočet hodnoty funkce $f(x) = \frac{1-\cos x}{x^2}$ pro velmi malé $x \approx 1.2 \cdot 10^{-8}$.

- Přímý výpočet: $1 - \cos x$ odečítá dvě téměř shodná čísla (blízká 1), výsledek má málo platných cifer.
- Vhodná úprava: $\frac{1-\cos x}{x^2} = \frac{2\sin^2(x/2)}{x^2} = \frac{1}{2}\left(\frac{\sin(x/2)}{x/2}\right)^2$, což je stabilní.

Příklad 1.4 (Nestabilní rekurence)**.** Uvažujme posloupnost definovanou rekurencí:

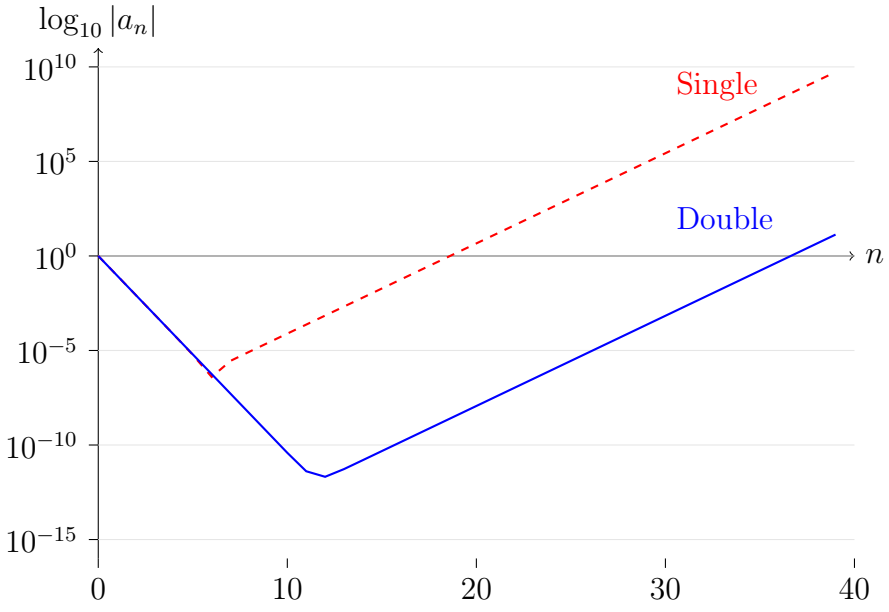
$$a_0 = 1, \quad a_1 = \frac{1}{11}, \quad a_{n+2} = \frac{34}{11}a_{n+1} - \frac{3}{11}a_n.$$

Matematicky je řešením $a_n = (1/11)^n$, což je rychle klesající posloupnost. V FPA se však vlivem zaokrouhlovacích chyb brzy projeví druhá (parazitní) složka řešení úměrná 3^n, která dominuje a způsobí exponenciální růst chyby.

Složky řešení uvidíme z maticového zápisu

$$\begin{bmatrix} a_{n+1} \\ a_{n+2} \end{bmatrix} = \frac{1}{11}\begin{bmatrix} 0 & 11 \\ -3 & 34 \end{bmatrix}\begin{bmatrix} a_n \\ a_{n+1} \end{bmatrix}$$

tak, že určíme vlastní čísla rekurenční matice.



Obrázek 1: Růst chyby při výpočtu nestabilní rekurence v aritmetice s plovoucí čárkou

Příklad 1.5 (Rumpův)**.** ukazuje, jak extrémně mohou zaokrouhlovací chyby ovlivnit výsledek výpočtu i u zdánlivě nevinného polynomu dvou proměnných:

$$u(x,y) = 333.75\,y^6 + x^2(11x^2y^2 - y^6 - 121y^4 - 2) + 5.5\,y^8 + \frac{x}{2y}$$

pro $x = 77617.0$ a $y = 33096.0$.

Pokud tento výraz vyčíslíme v plovoucí řádové čárce, dostaneme různé výsledky podle použité přesnosti:

- **Single precision:** $u(x, y) = 1{,}172603$
- **Double precision:** $u(x, y) = 1{,}1726039400531$
- **Quadruple precision:** $u(x, y) = 1{,}1726039400531788760$

Přitom správný matematický výsledek je záporný: $u(x, y) \approx -0{,}8273960599\ldots$

1.4 Stabilita algoritmů

Snahou je navrhnout takovou výpočetní metodu (a její implementaci), která nekumuluje zaokrouhlovací chyby.

DEFINICE 1.3: *Zpětná stabilita algoritmu*

Nechť $f(d)$ je výsledek spočtený přesným algoritmem a $\mathrm{fl}(f(d))$ výsledek spočtený v FPA. Řekneme, že algoritmus je **zpětně stabilní**, pokud existují modifikovaná data $(d + \Delta d)$ taková, že

$$\mathrm{fl}(f(d)) = f(d + \Delta d)$$

a navíc relativní změna vstupu $|\Delta d|/|d|$ je malá (řádu strojové přesnosti).

Zpětně stabilní algoritmus tedy dává *přesné* řešení pro *blízkou* úlohu. Je málo citlivý na zaokrouhlovací chyby.

Příklad 1.6 (Nestabilita prosté Gaussovy eliminace)**.** Uvažujme soustavu:

$$\begin{pmatrix} \epsilon & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$$

Po provedení eliminace bez pivotace (odečtení $1/\epsilon$-násobku prvního řádku od druhého) dostáváme v počítačové aritmetice pro velmi malé ϵ (blízko ϵ_{mach}):

$$\begin{pmatrix} \epsilon & 1 \\ 0 & 1 - 1/\epsilon \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 1 \\ 2 - 1/\epsilon \end{pmatrix} \quad \xrightarrow{\text{FPA}} \quad \begin{pmatrix} \epsilon & 1 \\ 0 & -1/\epsilon \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 1 \\ -1/\epsilon \end{pmatrix}$$

Zpětná substituce pak dá $x_2 = 1$, ale $x_1 = \frac{1 - x_2}{\epsilon} = 0$. Přesné řešení je přitom $x_1 \approx 1, x_2 \approx 1$. Malý pivot na diagonále způsobil velkou chybu.

Příklad 1.7 (Růst prvků při pivotaci)**.** Ani pivotace nemusí vždy pomoci. Pro matici tvaru

$$\begin{pmatrix} 1 & 0 & 0 & \ldots & 1 \\ -1 & 1 & 0 & \ldots & 1 \\ -1 & -1 & 1 & \ldots & 1 \\ \vdots & & & \ddots & \vdots \\ -1 & -1 & -1 & \ldots & 1 \end{pmatrix}$$

a pravou stranu $\big(0, -3, 0, -3, \ldots, 0, -3, 0, -2\big)^T$ dochází při eliminaci k postupnému zdvojnásobování prvků v posledním sloupci. Např. pro $n = 56$ to vede k tak obrovským číslům, že dojde k naprosté ztrátě přesnosti.

1.4.1 Gaussova eliminace a LU rozklad

Definice 1.4: *LU rozklad*

Nechť A je regulární matice. Rozklad tvaru $A = LU$, kde L je dolní trojúhelníková a U je horní trojúhelníková matice, nazveme **LU rozkladem** matice A.

Prostá Gaussova eliminace (pokud ji lze provést) odpovídá nalezení LU rozkladu. GE s řádkovou pivotací odpovídá rozkladu $PA = LU$, kde P je permutační matice.

Věta 1.1: *O stabilitě LU rozkladu*

Nechť L, U jsou výsledkem GE pro matici A řádu n počítané v FPA se strojovou přesností ϵ_{mach}, a označme $\|\cdot\|_\infty$ maximovou normu matice. Potom existuje matice ΔA tak, že $(A + \Delta A) = LU$, kde

$$\|\Delta A\|_\infty \leq 2n\epsilon_{\text{mach}}\|L\|_\infty\|U\|_\infty + O(\epsilon_{\text{mach}}^2).$$

Protože faktor

$$\|L\|_\infty\|U\|_\infty$$

může být velký, prostá GE nemusí být stabilní.

Důsledek 1.1 (Stabilita GE s řádkovou pivotací)**.**

- Prvky v matici L (násobitele řádků) jsou menší než 1, tedy $\|L\|_\infty \leq n$.
- Výpočet bude zpětně stabilní, pokud bude malý růstový faktor $\|U\|_\infty / \|A\|_\infty$.
- Říkáme, že LU rozklad s řádkovou pivotací je podmínečně zpětně stabilní.

Z tohoto odhadu plyne, že stabilita závisí na velikosti prvků v maticích L a U. Pokud prvky nerostou (což pivotace často zajistí, ale ne vždy, viz příklad výše), je metoda zpětně stabilní. Říkáme, že GE s pivotací je **podmíněčně zpětně stabilní**.

1.5 Analýza chyb

Při analýze numerického řešení rozlišujeme:

- **Přímá chyba:** $|\,\text{fl}(U(d)) - U(d)|$ (rozdíl mezi vypočteným a přesným řešením).
- **Zpětná chyba:** Nejmenší Δd, pro které platí $\text{fl}(U(d)) = U(d + \Delta d)$.

Základním cílem je používat dobře podmíněné modely (úlohy) a stabilní algoritmy. Ne vždy je to ale možné.

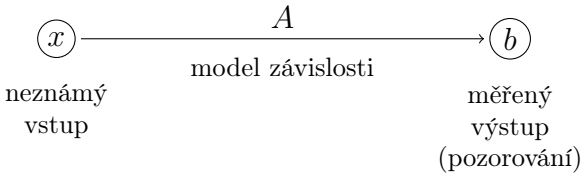
1.6 Přehled témat kurzu

- **1. část:** Soustavy lineárních rovnic ($Ax = b$), Lineární aproximační problémy ($Ax \approx b$), Částečný problém vlastních čísel ($Av = \lambda v$), Úplný problém vlastních čísel.
- **2. část:** Nelineární algebraické rovnice, Optimalizační úlohy, Aproximace funkcí, Aproximace integrálů, Obyčejné diferenciální rovnice.

2 Řešení soustav lineárních algebraických rovnic

- **Úloha:** $Ax = b$, $A \in \mathbf{C}^{n\times n}$ regulární, $b \in \mathbf{C}^n$

Lineární model



Metody řešení: přímé (Gaussova eliminace, QR rozklad, ...), iterační (probereme zde), kombinace přímých a iteračních

2.1 Přímé metody

Prostá Gaussova eliminace (bez pivotace)

Známe z LA1. Základní schéma se skládá ze dvou kroků:

1. **Přímý chod:**

$$[A \mid b] \sim \cdots \sim \left[\begin{array}{cccc|c} \times & \times & \dots & \times & \times \\ 0 & \times & \dots & \times & \times \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \dots & \times & \times \end{array}\right] = [U \mid \tilde{b}]$$

   kde U je horní trojúhelníková matice. Matice $[A \mid b]$ se označuje jeko rozšířená matice dat.

2. **Zpětný chod:**

$$[U \mid \tilde{b}] \uparrow \text{ zpětná substituce}$$

Maticový zápis

$$A = L \cdot U = \blacksquare \cdot \blacksquare,$$

kde L je matice kumulovaných elementárních transformací.

1. **Přímý chod:**

$$LUx = b \xrightarrow{\text{implicitně}} Ux = L^{-1}b \equiv \tilde{b}$$

2. **Zpětný chod:** řešení $Ux = \tilde{b}$.

GE s pivotací (částečnou – řádkovou)

- lze provést pro každou A regulární
- maticově: $PA = LU$, kde P je permutační matice.

Vlastnosti

- **Paměťové náklady** – L není třeba konstruovat a uchovávat, za běhu přepisujeme A, b novými hodnotami.
- **Výpočetní náklady** – $\frac{2}{3}n^3$.
- **Obecně nemusí být stabilní** –
  - GE s pivotací je podmínečně zpětně stabilní (růstový faktor $\frac{\|U\|}{\|A\|}$ lze monitorovat při výpočtu).

Poznámka 2.1 (GE pro HPD matici)**.** Je-li A HPD (Hermitovská pozitivně definitní), pak GE lze upravit tak, aby byla zpětně stabilní (tzv. Choleského rozklad $A = L \cdot L^*$, kde L je dolní trojúhelníková matice).

2.2 Iterační metody pro řešení soustav

Úloha $Ax = b$, $A \in \mathbf{C}^{n\times n}$ regulární, $b \in \mathbf{C}^n$

Praxe A je často velká ($n \geq 10^6$), řídká a může být daná jen implicitně (jako funkce realizující násobení Az, $z \in \mathbf{C}^n$)

DEFINICE 2.1: *Řídkost matice*

Říkáme, že matice $A \in \mathbf{C}^{n\times n}$ je *řídká*, pokud počet jejích nenulových prvků, označený l, splňuje $l \ll n^2$. Pro takovou matici lze součin Az vypočítat v čase $O(l)$ místo $O(n^2)$.

- Při GE pro řídkou matici dochází typicky k rychlému zaplnění a rostou výpočetní i paměťové náklady.

$\Longrightarrow$ chceme sestrojit posloupnost aproximací řešení

$$x_0, x_1, x_2, \ldots \xrightarrow{\text{konvergence}} x$$

jen pomocí násobení Az.

Příklad 2.1 (Jacobiho metoda pro $n = 3$)**.**

$$b = \begin{pmatrix} b_1 \\ b_2 \\ b_3 \end{pmatrix} \in \mathbf{C}^3, \quad x = \begin{pmatrix} \xi_1 \\ \xi_2 \\ \xi_3 \end{pmatrix} \in \mathbf{C}^3, \quad A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \in \mathbf{C}^{3\times 3}$$

Soustava rovnic pro $Ax = b$:

$$\begin{aligned} a_{11}\xi_1 + a_{12}\xi_2 + a_{13}\xi_3 = b_1 &\implies \xi_1 = \frac{1}{a_{11}}(-a_{12}\xi_2 - a_{13}\xi_3 + b_1) \\ a_{21}\xi_1 + a_{22}\xi_2 + a_{23}\xi_3 = b_2 &\implies \xi_2 = \frac{1}{a_{22}}(-a_{21}\xi_1 - a_{23}\xi_3 + b_2) \\ a_{31}\xi_1 + a_{32}\xi_2 + a_{33}\xi_3 = b_3 &\implies \xi_3 = \frac{1}{a_{33}}(-a_{31}\xi_1 - a_{32}\xi_2 + b_3) \end{aligned}$$

Volíme $x_0 = \begin{pmatrix} \xi_1^{(0)} \\ \xi_2^{(0)} \\ \xi_3^{(0)} \end{pmatrix}$ (počáteční aproximace) $\rightarrow$ dosadíme do pravé strany $\rightarrow$

$x_1 := \begin{pmatrix} \xi_1^{(1)} \\ \xi_2^{(1)} \\ \xi_3^{(1)} \end{pmatrix}$ je nová aproximace. Maticově máme

$$x_1 = \begin{bmatrix} \frac{1}{a_{11}} & 0 & 0 \\ 0 & \frac{1}{a_{22}} & 0 \\ 0 & 0 & \frac{1}{a_{33}} \end{bmatrix} \left(\begin{bmatrix} 0 & -a_{12} & -a_{13} \\ -a_{21} & 0 & -a_{23} \\ -a_{31} & -a_{32} & 0 \end{bmatrix} x_0 + b \right)$$

2.2.1 Jacobiho metoda

- Štěpení matice: $A = D - L - U$, kde D je diagonála, L striktně dolní trojúhelníková, U striktně horní trojúhelníková.



- **Předpoklad:** D je regulární ($a_{ii} \neq 0$).

Volíme počáteční aproximaci x_0 (typicky $x_0 \equiv 0$). Pak sekvence aproximací řešení $Ax = b$ Jacobiho metodou je tvaru:

$$x_k = D^{-1}(L + U)x_{k-1} + D^{-1}b, \quad k = 1, 2, \ldots$$

Algoritmus 1: Jacobiho metoda

Input: A, b, x_0, m (# iterací)
Output: x_m

```
for k = 1, ..., m do
    for i = 1, ..., n do
        ξ_i^(k) = (1/a_ii) ( − Σ_{j=1, j≠i}^{n} a_ij ξ_j^(k−1) + b_i )
    end
    // možno paralelizovat výpočet
end
```

Vlastnosti

- **Paměťové náklady** – uložení x_k, x_{k-1}.
- **Výpočetní náklady** – dominantní operace je jedno násobení (Az) v každé iteraci.

Důsledek 2.1 (Jacobiho iterace a residuum)**.** Jestliže D je regulární, potom Jacobiho iterace splňuje vztah

$$x_k = x_{k-1} + D^{-1}(b - Ax_{k-1}),$$

kde $r_{k-1} = b - Ax_{k-1}$ je residuum v iteraci $(k-1)$.

Důkaz.

$$\begin{aligned} x_k &= D^{-1}(L + U)x_{k-1} + D^{-1}b = D^{-1}(D - A)x_{k-1} + D^{-1}b \\ &= x_{k-1} - D^{-1}Ax_{k-1} + D^{-1}b = x_{k-1} + D^{-1}(b - Ax_{k-1}). \end{aligned}$$

□

Zastavovací kritérium V praxi neprovádíme fixní počet iterací, ale iterujeme, dokud se norma residua nesníží pod předem danou hranici (např. zvolenou jako $\text{TOL} = 10^{-4}\|b\|$).

$$\text{if } \|r_k\| < \text{TOL then STOP} \quad \text{(zvolená tolerance)}$$

2.2.2 Gauss-Seidelova metoda

Uvažujme Jacobiho metodu, kde použijeme spočtené hodnoty $\xi_j^{(k)}$ ihned, jakmile jsou k dispozici.

$\Longrightarrow$ řádek 3 Algoritmu 1 nahradíme předpisem

$$\xi_i^{(k)} := \frac{1}{a_{ii}} \left(- \sum_{j=1}^{i-1} a_{ij} \xi_j^{(k)} - \sum_{j=i+1}^{n} a_{ij} \xi_j^{(k-1)} + b_i \right)$$

$\Longrightarrow$ maticově platí:

$$x_k^{GS} := D^{-1} L x_k^{GS} + D^{-1} U x_{k-1}^{GS} + D^{-1} b, \quad k = 1, 2, \ldots$$

Algoritmus 2: Gauss-Seidelova metoda

Input: A, b, x_0, m (# iterací)
Output: x_m

```
1 for k = 1, ..., m do
2 |   for i = 1, ..., n do
3 |   |   ξ_i^(k) = (1/a_ii) ( − Σ_{j=1}^{i−1} a_ij ξ_j^(k) − Σ_{j=i+1}^{n} a_ij ξ_j^(k−1) + b_i )
4 |   end
5 end
```

Důsledek 2.2 (Gauss-Seidelova iterace a residuum)**.** Jestliže $(D - L)$ je regulární, pak Gauss-Seidelova iterace splňuje vztah

$$x_k^{GS} = x_{k-1}^{GS} + (D - L)^{-1}(b - A x_{k-1}^{GS}),$$

kde $r_{k-1}^{GS} = b - A x_{k-1}^{GS}$ je residuum v iteraci $(k - 1)$.

Důkaz.

$$D x_k^{GS} - L x_k^{GS} = U x_{k-1}^{GS} + b$$

$$\begin{aligned} (D - L) x_k^{GS} &= U x_{k-1}^{GS} + b = (D - L - A) x_{k-1}^{GS} + b \\ &= (D - L) x_{k-1}^{GS} + (b - A x_{k-1}^{GS}) \\ &\Longrightarrow x_k^{GS} = x_{k-1}^{GS} + (D - L)^{-1}(b - A x_{k-1}^{GS}) \end{aligned}$$

□

Vlastnosti

- **Paměť** – uložení jen 1 aproximace (přepisuji x_{k-1} na x_k).
- **Rychlost** – většinou rychlejší konvergence než Jacobiho metoda.
- **Sekvenční** – nelze paralelizovat.

Další metody (SOR, SSOR, ...)

SOR = Successive Over-Relaxation.

$$x_k^{SOR} := \omega \cdot \underbrace{[D^{-1}(Lx_k^{SOR} + Ux_{k-1}^{SOR} + b)]}_{\text{nová iterace GS}} + (1-\omega)x_{k-1}^{SOR}$$

kde $0 < \omega < 2$ je relaxační parametr. Pro $\omega = 1$ se SOR redukuje na GS.

2.3 Konvergence stacionárních metod

Obecné štěpení: $A = M - N$, kde M je regulární.

$$\begin{aligned} Ax &= b \\ Mx &= Nx + b \\ x &= M^{-1}Nx + M^{-1}b = x + M^{-1}(b - Ax) \end{aligned}$$

Stacionární (klasická) iterační metoda je tvaru:

$$x_k := (M^{-1}N)x_{k-1} + M^{-1}b = x_{k-1} + M^{-1}(b - Ax_{k-1}), \quad k = 1, 2, \ldots$$

kde $M^{-1}N$ je iterační matice (nezávisí na k) a $b - Ax_{k-1} = r_{k-1}$ je residuum. Speciálně:

- **Jacobi:** $M = D$, $N = L + U$
- **Gauss-Seidel:** $M = D - L$, $N = U$

Označme $\|e_k\| := x - x_k$ ... chyba aproximace v k-té iteraci.

VĚTA 2.1: *Konvergence stacionárních metod*

Nechť $x_k, k = 0, 1, 2, \ldots$ je posloupnost aproximací daná stacionární metodou. Pak pro chybu aproximace platí:

1. $e_k = (I - M^{-1}A)e_{k-1} = (I - M^{-1}A)^k e_0$
2. $\frac{\|e_k\|}{\|e_0\|} \leq \|(I - M^{-1}A)^k\| \leq \|I - M^{-1}A\|^k$

kde $\frac{\|e_k\|}{\|e_0\|}$ je relativní normovaná chyba aproximace.

Důkaz. Důkaz (1):

$$x_k = x_{k-1} + M^{-1}(Ax - Ax_{k-1}) = x_{k-1} + M^{-1}A(x - x_{k-1})$$

$$x - x_k = (x - x_{k-1}) - M^{-1}A(x - x_{k-1}) = (I - M^{-1}A)(x - x_{k-1})$$

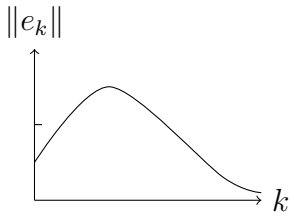
Tedy $e_k = (I - M^{-1}A)e_{k-1}$. Indukcí získáme $e_k = (I - M^{-1}A)^k e_0$.

Bod (2) plyne z bodu (1) a vlastností norem. □

Poznámka 2.2 (Alternativní zápis iterační matice)**.** Platí $(I - M^{-1}A) = M^{-1}N$. Tedy $e_k = (M^{-1}N)e_{k-1} = (M^{-1}N)^k e_0$.

Důsledek 2.3 (Postačující podmínka konvergence)**.** Nechť $\|M^{-1}N\| < 1$. Pak stacionární metoda konverguje $\forall x_0 \in \mathbf{C}^n$, tj. $\|x - x_k\| \xrightarrow{k\to\infty} 0$.

- **Přechodový jev** – na počátku může $\|e_k\|$ růst, dokázali jsme konvergenci jen v limitě.



- **Souvislost:** Banachova věta o pevném bodě:

$$e_k = \underbrace{(M^{-1}N)}_{\text{kontrahující zobrazení, je-li } \|M^{-1}N\|<1} e_{k-1}$$

DEFINICE 2.2: *Spektrální poloměr*

Nechť $A \in \mathbf{C}^{n\times n}$ a $\{\lambda_1, \ldots, \lambda_n\} = \mathrm{sp}(A)$ jsou vlastní čísla matice A. Pak číslo

$$\rho(A) := \max_{i=1,\ldots,n} |\lambda_i| = \max_{\lambda\in\mathrm{sp}(A)} |\lambda|$$

nazveme spektrální poloměr matice A.

LEMMA 2.2: *Vztah normy a spektrálního poloměru*

Pro každou maticovou normu $\|\cdot\|$ generovanou vektorovou normou platí, že $\forall A \in \mathbf{C}^{n\times n} : \rho(A) \le \|A\|$.

Důkaz. Nechť $\lambda \in \mathrm{sp}(A)$. Pak $\exists v \in \mathbf{C}^n, Av = \lambda v, \|v\| = 1$ (vlastní vektor). Proto

$$\|\lambda v\| = |\lambda| \cdot \|v\| = \|\lambda v\| = \|Av\| \le \|A\| \cdot \|v\| \implies |\lambda| \le \|A\|.$$

□

Poznámka 2.3. Lemma platí pro libovolnou maticovou normu generovanou vektorovou normou.

VĚTA 2.3: *Oldenburgova*

Nechť $B \in \mathbf{C}^{n\times n}$. Pak

$$\lim_{k\to\infty} B^k = 0 \iff \rho(B) < 1.$$

Důkaz. Dle Jordanovy věty lze psát $B = CJC^{-1}$, $J = \mathrm{diag}(J_1, \ldots, J_s)$, kde J_i jsou Jordanovy bloky. Pak $B^k = CJ^kC^{-1}$. Tvrzení tedy stačí dokázat pro každý Jordanův blok J_λ.

Dokazujeme tedy, že

$$J_\lambda^k = \begin{bmatrix} \lambda & 1 & & \\ & \lambda & \ddots & \\ & & \ddots & 1 \\ & & & \lambda \end{bmatrix}^k \xrightarrow{k\to\infty} 0 \iff |\lambda| < 1.$$

Pro $k > m$ (kde m je velikost bloku) platí

$$J_\lambda^k = \begin{bmatrix} \lambda^k & \binom{k}{1}\lambda^{k-1} & \dots & \binom{k}{m-1}\lambda^{k-m+1} \\ & \ddots & \ddots & \vdots \\ & & \ddots & \binom{k}{1}\lambda^{k-1} \\ 0 & & & \lambda^k \end{bmatrix} \xrightarrow{k\to\infty} 0 \iff |\lambda| < 1.$$

Zjevně $B^k \to 0 \iff |\lambda| < 1 \quad \forall \lambda \in \mathrm{sp}(A) \iff \rho(B) < 1.$ □

Věta 2.4:

Stacionární metoda konverguje $\forall x_0 \in \mathbf{C}^n \iff \rho(M^{-1}N) < 1.$

Důkaz. Protože $e_k = (M^{-1}N)^k e_0$, máme ihned $\|e_k\| \to 0 \iff \rho(M^{-1}N) < 1.$ □

- $\rho(M^{-1}N) < 1$ je nutná a postačující podmínka konvergence, ale nedá se obecně snadno ověřit.

2.4 Konvergence pro speciální matice

Definice 2.3: *Geršgorinův kruh*

Nechť $A \in \mathbf{C}^{n\times n}$. Kruh

$$D_i := \{z \in \mathbf{C} : |z - a_{ii}| \le \sum_{j=1, j\neq i}^{n} |a_{ij}|\}, \quad i = 1, \dots, n$$

nazveme i-tý *Geršgorinův kruh* matice A.

Věta 2.5: *Geršgorinova*

Nechť $A \in \mathbf{C}^{n\times n}$. Pak vlastní čísla A leží ve sjednocení jejích Geršgorinových kruhů, tj. $\mathrm{sp}(A) \subset \bigcup_{i=1}^n D_i$.

Důkaz. Nechť $\lambda \in \mathrm{sp}(A)$ libovolné. Pak $\exists v \in \mathbf{C}^n : Av = \lambda v, \|v\| \neq 0$. Nechť je v normován tak, že $v_i = 1$ (kde i je index složky s maximální absolutní hodnotou) a $|v_j| \le 1$ pro $j \neq i$.

Uvažujme i-tou složku vektoru $Av = \lambda v$:

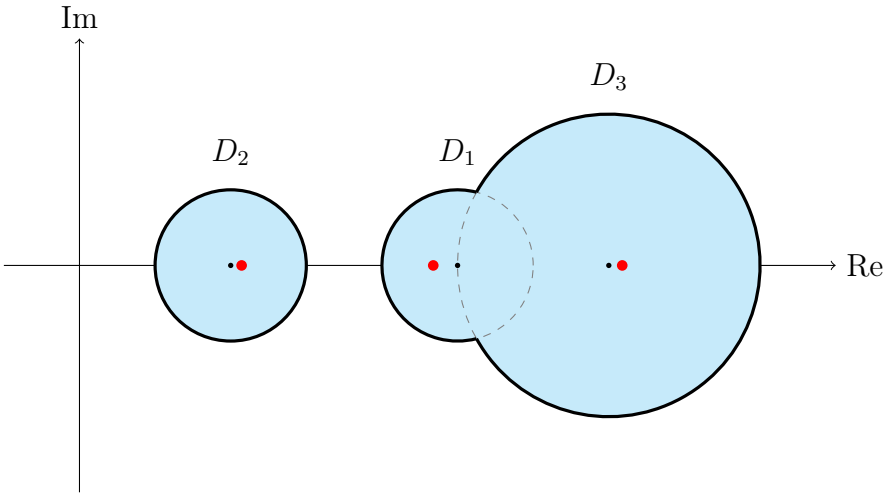
$$\sum_{j=1}^{n} a_{ij} v_j = \lambda v_i = \lambda \cdot 1 = \lambda$$

$$\sum_{j=1,j\neq i}^{n} a_{ij}v_j + a_{ii} = \lambda \implies \lambda - a_{ii} = \sum_{j=1,j\neq i}^{n} a_{ij}v_j$$

$$|\lambda - a_{ii}| \leq \sum_{j=1,j\neq i}^{n} |a_{ij}| \cdot |v_j| \leq \sum_{j=1,j\neq i}^{n} |a_{ij}| \cdot 1$$

Tedy $\lambda \in D_i$. □

Příklad 2.2. $A = \begin{bmatrix} 5 & 0 & -1 \\ 1 & 2 & 0 \\ 2 & 0 & 7 \end{bmatrix} \implies$ všechna vl.č. A leží ve sjednocení kruhů



Kruhy lze použít pro odhad polohy vlastních čísel, odhad $\rho(A)$, regularity A, ...

Definice 2.4: *Ostře diagonálně dominantní matice (ODD)*

Matici $A \in \mathbf{C}^{n\times n}$ nazveme ostře diagonálně dominantní (ODD), pokud

$$\sum_{j=1,j\neq i}^{n} |a_{ij}| < |a_{ii}|, \quad i = 1, \ldots, n.$$

- ODD matice vznikají v řadě aplikací.
- Dle Geršgorinovy věty jsou regulární ($0 \notin D_i$ pro žádné i, protože vzdálenost středu a_{ii} od nuly je větší než poloměr kruhu).

Věta 2.6: *Konvergence ODD*

Je-li $A \in \mathbf{C}^{n\times n}$ ODD, pak x_k^J (Jacobi) a x_k^{GS} (Gauss-Seidel) konvergují k x pro $\forall x_0 \in \mathbf{C}^n$.

Důkaz. Důkaz pro Jacobiho metodu: Chci ukázat, že $\rho(D^{-1}(L+U)) < 1$. Platí

$$\|D^{-1}(L+U)\|_\infty = \max_{1\leq i\leq n} \sum_{j=1,j\neq i}^{n} \frac{|a_{ij}|}{|a_{ii}|} < 1 \quad \text{(z definice ODD)}.$$

Protože $\|\cdot\|_\infty$ je generovaná maximovou normou pro vektory, tak $\rho(B) \leq \|B\|_\infty$ pro všechny matice B. Z toho plyne, že $\rho(D^{-1}(L+U)) \leq \|D^{-1}(L+U)\|_\infty < 1$. □

Poznámka 2.4 (Konvergence GS pro HPD matici)**.** Lze dokázat, že je-li A HPD (Hermitovská pozitivně definitní), pak GS metoda konverguje $\forall x_0 \in \mathbf{C}^n$. (Překvapivě toto neplatí pro Jacobiho metodu).

2.5 Iterační zpřesnění (čtení navíc)

- Slouží k zpřesnění dané aproximace $\tilde{x}$ řešení soustavy $Ax = b$.
- $\tilde{x}$ – dáno z měření, spočteno GE, stacionární metodou, ...

Idea

$$r := b - A\tilde{x} \quad \ldots \text{reziduum}$$
$$e := x - \tilde{x} \quad \ldots \text{chyba řešení/aproximace}$$

Platí

$$A \cdot e = r \quad \ldots \text{residuová rovnice}$$

Důkaz: $Ae = A(x - \tilde{x}) = Ax - A\tilde{x} = b - A\tilde{x} = r$.

Algoritmus 3: Iterační zpřesnění

Input: $A, b, \tilde{x}$
Output: $\hat{x}$ – nová aproximace

1 $r := b - A\tilde{x}$
2 řeš residuovou rovnici $Ae = r$ (pomocí GE, stacionární metodou, ...)
3 $\hat{x} := \tilde{x} + e$

Poznámka 2.5 (Úspora výpočetního času při iteračním zpřesnění)**.** Počítáme-li $\tilde{x}$ i $\hat{x}$ pomocí GE, stačí provádět GE jednou (odpovídá výpočtu $LU = A$).

3 Ortogonální transformace

Motivace Při řešení úloh je výhodné problém transformovat tak, aby jeho řešení bylo v nějakém smyslu jednodušší.

Příklad 3.1 (Úpravy soustavy pomocí GE)**.**

$$Ax = b \xrightarrow[\text{implicitně } A=LU]{} Ux = L^{-1}b$$

GE uplatňuje implicitně transformaci L^{-1} na matici A a vektor b. Výsledná úloha s horní trojúhelníkovou maticí je snáze řešitelná. Jaké transformace je ale vhodné používat?

3.1 Citlivost řešení lineárních algebraických rovnic na chyby v pravé straně

Mějme úlohu $Ax = b$ s přesnými daty. V praxi data obsahují chyby (měření, modelu, ...), proto uvažujeme, že b je kontaminováno malou perturbací Δb. Jak se může změnit řešení úlohy?

Označme perturbovanou rovnici:

$$A(x + \underbrace{\Delta x}_{\substack{\text{chyba}\\ \text{řešení}}}) = b + \underbrace{\Delta b}_{\substack{\text{perturbace}\\ \text{měření}}}$$

Věta 3.1: *O podmíněnosti řešení soustavy*

Platí

$$\frac{\|\Delta x\|}{\|x\|} \leq \frac{\|\Delta b\|}{\|b\|} \cdot \varkappa(A),$$

kde $\varkappa(A) \equiv \|A\| \cdot \|A^{-1}\|$ je **číslo podmíněnosti** matice A.

Důkaz. Z rovnice $A(x + \Delta x) = b + \Delta b$ a $Ax = b$ plyne $A\Delta x = \Delta b$, tedy $\Delta x = A^{-1}\Delta b$. Odtud máme nerovnost

$$\|\Delta x\| \leq \|A^{-1}\|\|\Delta b\|.$$

Zároveň z $Ax = b$ plyne $\|b\| \leq \|A\|\|x\|$, což dává $\frac{1}{\|x\|} \leq \frac{\|A\|}{\|b\|}$.

Vynásobením těchto nerovností dostáváme

$$\frac{\|\Delta x\|}{\|x\|} \leq \|A^{-1}\|\|\Delta b\| \cdot \frac{\|A\|}{\|b\|} = \underbrace{\|A\| \cdot \|A^{-1}\|}_{\varkappa(A)} \frac{\|\Delta b\|}{\|b\|}.$$

□

Vlastnosti $\varkappa(A)$:

1. $\varkappa(A) \geq 1 \quad \forall A \in \mathbf{C}^{n\times n}$.
2. $\varkappa(A) = 1 \iff A$ je násobek unitární matice. (Důkaz jako cvičení).

Poznámka 3.1. Při velkém $\varkappa(A)$ může i malá perturbace v b způsobit velkou chybu v x.

Příklad 3.2 (GE)**.** Pro Gaussovu eliminaci platí

$$\varkappa(U) = \varkappa(L^{-1}A) \neq \varkappa(A).$$

Číslo podmíněnosti může být po transformaci větší ($\varkappa(U)$ může být větší než $\varkappa(A)$).

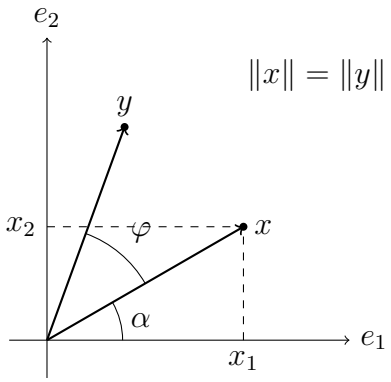
Požadavek na transformace: Chceme transformace, které nemění vzdálenosti v prostoru, normy a čísla podmíněnosti matic.

$$\text{ISOMETRIE} \equiv \text{UNITÁRNÍ (v } \mathbf{C}^n) \;\equiv\; \text{ORTOGONÁLNÍ (v } \mathbf{R}^n)$$

3.2 Givensovy rotace v R

V $\mathbf{R}^2$

Mějme $x = (x_1, x_2)^T$. Chceme otočit x o úhel φ na $y = (y_1, y_2)^T$, kde $\|x\| = \|y\|$.



Obrázek 2: Rotace vektoru v rovině

Pomocí polárních souřadnic:

$$x = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \|x\| \begin{pmatrix} \cos\alpha \\ \sin\alpha \end{pmatrix}$$

Po otočení o úhel φ:

$$y = \begin{pmatrix} y_1 \\ y_2 \end{pmatrix} = \|y\| \begin{pmatrix} \cos(\varphi+\alpha) \\ \sin(\varphi+\alpha) \end{pmatrix} = \|x\| \begin{pmatrix} \cos\varphi\cos\alpha - \sin\varphi\sin\alpha \\ \sin\varphi\cos\alpha + \cos\varphi\sin\alpha \end{pmatrix}$$

Využitím vztahů pro x_1, x_2 ($x_1 = \|x\|\cos\alpha, x_2 = \|x\|\sin\alpha$):

$$y = \underbrace{\begin{pmatrix} \cos\varphi & -\sin\varphi \\ \sin\varphi & \cos\varphi \end{pmatrix}}_{G(\varphi)} \underbrace{\begin{pmatrix} x_1 \\ x_2 \end{pmatrix}}_{x} \equiv G(\varphi)x$$

Definice 3.1: *Elementární Givensova rotace*

Matici $G(\varphi)$ realizující otočení (rotaci) vektoru o úhel φ nazveme **elementární Givensovou rotací v $\mathbf{R}^2$**.

V $\mathbf{R}^n$

Pro $x \in \mathbf{R}^n$: Otočení provedeme jen v rovině určené osami e_i, e_j (i-tá a j-tá pozice). Ostatní složky se nemění.

Definice 3.2: *Givensova rotace v $\mathbf{R}^n$*

Matice elementární Givensovy rotace, která otočí libovolný vektor $x \in \mathbf{R}^n$ v rovině $\{e_i, e_j\}$ $(i < j)$ o úhel φ směrem od e_i k e_j, je tvaru

$$G_{i,j}(\varphi) = \begin{pmatrix} 1 & \cdots & 0 & & & & \\ \vdots & & \vdots & & & & \\ 0 & \cdots & c & \cdots & s & & \\ & & \vdots & & \vdots & & \\ & & -s & \cdots & c & \cdots & 0 \\ & & & & \vdots & & \vdots \\ & & & & 0 & \cdots & 1 \end{pmatrix} \in \mathbf{R}^{n \times n}.$$

Zde $c = \cos\varphi$ je na pozicích $(i,i), (j,j)$ (zbytek diagonálních prvků je rovno 1), $s = \sin\varphi$ je na pozici (i,j) a $-s$ je na pozici (j,i) (zbytek mimodiagonálních prvků je rovno 0).

Lemma 3.2: *Vlastnosti Givensovy rotace*

Pro elementární Givensovu rotaci $G_{i,j}(\varphi)$ v $\mathbf{R}^n$ platí:

1. Inverze Givensovy rotace je Givensova rotace o úhel $-\varphi$, což je zároveň transpozice původní Givensovy rotace:

   $$G_{i,j}(\varphi)G_{i,j}(-\varphi) = G_{i,j}(\varphi)G_{i,j}(\varphi)^T = I.$$

   Matice elementární Givensovy rotace je ortogonální a tedy isometrie.

2. Zjevně $\det(G_{i,j}(\varphi)) = 1$, matice tedy zachovává orientaci vektorů.

3. Násobení $G_{i,j}(\varphi)x$ mění jen 2 složky vektoru x. Součin s vektorem $G_{i,j}(\varphi)x$ má proto časovou složitost $O(1)$, součin s čtvercovou maticí $G_{i,j}(\varphi)A$ má složitost $O(n)$.

Důkaz. 1. **Ortogonalita:** Protože $G_{i,j}(\varphi)$ se liší od identity pouze ve 2x2 bloku odpovídajícím indexům (i,j), který má tvar

$$\begin{pmatrix} \cos\varphi & \sin\varphi \\ -\sin\varphi & \cos\varphi \end{pmatrix},$$

stačí ukázat, že pro tento blok platí

$$\begin{pmatrix} \cos\varphi & \sin\varphi \\ -\sin\varphi & \cos\varphi \end{pmatrix} \begin{pmatrix} \cos\varphi & -\sin\varphi \\ \sin\varphi & \cos\varphi \end{pmatrix} = \begin{pmatrix} \cos^2\varphi + \sin^2\varphi & 0 \\ 0 & \cos^2\varphi + \sin^2\varphi \end{pmatrix} = I_2.$$

Protože blok odpovídající indexům (i,j) se zachovává při transpozici, $G_{i,j}(\varphi)^{-1} = G_{i,j}(\varphi)^T$ vyplývá z toho, že $G(\varphi)^{-1} = G(\varphi)^T$ (dosazením).

2. **Determinant:** Determinant celé matice je dán determinatem 2x2 bloku, tedy

$$\det(G_{i,j}(\varphi)) = \det(G(\varphi)) = \det \begin{pmatrix} \cos\varphi & \sin\varphi \\ -\sin\varphi & \cos\varphi \end{pmatrix} = \cos^2\varphi + \sin^2\varphi = 1.$$

Zbývající diagonální prvky jsou jedničky, takže determinant celé matice je 1.

3. **Složitost:** Protože $G_{i,j}(\varphi)$ se liší od identity pouze ve složkách i a j, ovlivní při násobení $G_{i,j}(\varphi)x$ pouze x_i a x_j, zbytek zůstává nezměněn. Proto má tato operace konstantní časovou složitost $O(1)$ vzhledem k n. Při násobení maticí tuto operaci provedeme pro každý její sloupeček, což zvyšuje složitost na $O(n)$.

□

Použití k nulování prvků x

- **V $\mathbf{R}^2$:** Chceme $x = (x_1, x_2)^T \xrightarrow{G(\varphi)} y = (\|x\|, 0)^T$. Tedy chceme otočit vektor na osu e_1 (kladnou část). To odpovídá volbě $\varphi = -\alpha$.

$$G(\varphi) = \begin{pmatrix} \cos\alpha & \sin\alpha \\ -\sin\alpha & \cos\alpha \end{pmatrix} = \frac{1}{\|x\|} \begin{pmatrix} x_1 & x_2 \\ -x_2 & x_1 \end{pmatrix}.$$

  Potom $G(\varphi)x = \begin{pmatrix} \|x\| \\ 0 \end{pmatrix}$.

- **V $\mathbf{R}^n$:** Obecně potřebujeme $(n-1)$ elementárních rotací k vynulování prvků vektoru, kromě prvního (transformace na násobek e_1). Postupně nulujeme $x_n, x_{n-1}, \ldots, x_2$ pomocí rotací s předchozím prvkem (nebo s x_1).

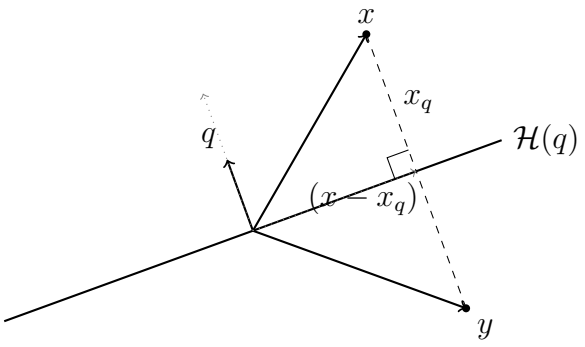
$$x = \begin{pmatrix} x_1 \\ \vdots \\ x_{n-1} \\ x_n \end{pmatrix} \xrightarrow{G_{n-1,n}} \begin{pmatrix} x_1 \\ \vdots \\ \tilde{x}_{n-1} \\ 0 \end{pmatrix} \xrightarrow{G_{n-2,n-1}} \ldots \xrightarrow{G_{1,2}} \begin{pmatrix} \|x\| \\ 0 \\ \vdots \\ 0 \end{pmatrix}.$$

  Složená transformace $\Gamma = G_{1,2} \ldots G_{n-1,n}$ je ortogonální.

Poznámka 3.2 (Vlastnosti při nulování poddiagonály Givensovými rotacemi)**.** Γ není určena jednoznačně, můžeme nulovat prvky v jiném pořadí.
Paměť: Γ nekonstruujeme, ukládáme jen c_i, s_i.
Výpočetní náklady: $O(n)$.

3.3 Householderovy reflexe v $\mathbf{R}^n$

Mějme $x, y \in \mathbf{R}^n$, $\|x\| = \|y\|$. Chceme najít nadrovinu zrcadlení, která zrcadlí x na y.



Obrázek 3: Householderova reflexe podle nadroviny $\mathcal{H}(q)$ dané normálovým vektorem $q \in \mathbf{R}^n$

Definice 3.3: *Nadrovina zrcadlení*

Necht $q \in \mathbf{R}^n, \|q\| = 1$. Množinu

$$\mathcal{H}(q) := \{z \in \mathbf{R}^n : z \perp q\}$$

nazveme **nadrovinou zrcadlení** danou normálovým vektorem q.

Nadrovina zrcadlení převádějící x na y má normálový vektor ve směru $x - y$:

$$q := \frac{x - y}{\|x - y\|}.$$

Uvažujme rozklad vektoru x do směru q a do nadroviny $\mathcal{H}(q)$:

$$x = \underbrace{(q^T x)q}_{x_q \| q} + \underbrace{(x - (q^T x)q)}_{(x - x_q) \perp q}.$$

Zrcadlení y vznikne negací komponenty x_q:

$$y = (x - x_q) - x_q = x - 2x_q = x - 2(q^T x)q = (I - 2qq^T)x.$$

Definice 3.4: *Householderova reflexe v* $\mathbf{R}^n$

Nechť $q \in \mathbf{R}^n, \|q\| = 1$. Pak matici

$$H(q) = I - 2qq^T \in \mathbf{R}^{n \times n}$$

nazveme **maticí Householderovy reflexe** (zrcadlení) vzhledem k $\mathcal{H}(q)$.

Lemma 3.3: *Vlastnosti Householderovy reflexe*

Pro $H(q)$, matici Householderovy reflexe vzhledem k nadrovině $\mathcal{H}(q)$, platí:

1. $H(q)$ je symetrická.
2. $\det(H(q)) = -1$, reflexe tedy mění orientaci vektorů.
3. $H(q)^{-1} = H(q)$ (reflexe reflexe je identita).
4. $H(q)$ je ortogonální a tedy isometrie.

Důkaz. Označme $P = qq^T$ matici projekce na $\operatorname{span}\{q\}$. Householderovu reflexi zapíšeme $H(q) = I - 2P$.

1. **Symetrie:** Matice P je symetrická: $P^T = (qq^T)^T = (q^T)^T q^T = qq^T = P$. Proto
$$H(q)^T = (I - 2P)^T = I - 2P^T = I - 2P = H(q).$$

2. **Determinant:** Platí $Pq = qq^T q = q\|q\|^2 = q$, tedy q je vlastní vektor P s vlastním číslem 1. Pro $v \perp q$ platí $Pv = qq^T v = 0$, tedy $\ker(P) = \operatorname{span}\{q\}^\perp$.

   Vektor q je vlastní vektor $H(q)$ s vlastním číslem -1:
$$H(q)q = (I - 2P)q = q - 2Pq = q - 2q = -q.$$

Každý vektor $v \perp q$ je vlastní vektor s vlastním číslem 1:

$$H(q)v = (I - 2P)v = v - 2Pv = v.$$

Matice $H(q)$ má tedy vlastní čísla -1 (jednonásobné) a 1 ($(n-1)$-násobné). Determinant je součin vlastních čísel:

$$\det(H(q)) = (-1) \cdot 1^{n-1} = -1.$$

3. **Self inverse:** Matice P je idempotentní: $P^2 = qq^Tqq^T = q\|q\|^2q^T = qq^T = P$. Proto

$$H(q)^2 = (I - 2P)^2 = I - 4P + 4P^2 = I - 4P + 4P = I.$$

Tedy $H(q)^{-1} = H(q)$.

4. **Ortogonalita:** Plyne z předchozích bodů. $H(q)^TH(q) = H(q)H(q) = I$.

□

Použití k nulování prvků x

Chceme $x \xrightarrow{H(q)} y = (\pm\|x\|e_1) = (\pm\|x\|, 0, \ldots, 0)^T$. Položíme

$$q := \frac{x - y}{\|x - y\|}, \quad \text{kde } y = \pm\|x\|e_1.$$

Znaménko volíme tak, aby v případě dominantní první složky vektoru x nedošlo k odečtení blízkých čísel ve jmenovateli (ztráta přesnosti, cancellation error). Tedy

$$y_1 = -\operatorname{sgn}(x_1)\|x\|.$$

(Například pokud $x_1 > 0$, volíme $y_1 = -\|x\|$, aby $x_1 - y_1$ bylo sčítání).

To vede na volbu:

$$q := \frac{x + \operatorname{sgn}(x_1)\|x\|e_1}{\|x + \operatorname{sgn}(x_1)\|x\|e_1\|}.$$

Poznámka 3.3 (Výpočetní náročnost Householderovy reflexe)**.** Pro implementaci Householderovy reflexe ukládáme pouze vektor q, nikoliv celou matici $H(q)$. Přímé násobení vektoru x transformací $H(q)$ tedy efektivně realizujeme dle vzorce

$$H(q)x = x - 2q(q^Tx),$$

kde q^Tx je skalární součin (tedy jen číslo). Celková výpočetní složitost je $O(n)$. Tato úspora paměti a výpočetního času je zásadní pro praktické použití Householderových transformací v numerických výpočtech.

Zobecnění OG transformací do $\mathbf{C}^n$: Lze, ale není zcela přímočaré.

3.4 Zpětná stabilita

(Čtení navíc)

- Analýza je těžká (Wilkinson, Turing).
- Lze dokázat, že násobení unitární maticí je zpětně stabilní.

Věta 3.4: *Zpětná stabilita unitárních transformací*

Nechť $A \in \mathbf{C}^{n\times n}$ a $U \in \mathbf{C}^{n\times n}$ je matice elementární Givensovy rotace nebo Householderovy reflexe. Pak existuje $E \in \mathbf{C}^{n\times n}$ taková, že

$$\mathrm{fl}(UA) = U(A+E),$$

kde

$$\frac{\|E\|}{\|A\|} \leq \gamma \cdot n^2 \cdot \epsilon_{\text{mach}} + O(\epsilon_{\text{mach}}^2).$$

Vysvětlivky:

- $\mathrm{fl}(\cdot)$ značí výsledek v aritmetice s plovoucí řádovou čárkou (FPA).
- E reprezentuje perturbaci (chybu).
- γ je konstanta (malá).
- ϵ_{mach} je strojová přesnost.

Zjednodušeně:

$$\|E\| \approx \|A\| \cdot \epsilon_{\text{mach}}.$$

Chyba v FPA je úměrná $\|A\| \cdot \epsilon_{\text{mach}}$.

$\Longrightarrow$ Vyřešili jsme úlohu blízkou původní (zpětná stabilita).

3.5 QR rozklad

Rozklady realizované ortogonálními transformacemi lze spočítat zpětně stabilně, což znamená, že nezhoršují numerické vlastnosti matic (úloh).

Definice 3.5: *QR rozklad*

Nechť $A \in \mathbf{C}^{n\times m}$. Pak **QR rozklad** matice A je rozklad tvaru

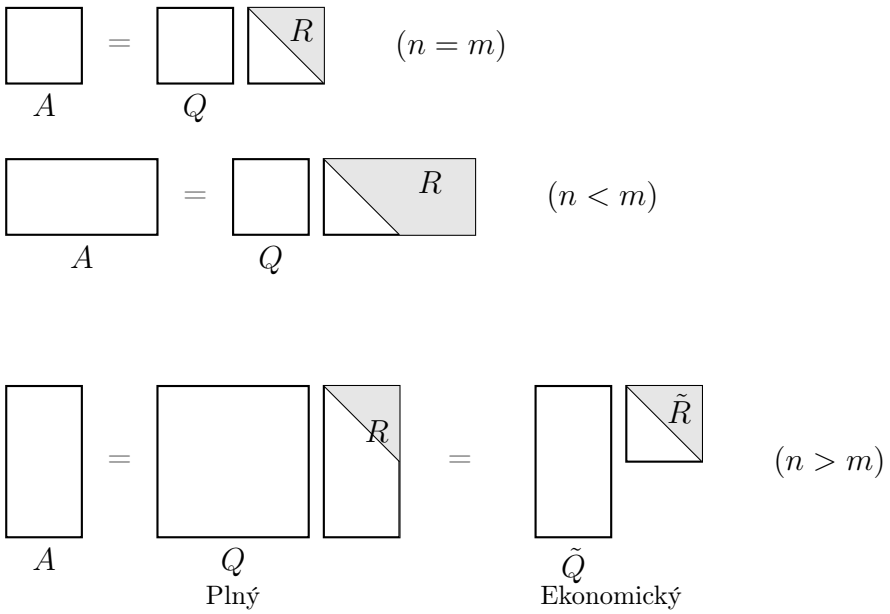
$$A = Q \cdot R,$$

kde $Q \in \mathbf{C}^{n\times n}$ je unitární a $R \in \mathbf{C}^{n\times m}$ je horní trojúhelníková. Pro hodnost platí $\mathrm{rank}(A) = \mathrm{rank}(R)$.

Tvary podle rozměrů:

- $n = m$: Čtvercové matice A, Q, R.

- $n < m$: $Q \in \mathbf{C}^{n\times n}$ je menšího rozměru.
- $n > m$: Zde rozlišujeme:
  - **Plný QR rozklad:** $A = QR$, kde Q je $n \times n$ (unitární), R je $n \times m$ (horní trojúhelníková s nulovým blokem dole).
  - **Ekonomický (redukovaný) QR rozklad:** $A = \tilde{Q}\tilde{R}$, kde $\tilde{Q}$ je $n \times m$ (má ortonormální sloupce), $\tilde{R}$ je $m \times m$ (čtvercová horní trojúhelníková).



Obrázek 4: Tvary matic při QR rozkladu pro různé dimenze

Poznámka 3.4 (Nejednoznačnost QR rozkladu)**.**

- QR rozklad **není jednoznačný**.
  Příklad: $A = QR = (QD^*)(DR) = \hat{Q}\hat{R}$ pro $D^*D = I$ (kde D je např. diagonální unitární matice).
- Pokud $\operatorname{rank}(A) = m$ (plná sloupcová hodnost), pak existuje **právě jedna** $\tilde{R} \in \mathbf{C}^{m\times m}$ (v ekonomickém rozkladu), která má reálné kladné prvky na diagonále.

Výpočet pomocí OG transformací

BÚNO: $n \geq m$.

Matici A postupně násobíme zleva ortogonálními (unitárními) transformacemi (Householderovými reflexemi nebo Givensovými rotacemi) tak, abychom nulovali prvky pod diagonálou v prvním sloupci.

$$A = \begin{bmatrix} | & & | \\ a_1 & \dots & a_m \\ | & & | \end{bmatrix} \xrightarrow[\text{rotující } a_1 \text{ na } \|a_1\|e_1]{\Gamma_1 \in \mathbf{C}^{n\times n}} \Gamma_1 A \equiv A^{(1)} = \begin{bmatrix} \|a_1\| & \times & \dots & \times \\ 0 & | & & | \\ \vdots & a_2^{(1)} & \dots & a_m^{(1)} \\ 0 & | & & | \end{bmatrix}$$

Zde Γ_1 je složená Givensova rotace. Analogicky lze použít Householderova reflexe H_1 reflektující a_1 na $\|a_1\|e_1$.

V dalším kroku pracujeme s podmaticí:

$$\Gamma_2 = \begin{bmatrix} 1 & 0\dots 0 \\ 0 & \tilde{\Gamma}_2 \end{bmatrix}, \quad \text{kde } \tilde{\Gamma}_2 \text{ rotuje } a_2^{(1)} \text{ na } \|a_2^{(1)}\|e_1 \in \mathbf{C}^{n-1}.$$

Analogicky lze použít $H_2 = \begin{bmatrix} 1 & 0 \\ 0 & \tilde{H}_2 \end{bmatrix}$. Pak

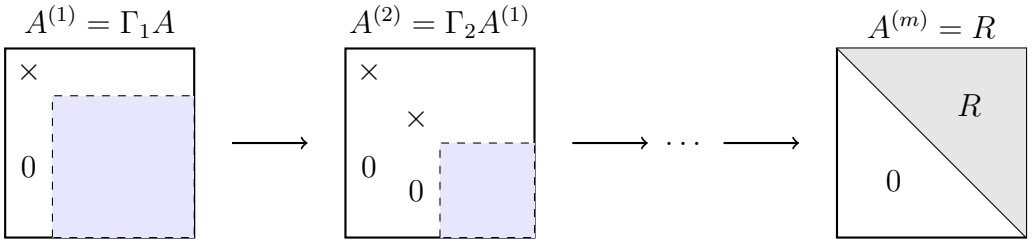
$$\Gamma_2\Gamma_1 A \equiv A^{(2)} = \begin{bmatrix} \|a_1\| & \times & \times\cdots\times \\ 0 & \|a_2^{(1)}\| & \times\cdots\times \\ 0 & 0 & \times\cdots\times \\ 0 & 0 & \times\cdots\times \end{bmatrix}.$$

Tímto postupem po m krocích (resp. $m-1$ pro A čtvercovou) získáme horní trojúhelníkovou matici R, kde

$$\Gamma_m \dots \Gamma_1 A \equiv A^{(m)} = \begin{bmatrix} \tilde{R} \in \mathbf{C}^{m\times m} \\ \\ \mathbf{0} \in \mathbf{C}^{(n-m)\times m} \end{bmatrix} \dots \text{plný QR rozklad.}$$

Platí:

$$\Gamma_k \dots \Gamma_1 A = R \implies A = (\Gamma_k \dots \Gamma_1)^{-1} R = \underbrace{\Gamma_1^* \dots \Gamma_k^*}_{Q} R.$$



Obrázek 5: Postup QR rozkladu pomocí ortogonálních transformací

Vlastnosti a implementace:

- **Výpočetní náklady:** Při efektivní transformaci A na R (bez explicitního sestavení Q), pro $n = m$:
  - Pomocí Householderových reflexí: cca $\frac{4}{3}n^3$.
  - Pomocí Givensových rotací: cca $2n^3$.
- **Paměť:** Ukládáme jen definující vektory q_i pro H_i nebo parametry c_i, s_i pro Γ_i.

Využití k řešení soustav rovnic

Úloha: $Ax = b$, $A \in \mathbf{C}^{n\times n}$. Pokud známe QR rozklad: $A = QR \implies Rx = Q^*b$.

Postup

1. Aplikujeme OG transformace na rozšířenou matici $[A \mid b]$ zleva:

$$[A \mid b] \xrightarrow{\text{OG transf.}} [R \mid \underbrace{Q^* b}_{\tilde{b}}].$$

   Vektor $\tilde{b}$ vznikne implicitně aplikací transformací, Q nesestavujeme, neukládáme.

2. Zpětná substituce: Vyřešíme $Rx = \tilde{b}$ (soustava s horní trojúhelníkovou maticí).

Srovnání s Gaussovou eliminací (GE)

- QR rozklad realizovaný pomocí OG transformací je **zpětně stabilní** (na rozdíl od GE, která může být nestabilní).
- Výpočetní náklady jsou cca 2× větší než u GE:

$$GE : \frac{2}{3}n^3, \quad QR \text{ (Householder)} : \frac{4}{3}n^3.$$

Další využití: Řešení lineárních aproximačních úloh (problém nejmenších čtverců) – bude v následující kapitole.

3.6 Gram-Schmidtova ortogonalizace

BÚNO: $n \geq m, \operatorname{rank}(A) = m$.

Mějme matici $A = [a_1, \ldots, a_m]$. Hledáme $Q = [q_1, \ldots, q_m]$ s ortonormálními sloupci tak, aby

$$\operatorname{span}\{a_1, \ldots, a_k\} = \operatorname{span}\{q_1, \ldots, q_k\}, \quad k = 1, \ldots, m.$$

Označme $\mathcal{S}_k = \operatorname{span}\{q_1, \ldots, q_k\}$.

Klasický Gram-Schmidtův (CGS) proces

Postupně konstruujeme q_k z a_k ortogonalizací vůči předchozím $q_1, \ldots, q_{k-1}$.

1. $k = 1$:

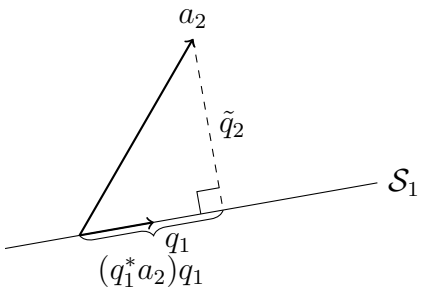
$$q_1 := \frac{a_1}{r_{11}}, \quad r_{11} := \|a_1\|.$$

2. $k = 2$: Chceme odečíst průmět a_2 do směru q_1.

$$\tilde{q}_2 := a_2 - \underbrace{(q_1^* a_2)}_{r_{12} \in \mathbf{C}} q_1 = a_2 - (q_1 q_1^*) a_2 = (I - q_1 q_1^*) a_2.$$

   Pozorování:

   - $(q_1 q_1^*)$ je matice realizující **ortogonální projekci** vektoru na $\operatorname{span}\{q_1\} = \mathcal{S}_1$.
   - $(I - q_1 q_1^*)$ je matice realizující **ortogonální projekci** vektoru na $\mathcal{S}_1^\perp$ (doplněk).



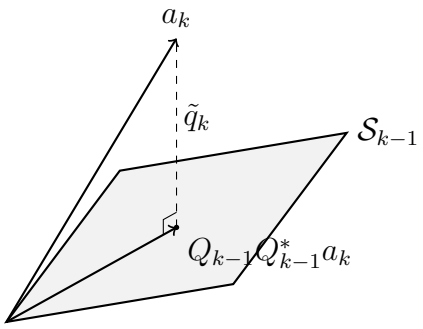
Obrázek 6: Projekce a_2 na $\mathcal{S}_1 = \text{span}\{q_1\}$.

Normalizace:

$$q_2 := \frac{\tilde{q}_2}{r_{22}}, \quad r_{22} := \|\tilde{q}_2\|.$$

3. **Obecně pro** $k \in \{2, \ldots, m\}$**:**

$$\tilde{q}_k := a_k - \sum_{i=1}^{k-1} \underbrace{(q_i^* a_k)}_{r_{ik} \in \mathbf{C}} q_i = a_k - \sum_{i=1}^{k-1} (q_i q_i^*) a_k = a_k - \underbrace{Q_{k-1} Q_{k-1}^*}_{\text{proj. na } \mathcal{S}_{k-1}} a_k = (I - Q_{k-1} Q_{k-1}^*) a_k.$$



Obrázek 7: Projekce a_k na $\mathcal{S}_{k-1}$ a získání $\tilde{q}_k$.

Zde značíme $Q_{k-1} = [q_1, \ldots, q_{k-1}]$. Matice $Q_{k-1}Q_{k-1}^*$ je matice ortogonální projekce na $\mathcal{S}_{k-1}$. Matice $(I - Q_{k-1}Q_{k-1}^*)$ je matice ortogonální projekce na $\mathcal{S}_{k-1}^{\perp}$.

Výsledek:

$$A = QR, \quad Q = [q_1, \ldots, q_m] \in \mathbf{C}^{n \times m}, \quad R = \begin{pmatrix} r_{11} & r_{12} & \ldots & r_{1m} \\ & r_{22} & & \\ 0 & & \ddots & \vdots \\ & & & r_{mm} \end{pmatrix} \in \mathbf{C}^{m \times m}.$$

Získáme tedy **ekonomický QR rozklad**.

LEMMA 3.5: *Rozklad ortogonálního projektoru*

Nechť $Q_k \in \mathbf{C}^{n \times k}$ má ortonormální sloupce. Pak ortogonální projektor $(I - Q_k Q_k^*)$ splňuje

$$I - Q_k Q_k^* = (I - q_k q_k^*)(I - q_{k-1} q_{k-1}^*) \ldots (I - q_1 q_1^*).$$

Důkaz. Triviálně roznásobením a využitím ortogonality $q_i^* q_j = 0$ pro $i \neq j$ a $q_i^* q_i = 1$. □

Důsledek:

$$\tilde{q}_k = (I - q_{k-1}q_{k-1}^*)\dots(I - q_1 q_1^*)a_k.$$

To znamená, že můžeme ekvivalentně provádět sekvenční ortogonalizaci vektoru a_k proti jednomu vektoru g_i po druhém. Toto vede na **Modifikovaný Gram-Schmidtův (MGS) proces**.

Algoritmus CGS vs. MGS

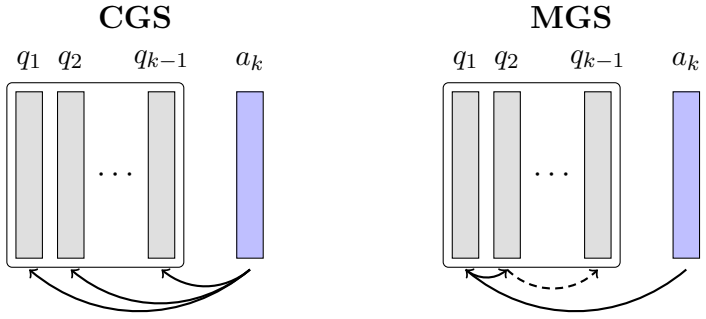
Algoritmus 4: CGS (Classical Gram-Schmidt)

```
for k = 1, ..., m do
    z = a_k;
    for i = 1, ..., k − 1 do
        r_ik = q_i^* a_k;
    end
    for i = 1, ..., k − 1 do
        z = z − r_ik q_i;
    end
    r_kk = ‖z‖;
    q_k = z/r_kk;
end
```

Spojením vnitřních for cyklů dostaneme MGS.

Algoritmus 5: MGS (Modified Gram-Schmidt)

```
q_1 = a_1/r_11, r_11 = ‖a_1‖;
for k = 2, ..., m do
    z = a_k;
    for i = 1, ..., k − 1 do
        r_ik = q_i^* z;
        z = z − r_ik q_i;
    end
    r_kk = ‖z‖;
    q_k = z/r_kk;
end
```



Obrázek 8: Srovnání CGS a MGS. V CGS počítáme všechny koeficienty proti původnímu vektoru a_k, pak ortogonalizujeme. V MGS ortogonalizujeme postupně.

Srovnání CGS a MGS

- CGS a MGS jsou **matematicky ekvivalentní** (pouze různé uspořádání operací), ale **neplatí to v FPA** (aritmetice s plovoucí řádovou čárkou).
- **Výpočetní náklady:** Stejné, $2m^3$ (pro $n = m$).
- **Paralelizace:** CGS lze paralelizovat lépe než MGS.
- **Numerická stabilita:** MGS zachovává ortogonalitu lépe než CGS.

Srovnání s Householder/Givens QR

- GS je výpočetně výhodnější, pokud $m \ll n$ a chceme jen ortonormální bázi.
- GS se **nehodí** pro řešení soustav rovnic (konstruuje Q, nikoliv unitární transformace aplikované na b, a je méně stabilní).

Numerická stabilita QR rozkladů (čtení navíc): Kvalita zachování ortogonality sloupců Q (měříme normou chyby $\|I - \mathrm{fl}(Q)^* \,\mathrm{fl}(Q)\|$):

- Householder / Givens: $\approx \epsilon_{\text{mach}}$
- MGS: $\approx \varkappa(A) \cdot \epsilon_{\text{mach}}$ (ztráta ortogonality závisí na podmíněnosti A).
- CGS: $\approx \varkappa(A)^2 \cdot \epsilon_{\text{mach}}$ (závisí na podmíněnosti A na druhou).

Poznámka: Pro CGS se někdy používá reortogonalizace (algoritmus ICGS, platí "twice is enough"), ale tím výrazně zvyšujeme výpočetní náklady.

VĚTA 3.6: *Zpětná stabilita QR rozkladů*

Nechť $\mathrm{fl}(Q), \mathrm{fl}(R)$ jsou faktory spočtené pomocí CGS, MGS, ICGS, Givensových rotací nebo Householderových reflexí v FPA s přesností ϵ_{mach}. Pak

$$\|A - \mathrm{fl}(Q)\,\mathrm{fl}(R)\| \approx \epsilon_{\text{mach}} \|A\|.$$

To znamená, že máme přesný QR rozklad "blízké"matice $\hat{A}$, kde $\frac{\|A-\hat{A}\|}{\|A\|} \approx \epsilon_{\text{mach}}$. Vyřešili jsme úlohu blízkou původní (zpětná stabilita algoritmu pro výpočet $A = QR$, nikoliv nutně pro ortogonalitu Q).

4 Řešení lineárních aproximačních úloh

Úloha:

$$Ax \approx b, \quad A \in \mathbf{C}^{n\times m}, b \in \mathbf{C}^n$$

Lineární model je dán obecně obdélníkovou maticí.

Chyby modelovací, měření, ukládání dat v FPA, ... často způsobují, že $b \notin \mathrm{im}(A)$, tedy problém je nekompatibilní. Protože neexistuje přesné řešení x, musíme aproximovat (píšeme „$\approx$“) v nějakém smyslu.

Notace:

- $\mathrm{im}(A) = \mathcal{R}(A)$ ... image, range A
- $\ker(A) = \mathcal{N}(A)$ ... jádro, nulový prostor A
- $\mathrm{rank}(A)$ ... hodnost A

4.1 Problém nejmenších čtverců

Uvažujme úlohu, kde chyby jsou pouze v b, zatímco A je přesné.

Idea: najdi nejmenší korekci pravé strany b tak, aby opravená úloha byla kompatibilní

$$f =? : f \in \mathbf{C}^n, \|f\|\text{-malá}, \quad Ax = b + f \quad \text{(kompatibilní opravené pozorování)}$$

$$\implies f = Ax - b =: -r \ldots \text{residuum}$$

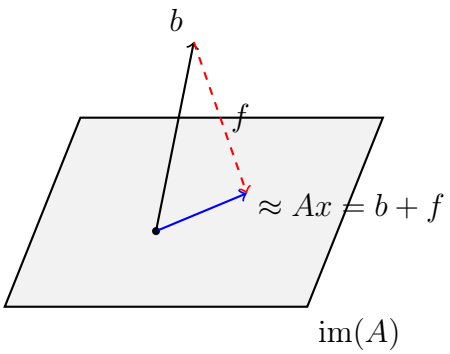
$$\implies \text{ekvivalentně hledám } x \in \mathbf{C}^m \text{ tak, aby } \|b - Ax\| \text{ byla minimální.}$$

Definice 4.1: *Problém nejmenších čtverců (LS)*

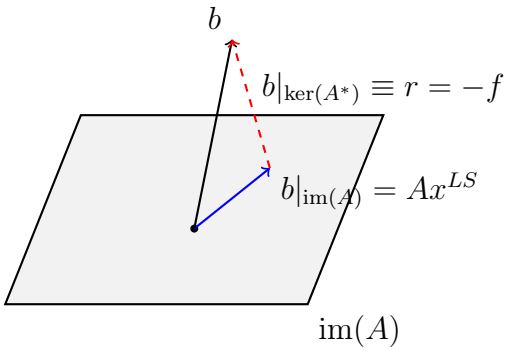
Nechť $A \in \mathbf{C}^{m\times n}, b \in \mathbf{C}^n$. Problém nejmenších čtverců (LS) nazveme úlohu nalezení $x^{LS} \in \mathbf{C}^m$ tak, aby

$$\|f\| = \min_{\hat{f}\in\mathbf{C}^n} \|\hat{f}\| \quad \text{za podmínky} \quad Ax^{LS} = b + f.$$

$$\iff x^{LS} = \operatorname*{argmin}_{x\in\mathbf{C}^m} \|b - Ax\|$$



Obrázek 9: Geometrická interpretace problému nejmenších čtverců



Obrázek 10: Rozklad vektoru b vzhledem k $\mathrm{im}(A)$

Víme: $\mathbf{C}^n = \mathrm{im}(A) \oplus \ker(A^*)$, $\mathrm{im}(A) \perp \ker(A^*)$. Proto

$$b - Ax = \underbrace{b|_{\mathrm{im}(A)} - Ax}_{\in \mathrm{im}(A)} + \underbrace{b|_{\ker(A^*)}}_{\in \ker(A^*)}$$

a po znormování

$$\|b - Ax\|^2 = \|b|_{\mathrm{im}(A)} - Ax\|^2 + \|b|_{\ker(A^*)}\|^2 \geq \|b|_{\ker(A^*)}\|^2.$$

Rovnice $Ax = b|_{\mathrm{im}(A)}$ má jistě řešení x^{LS}, protože $b|_{\mathrm{im}(A)} \in \mathrm{im}(A)$. Pro něj pak platí

$$\|b - Ax^{LS}\|^2 = \|b|_{\mathrm{im}(A)} - Ax^{LS}\|^2 + \|b|_{\ker(A^*)}\|^2 = \|b|_{\ker(A^*)}\|^2,$$

tedy dostaneme minimální normu residua.

Existence a jednoznačnost f a x^{LS}

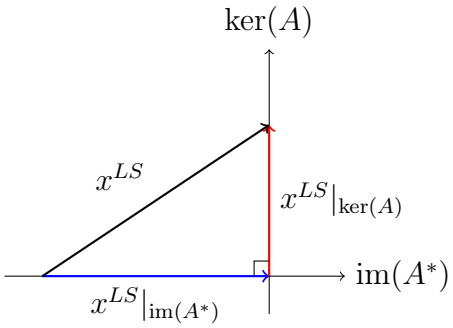
- $f = -r = b|_{\ker(A^*)}$ je jednoznačně určená minimální korekce b.
- x^{LS} existuje vždy a je řešením opravené soustavy $Ax^{LS} = b|_{\mathrm{im}(A)}$.
- x^{LS} je určeno jednoznačně $\iff$ lze $b|_{\mathrm{im}(A)}$ reprezentovat jednoznačně jako lineární kombinaci sloupců matice A

  $$\iff A \text{ má LN sloupce} \iff \mathrm{rank}(A) = m$$

  Potom pro $A = [a_1 \dots a_m], x^{LS} = [x_1^{LS}, \dots, x_m^{LS}]$ je

  $$\sum_{i=1}^{m} x_i^{LS} a_i = b|_{\mathrm{im}(A)}$$

  .
- $x^{LS} = 0 \iff b \perp \mathrm{im}(A)$ ... pozorování b není korelováno s modelem A.



Obrázek 11: Rozklad LS řešení v případě $\mathrm{rank}(A) < m$

4.1.1 Řešení LS s minimální normou

Co se bude dít v případě, kdy $\mathrm{rank}(A) < m$?

Pozorování: $z \in \ker(A) \implies A(x^{LS} + z) = Ax^{LS}. \implies \exists\, \infty$ mnoho x^{LS}.

Pro normu LS řešení platí

$$\|x^{LS}\|^2 = \|x^{LS}|_{\mathrm{im}(A^*)}\|^2 + \|x^{LS}|_{\ker(A)}\|^2 \geq \|x^{LS}|_{\mathrm{im}(A^*)}\|^2.$$

Navíc $x^{LS}|_{\mathrm{im}(A^*)}$ je také LS řešení $Ax \approx b$ a má minimální normu.

Věta 4.1: *LS řešení s minimální normou*

Existuje právě jedno x^{LS}_{min} řešení LS problému s minimální normou a je dáno vztahy

$$Ax^{LS}_{min} = b|_{\mathrm{im}(A)}, \quad x^{LS}_{min} \in \mathrm{im}(A^*).$$

Důkaz. Zvlášť dokážeme existenci a jednoznačnost.

1. **Existence:** Z konstrukce. Nechť x^{LS} je LS řešení. Pak $x^{LS}_{min} = x^{LS}|_{\mathrm{im}(A^*)}$ je také LS řešení a splňuje podmínky věty.

2. **Jednoznačnost:** Sporem. Nechť existují x_1, x_2 splňující vztahy výše. Pak

$$Ax_1 - Ax_2 = b|_{\mathrm{im}(A)} - b|_{\mathrm{im}(A)} = 0$$

$$\implies A(x_1 - x_2) = 0 \implies (x_1 - x_2) \in \ker(A).$$

Zároveň $(x_1 - x_2) \in \mathrm{im}(A^*)$, protože $x_1, x_2 \in \mathrm{im}(A^*)$. Pak

$$(x_1 - x_2) \in \ker(A) \cap \mathrm{im}(A^*) = \{0\}$$

$$\implies x_1 - x_2 = 0 \implies x_1 = x_2.$$

□

Množina všech LS řešení je tvaru: $x^{LS}_{min} + \ker(A)$.

4.2 Úplný problém nejmenších čtverců

Uvažujeme realistickou situaci, kdy A i b obsahují chyby.

Idea: najdi nejmenší korekci A, b tak, aby úloha byla kompatibilní.

Definice 4.2: *Problém úplných nejmenších čtverců (TLS)*

Nechť $A \in \mathbf{C}^{n\times m}, b \in \mathbf{C}^n$. Problém úplných nejmenších čtverců (TLS) nazveme úlohu nalezení $x^{TLS} \in \mathbf{C}^m$ tak, aby

$$\min_{\substack{f\in\mathbf{C}^n \\ E\in\mathbf{C}^{n\times m}}} \|[f,E]\|_F \quad \text{za podmínky} \quad \underbrace{(A+E)}_{\substack{\uparrow \\ \text{oprava} \\ \text{modelu}}} x = \underbrace{b+f}_{\substack{\uparrow \\ \text{oprava} \\ \text{pozorování}}} .$$

Definice 4.3: *Frobeniova norma matice*

Nechť $A = (a_{ij}) \in \mathbf{C}^{n\times m}$. Frobeniova norma matice A je dána vzorcem

$$\|A\|_F = \sqrt{\sum_{i=1}^{n}\sum_{j=1}^{m} |a_{ij}|^2}.$$

Příklad 4.1 (Neexistence TLS řešení)**.** TLS problém nemusí mít řešení, což ilusturujeme na úloze pro $A = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}, b = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$. Oprava $E_\varepsilon = \begin{bmatrix} 0 & 0 \\ 0 & \varepsilon \end{bmatrix}$ vede na kompatibilní úlohu $(A+E)x = b$ s řešením $x_\varepsilon = \begin{bmatrix} 1 \\ \varepsilon^{-1} \end{bmatrix}$, $\|E_\varepsilon\|_F = \varepsilon$.

Ukážeme, že tato úloha nemá minimální korekci. Pro spor předpokládejme existenci korekce f, E takové, že $\|f|E\|_F$ je minimální. Protože původní úloha byla nekompatibilní, norma korekce je nenulová, tedy existuje $0 < \tilde{\varepsilon} < \|[f|E]\|_F$. Korekce $E_{\tilde{\varepsilon}}$ má menší normu než $[f|E]$, což je spor s minimalitou.

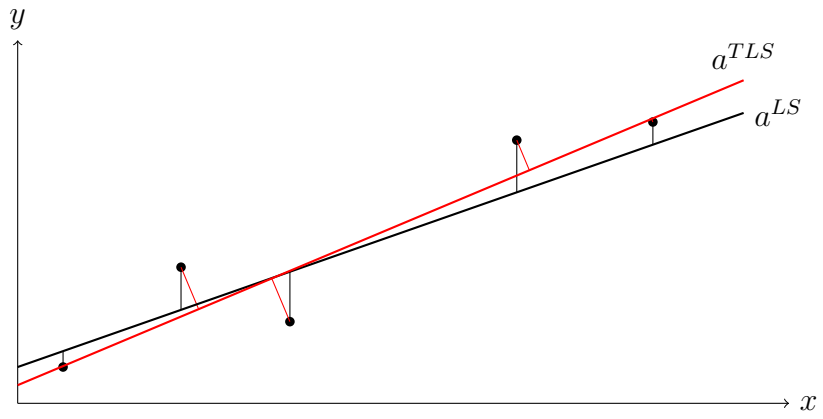
Analýza TLS úlohy se provádí přes SVD a aproximaci matice $[b|A]$ maticí menší hodnosti.

Příklad 4.2 (Srovnání LS a TLS)**.** Máme měřené body $[x_i, y_i], i = 1, \ldots, m$. Předpokládáme model lineární závislosti, tedy body mají ležet na přímce $ax = y$ s neznámým a. Sestavíme úlohu

$$\begin{bmatrix} x_1 \\ \vdots \\ x_m \end{bmatrix} a \approx \begin{bmatrix} y_1 \\ \vdots \\ y_m \end{bmatrix}$$

LS (lineární regrese): opraví jen souřadnice y_i.
TLS (ortogonální regrese): opraví obě souřadnice bodů.

Další možná zobecnění: Škálování, dodatečné podmínky (constraints), násobná měření, ...



Obrázek 12: Srovnání LS a TLS regrese

Příklad 4.3 (Aproximace obecné funkce $f(x)$)**.** Máme přibližné hodnoty $b_i \approx f(x_i)$ v bodech x_i pro $i = 1, \ldots, m$. Zvolíme vhodné lineárně nezávislé bázové funkce $\varphi_j(x)$ a chceme najít koeficienty c_j tak, aby

$$f(x) \approx \sum_{j=1}^{n} c_j \varphi_j(x).$$

Pomocí bodů $x := x_i$ sestavíme aproximační problém

$$b_i \approx \sum_{j=1}^{n} c_j \varphi_j(x_i), \quad i = 1, \ldots, m$$

LS:

$$\min_{c_1,\ldots,c_n \in \mathbf{R}} \sum_{i=1}^{m} \omega_i \underbrace{(b_i - \sum c_j \varphi_j(x_i))^2}_{\text{váhy } \omega_i \text{ dle přesnosti } b_i}$$

4.3 Metody řešení LS problému v případě triviálního jádra

Uvažujme $A \in \mathbf{C}^{n\times m}$, tedy $n \geq m$ (soustava je přeurčená), a $\operatorname{rank}(A) = m$. Pak existuje právě jedno řešení x^{LS}.

Poznámka 4.1 (Unitární invariance LS)**.** Platí

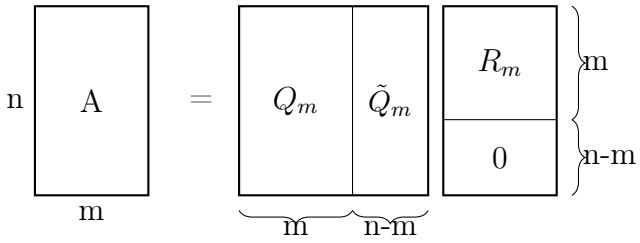
$$\|b - Ax\| = \|U^*(b - Ax)\| \quad \forall\, U \in \mathbf{C}^{n\times n} \text{ unitární}$$

$\implies$ LS (i TLS) je unitárně invariantní. $\implies$ Unitární transformace dat nemění řešení.

4.3.1 Řešení pomocí QR rozkladu

Ekonomický rozklad:

$$A = QR = [Q_m, \tilde{Q}_m] \begin{bmatrix} R_m \\ 0 \end{bmatrix} = Q_m R_m.$$



Obrázek 13: Schéma QR rozkladu

Lemma 4.2: *Vlastnosti QR rozkladu*

Platí:

1. $R_m \in \mathbf{C}^{m\times m}$ je regulární (horní trojúhelníková).
2. $Q_m = [q_1, \ldots, q_m]$, $\operatorname{im}(A) = \operatorname{span}\{q_1, \ldots, q_m\}$.
3. $\tilde{Q}_m = [q_{m+1}, \ldots, q_n]$, $\ker(A^*) = \operatorname{span}\{q_{m+1}, \ldots, q_n\}$.

Odvození:

$$Ax \approx b \quad /Q^* \iff Rx \approx Q^*b \iff \begin{bmatrix} R_m \\ 0 \end{bmatrix} x \approx \begin{bmatrix} Q_m^*b \\ \tilde{Q}_m^*b \end{bmatrix} \begin{matrix} \}m \\ \}n-m \end{matrix}$$

Vektor x^{LS} tedy získáme řešením sosutavy s regulární maticí

$$\underbrace{R_m x^{LS} = Q_m^* b}_{\text{zpětná substituce}}.$$

Navíc dostáváme snadno normu residua $\|b - Ax^{LS}\| = \|\tilde{Q}_m^* b\|$.

Implicitní postup výpočtu Matici Q *nekonstruujeme*, převádíme ortogonálně:

$$[A|b] \xrightarrow[\text{(nebo Giv. rotace)}]{\text{Hous. reflexe}} H_m \ldots H_1 [A|b] = \left[\begin{array}{c|c} R_m & Q_m^* b \\ \hline 0 & \tilde{Q}_m^* b \end{array}\right] \begin{matrix} \}m \\ \}n-m \end{matrix}$$

Postup výpočtu přes CGS (MGS)

- GS počítá ekonomický rozklad $A = Q_m R_m$.
- Provedeme $(m+1)$ kroků GS na matici $[A|b]$:

$$[A|b] = [Q_m|q] \cdot \begin{bmatrix} R_m & b_1 \\ 0 & b_2 \end{bmatrix} \begin{matrix} \}m \\ \}1 \end{matrix}$$

  Co je v posledním sloupci trojúhelníkové matice $(b_1, b_2)^T$?

CGS (krok $m+1$)

$$\tilde{q} := b - Q_m Q_m^* b = b|_{\mathrm{im}(A)^\perp}, \quad q := \frac{\tilde{q}}{\|\tilde{q}\|}$$

$$\implies b_1 = Q_m^* b = \begin{bmatrix} q_1^* b \\ \vdots \\ q_m^* b \end{bmatrix}, \quad b_2 = \|\tilde{q}\| = \|b|_{\mathrm{im}(A)^\perp}\| = \|b - Ax^{LS}\|,$$

tedy b_2 je přímo norma residua.

Shrnutí

$$[A|b] \xrightarrow[\text{MGS}]{\text{CGS}} [Q_m|q] \cdot \begin{bmatrix} R_m & Q_m^* b \\ 0 & \|b - Ax^{LS}\| \end{bmatrix}$$

Pak provedeme zpětnou substituci $R_m x^{LS} = Q_m^* b$.

Vlastnosti

$$\varkappa(A) = \frac{\sigma_1}{\sigma_m} = \varkappa(R_m)$$

$\implies$ Je-li $\varkappa(A) \ll \frac{1}{\epsilon_{\text{mach}}}$, pak R_m je numericky regulární a výpočet x^{LS} přes QR je spolehlivý.

4.3.2 Řešení pomocí normálních rovnic

- Sestavení A^*A je výhodné jen výjimečně, pro $n \gg m$ a m malé (neboť dostaneme malou soustavu rovnic $m \times m$).
- **Iterační metody** – používám jen násobení $(A^*A)z = A^*(Az)$.
  - Existují speciální iterační metody pro HPD matice.
- **Přímé metody** – Choleského rozklad ≡ speciální varianta Gaussovy eliminace pro HPD matice.

Vlastnosti Normální rovnice mají mnohem větší podmíněnost:

$$\varkappa(A^*A) = \frac{\sigma_1(A^*A)}{\sigma_m(A^*A)} = \frac{\sigma_1^2(A)}{\sigma_m^2(A)} = \varkappa^2(A).$$

$\implies$ Spolehlivé, je-li $\varkappa^2(A) \ll \frac{1}{\epsilon_{\text{mach}}}$.

4.3.3 Rozšířená matice

$$\underbrace{\begin{bmatrix} I & A \\ A^* & 0 \end{bmatrix}}_{\text{sedlová matice}} \begin{bmatrix} r \\ x \end{bmatrix} = \begin{bmatrix} b \\ 0 \end{bmatrix}, \quad r = b - Ax$$

Sedlobodová matice je regulární, hermitovská a indefinitní.

Věta 4.3: *Řešení LS pomocí rozšířené matice*

Je-li $\operatorname{rank}(A) = m$, pak x^{LS} je řešením sedlové soustavy výše.

Důkaz. Soustava se sedlobodovou maticí je ekvivalentní:

$$\begin{aligned} r + Ax = b &\iff r = b - Ax \\ A^*r = 0 & \end{aligned}$$

Dosazením za r do druhé rovnice máme

$$A^*(b - Ax) = 0 \iff A^*Ax = A^*b.$$

Což jsou normální rovnice, jejichž řešením je x^{LS}. □

4.4 Metody řešení LS problému v případě netriviálního jádra

Existuje právě jedno řešení x^{LS}_{min}. Označme $r := \operatorname{rank}(A)$.

4.4.1 Řešení pomocí SVD

Uvažujeme plný a ekonomický singulární rozklad (SVD)

$$A = U\Sigma V^* = U_r \Sigma_r V_r^*,$$

kde U, V jsou unitární, $\Sigma \in \mathbf{C}^{n\times m}$ (z plného SVD) a U_r, V_r^* mají ortonormální sloupce, $\Sigma_r \in \mathbf{C}^{r\times r}$ (ekonomické SVD). Dále

$$\Sigma_r = \operatorname{diag}(\sigma_1, \ldots, \sigma_r), \quad \sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0.$$

Víme:

- U_r má ve sloupcích bázi $\operatorname{im}(A)$.
- V_r má ve sloupcích bázi $\operatorname{im}(A^*)$.

Odvození: Označme x^{LS} libovolné LS řešení.

$$Ax^{LS} = b|_{\operatorname{im}(A)} = U_r U_r^* b \quad \text{(ortoprojektor na } \operatorname{im}(A)\text{)}$$

Dosadíme za A:

$$\begin{aligned} U_r \Sigma_r V_r^* x^{LS} &= U_r U_r^* b \quad /U_r^* \\ \Sigma_r V_r^* x^{LS} &= U_r^* b \\ V_r^* x^{LS} &= \Sigma_r^{-1} U_r^* b \quad \ldots \text{koeficienty } x^{LS} \text{ v } \operatorname{im}(A^*) \end{aligned}$$

x^{LS} je minimální v normě $\iff x^{LS} \in \mathrm{im}(A^*)$.

$$\iff \exists y \in \mathbf{C}^r : x^{LS} = V_r y$$

Dosadíme do rovnice pro koeficienty:

$$V_r^* V_r y = \Sigma_r^{-1} U_r^* b \implies y = \Sigma_r^{-1} U_r^* b \quad (\text{protože } V_r^* V_r = I)$$

$$\implies x_{min}^{LS} = V_r \Sigma_r^{-1} U_r^* b \quad \ldots \text{je LS řešení s minimální normou.}$$

DEFINICE 4.4: *Pseudoinverze matice (Moore-Penroseova)*

Matici

$$A^+ := V_r \Sigma_r^{-1} U_r^*$$

nazveme pseudoinverzí matice A.

Poznámka 4.2. Speciálně: je-li A regulární ($n = m = r$), pak

$$A^+ = A^{-1} = V \Sigma^{-1} U^*.$$

Vlastnosti Výpočet přes SVD je zpětně stabilní (díky ortogonálním transformacím), ale stojí $\mathcal{O}(n^3)$, tedy je výpočetně náročný.

4.4.2 Rank-Revealing QR

Čtení navíc

Existuje permutační matice $P \in \mathbf{C}^{m \times m}$ taková, že:

$$AP = [\tilde{A}|\hat{A}], \quad \mathrm{rank}(\tilde{A}) = r \ldots \text{plný}.$$

$$\implies AP = \underbrace{[Q_1|Q_2]}_{\text{ekonom. QR}} \cdot \left[\begin{array}{c|c} R_1 & R_2 \\ \hline 0 & 0 \end{array}\right] \begin{matrix} \}r \\ \}m-r \end{matrix} = Q_1[R_1|R_2].$$

Tento rozklad (QR s pivotací) se počítá např. přes MGS se sloupcovou pivotací.

Řešení:

$$Ax \approx b \iff (AP)(P^T x) \approx b$$

Označme $\hat{x} := P^T x$.

$$[AP|b] \xrightarrow{\text{MGS}} [Q_1|Q_2]^*[AP|b] = \left[\begin{array}{c|c|c} R_1 & R_2 & Q_1^* b \\ \hline 0 & 0 & Q_2^* b \end{array}\right] \begin{matrix} \}r \\ \}n-r \end{matrix}$$

$$\iff \begin{bmatrix} R_1 & R_2 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} \hat{x}_1 \\ \hat{x}_2 \end{bmatrix} \approx \begin{bmatrix} Q_1^* b \\ Q_2^* b \end{bmatrix}$$

$\hat{x}$ není jednoznačné a splňuje:

$$R_1 \hat{x}_1 + R_2 \hat{x}_2 = Q_1^* b \iff \hat{x}_1 = R_1^{-1}(Q_1^* b - R_2 \hat{x}_2)$$

(kde $\hat{x}_2$ lze volit, např. $\hat{x}_2 = 0$).

Shrnutí:

$$x^{LS} = P \begin{bmatrix} R_1^{-1} Q_1^* b \\ 0 \end{bmatrix}$$

je LS řešení (ale ne nutně s nejmenší normou, pokud $R_2 \neq 0$).

$$\|r\| = \|b - Ax^{LS}\| = \|Q_2^* b\| \ldots \text{reziduum.}$$

Vlastnosti:

- Jako QR pro $\text{rank}(A) = m$.
- Pivotujeme za běhu (jako u GE s pivotací).

5 Částečný problém vlastních čísel

Úloha: Mějme $A \in \mathbf{C}^{n\times n}$, vlastní čísla seřazená dle velikosti: $|\lambda_1| \geq |\lambda_2| \geq \cdots \geq |\lambda_n| \geq 0$.

Cíl: Najít aproximaci několika vlastních čísel a vektorů.

- Např. největší – odhad $\|A\|$, Google PageRank problém.
- Nejmenší – regularita A.
- λ_{min} a λ_{max} – odhad $\varkappa(A)$.
- Aproximace A maticí menší hodnosti.

5.1 Mocninná metoda

Slouží k odhadu jednoho vlastního páru (dominantního) $\equiv$ dvojice (λ_1, v_1).

Speciální případ: Nechť $|\lambda_1| > |\lambda_2|$, tj. dominantní vlastní číslo je jednoznačně určeno. Nechť navíc A má úplný systém vlastních vektorů $s_1, \ldots, s_n$, tj. je diagonalizovatelná. Pak platí

$$A = SDS^{-1}, \quad S = [s_1, \ldots, s_n], \quad D = \operatorname{diag}(\lambda_1, \ldots, \lambda_n).$$

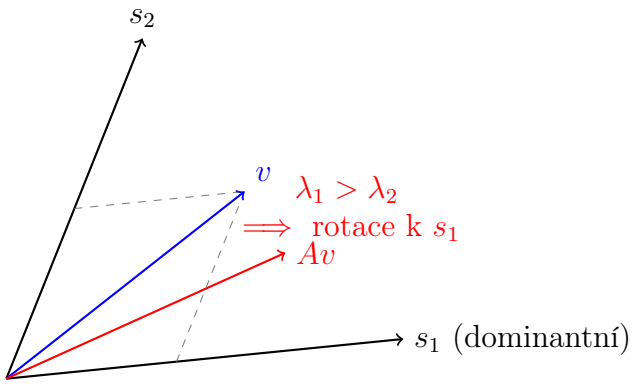
Odvození: Zvolme $v \in \mathbf{C}^n, v \neq 0$ libovolný startovní vektor. Co se stane, budu-li postupně násobit $Av, A^2v \ldots$? Vyjádřeme v v bázi vlastních vektorů A (s koeficienty c_i):

$$\exists\, c_1, \ldots, c_n \in \mathbf{C} : v = c_1 s_1 + c_2 s_2 + \cdots + c_n s_n.$$

Vynásobíme v maticí A a využijeme, že $As_i = \lambda_i s_i$,

$$Av = c_1 As_1 + c_2 As_2 + \cdots + c_n As_n = c_1\lambda_1 s_1 + c_2\lambda_2 s_2 + \cdots + c_n\lambda_n s_n.$$

Protože $|\lambda_1|$ je největší, vektor se „pootočí“ do směru s_1.



Obrázek 14: Princip mocninné metody: pootočení vektoru směrem k dominantnímu vlastnímu vektoru s_1.

Sestrojíme sekvenci $v, Av, A^2v, \ldots$. Platí:

$$A^k v = c_1\lambda_1^k s_1 + c_2\lambda_2^k s_2 + \cdots + c_n\lambda_n^k s_n \quad / : \lambda_1^k$$

$$\frac{1}{\lambda_1^k} A^k v = c_1 s_1 + \underbrace{\left[c_2 \left(\frac{\lambda_2}{\lambda_1}\right)^k s_2 + \cdots + c_n \left(\frac{\lambda_n}{\lambda_1}\right)^k s_n \right]}_{\xrightarrow{k\to\infty} 0}$$

Tedy nutně

$$\left(\frac{1}{\lambda_1^k}\right) A^k v \xrightarrow{k\to\infty} c_1 s_1, \quad \text{pokud } c_1 \neq 0.$$

Chceme ale normovaný vlastní vektor, proto v každém kroku (místo škálování neznámým λ_1) normujeme. Konvergence do směru dominantního vlastního vektoru se zachovává:

$$\frac{A^k v}{\|A^k v\|} \xrightarrow{k\to\infty} s, \quad \text{kde } \|s\| = 1 \text{ a } s \text{ je vlastní vektor k } \lambda_1.$$

5.1.1 Aproximace vlastního čísla

Idea: $As_1 = \lambda_1 s_1 \implies \lambda_1 = \frac{s_1^* A s_1}{s_1^* s_1}$. Je-li $\tilde{v} \approx s_1$, pak $\mu := \frac{\tilde{v}^* A \tilde{v}}{\tilde{v}^* \tilde{v}} \approx \lambda_1$.

Definice 5.1: *Rayleighův podíl*

Nechť $\tilde{v}$ je aproximace vlastního vektoru matice A. Pak číslo

$$\mu = \frac{\tilde{v}^* A \tilde{v}}{\tilde{v}^* \tilde{v}}$$

nazveme Rayleighův podíl aproximující příslušné vlastní číslo.

Algoritmus 6: Mocninná metoda

Input: $A \in \mathbf{C}^{n\times n}, v \in \mathbf{C}^n \setminus \{0\}$, m (max. počet iterací)
Output: v_m, μ_m (aproximace vlastního páru)

```
v_0 := v/‖v‖;
for k = 1 to m do
    w := A v_{k-1} ;          // násobení matice vektorem
    v_k := w/‖w‖ ;            // normování
    μ_k := v_k^* A v_k ;      // Rayleighův podíl
end
```

Věta 5.1: *Konvergence mocninné metody*

Nechť $A \in \mathbf{C}^{n\times n}$ je diagonalizovatelná, $|\lambda_1| > |\lambda_2|$ a $v \in \mathbf{C}^n$ takové, že $c_1 \neq 0$. Pak pro mocninnou metodu platí, že

- $v_k \xrightarrow{k\to\infty} s$, kde $\|s\| = 1$, $\operatorname{span}\{s_1\} = \operatorname{span}\{s\}$,
- $\mu_k \xrightarrow{k\to\infty} \lambda_1$.

Konvergenci nazveme lineární s rychlostí $\left|\frac{\lambda_2}{\lambda_1}\right|$.

Důkaz. Nechť $s_1, \ldots, s_n$ je báze vlastních vektorů příslušných vlastním číslům $\lambda_1, \ldots, \lambda_n$. Libovolný vektor v lze vyjádřit jako lineární kombinaci $v = \sum_{i=1}^n c_i s_i$. Aplikací mocniny

matice A dostáváme

$$A^k v = \sum_{i=1}^{n} c_i \lambda_i^k s_i = \lambda_1^k \left(c_1 s_1 + \sum_{i=2}^{n} c_i \left(\frac{\lambda_i}{\lambda_1} \right)^k s_i \right).$$

Jelikož $|\lambda_1| > |\lambda_i|$ pro $i = 2, \ldots, n$, platí $\left|\frac{\lambda_i}{\lambda_1}\right| < 1$ a tedy $\left(\frac{\lambda_i}{\lambda_1}\right)^k \to 0$ pro $k \to \infty$. Vektor $A^k v$ se tedy blíží směru s_1 (za předpokladu $c_1 \neq 0$). Normalizovaný vektor v_k pak konverguje k s (kde s je násobek s_1 s jednotkovou normou).

Pro Rayleighův podíl platí:

$$\mu_k = v_k^* A v_k \xrightarrow{k\to\infty} s^* A s = s^* \lambda_1 s = \lambda_1 \|s\|^2 = \lambda_1.$$

Rychlost konvergence je určena nejpomalejším členem v sumě, tj. členem s $\left|\frac{\lambda_2}{\lambda_1}\right|^k$. □

Poznámka 5.1 (Vlastnosti mocninné metody)**.**

- Výpočetní náklady – $1\times$ násobení Av v iteraci.
- Paměť – ukládáme jen v_k, v_{k-1}.
- Rychlost konvergence závisí na tom, jak dobře je λ_1 odděleno od ostatních vlastních čísel a jak blízko je v k s_1 (velikost c_1 oproti $c_2, \ldots, c_n$).

Poznámka 5.2 (Předpoklady mocninné metody v praxi)**.** Podmínku $c_1 \neq 0$ nelze v praxi kontrolovat, ale většinou je splněna (pokud v volíme jako náhodný vektor). V některých případech se nenulová složka „nageneruje“ díky zaokrouhlovacím chybám během výpočtu v FPA.

Obecný případ: $|\lambda_1| = \cdots = |\lambda_p| > |\lambda_{p+1}|$

- Pokud $\lambda_1 = \cdots = \lambda_p$ (vícenásobné dominantní vlastní číslo), metoda konverguje k vlastnímu vektoru (resp. do podprostoru vlastních vektorů).
- Jinak (např. komplexně sdružená vlastní čísla se stejnou absolutní hodnotou) metoda může divergovat.

Příklad 5.1 (Selhání mocninné metody)**.**

$$A = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}, \quad v = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

$$\implies Av = \begin{bmatrix} 1 \\ -1 \end{bmatrix} \implies A^2 v = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \ldots$$

Posloupnost osciluje, nekonverguje.

Poznámka 5.3 (Zobecnění mocninné metody pro nediagonalizovatelné matice)**.** Analogické výsledky platí i pro A nediagonalizovatelnou – mnohem širší důkazy.

5.1.2 Inverzní mocninná metoda

Cíl: Najít λ_n, s_n, tj. nejmenší (v absolutní hodnotě) vlastní číslo a příslušný vlastní vektor.

Idea: Pro regulární matici A platí:

$$\lambda_1, \ldots, \lambda_n \text{ jsou vlastní čísla } A \implies \frac{1}{\lambda_1}, \ldots, \frac{1}{\lambda_n} \text{ jsou vlastní čísla } A^{-1}.$$

Pokud $|\lambda_1| \geq \cdots \geq |\lambda_n|$, pak pro vlastní čísla A^{-1} platí:

$$\left|\frac{1}{\lambda_n}\right| \geq \cdots \geq \left|\frac{1}{\lambda_1}\right|.$$

Tedy $1/\lambda_n$ je dominantní vlastní číslo matice A^{-1}.

Postup: Aplikujeme mocninnou metodu na matici A^{-1}. V algoritmu mocninné metody krok násobení $w := A^{-1}v_{k-1}$ nahradíme řešením soustavy lineárních rovnic:

$$Aw = v_{k-1}.$$

Tím se vyhneme explicitnímu výpočtu inverzní matice. Normalizace $v_k := w/\|w\|$ zůstává stejná. Posloupnost v_k konverguje k vlastnímu vektoru s_n odpovídajícímu λ_n. Proto příslušný Rayleighův podíl konverguje k λ_n.

5.2 Krylovovy prostory

Úloha: Chceme aproximaci *více* vlastních párů matice A. Místo jediného vektoru vezmeme celý prostor generovaný vektory $v, Av, \ldots, A^{m-1}v$.

DEFINICE 5.2: *Krylovův prostor*

Nechť $A \in \mathbf{C}^{n\times n}$ je regulární, $v \in \mathbf{C}^n, v \neq 0$ a $m \geq 1$. Prostor

$$\mathcal{K}_m(A, v) = \operatorname{span}\{v, Av, \ldots, A^{m-1}v\}$$

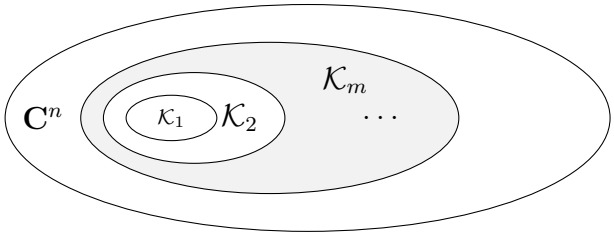
nazýváme m-tý Krylovův (pod)prostor matice A vzhledem k v.

Platí vnoření prostorů:

$$\mathcal{K}_1(A, v) \subseteq \mathcal{K}_2(A, v) \subseteq \cdots \subseteq \mathbf{C}^n.$$

Dimenze těchto prostorů splňují:

$$\dim(\mathcal{K}_m(A, v)) \leq m \leq n.$$



Obrázek 15: Vnoření Krylovových prostorů

Poznámka 5.4 (Krylovův prostor vlastního vektoru)**.** Pokud je v vlastní vektor matice A ($Av = \lambda v$), pak Av je lineárně závislé s v, a tedy $\dim(\mathcal{K}_m(A, v)) = 1$ pro všechna $m \geq 1$. Vidíme, že dimenze Krylovových postorů nemusí být plná.

DEFINICE 5.3: *Stupeň vektoru*

Stupněm vektoru v vzhledem k matici A nazveme číslo

$$d(A, v) := \min\{m \in \mathbf{N} : \dim(\mathcal{K}_m) = \dim(\mathcal{K}_{m+1})\}.$$

VĚTA 5.2: *Vlastnosti stupně vektoru*

Pro stupeň vektoru $d = d(A, v)$ platí:

1. $d(A, v) \leq n$.
2. $\dim(\mathcal{K}_m(A, v)) = m$ pro $m = 1, \ldots, d(A, v)$.
3. $\dim(\mathcal{K}_m(A, v)) = d(A, v)$ pro $m \geq d(A, v)$.

Důkaz. 1. Triviální, podprostor v $\mathbf{C}^n$ má dimenzi maximálně n.

2. Z definice $d(A, v)$ jako minima. Dokud dimenze roste, roste o 1 (přidáváme jeden vektor).

3. $A^d v$ je lineární kombinací $v, \ldots, A^{d-1}v$. Tedy

$$\exists\, \alpha_i, i = 0, \ldots, d-1 : A^d v = \sum_{i=0}^{d-1} \alpha_i A^i v.$$

Vynásobíme A,

$$A^{d+1}v = \sum_{i=0}^{d-1} \alpha_i A^{i+1} v = \sum_{i=0}^{d-2} \alpha_i A^{i+1} v + \alpha_{d-1} A^d v.$$

Dosadíme za $A^d v$ z prvního vztahu,

$$= \sum_{k=1}^{d-1} \alpha_{k-1} A^k v + \alpha_{d-1} \left(\sum_{j=0}^{d-1} \alpha_j A^j v \right).$$

Výsledkem je lineární kombinace vektorů z $\mathcal{K}_d(A, v)$, tedy $A^{d+1}v \in \mathcal{K}_d(A, v)$. Indukcí totéž platí pro všechna vyšší mocniny.

□

Důsledek 5.1 (Ostrá inkluze nemaximálních Krylovových prostorů)**.**

$$\mathcal{K}_1(A, v) \subset \mathcal{K}_2(A, v) \subset \cdots \subset \mathcal{K}_d(A, v) = \mathcal{K}_{d+1}(A, v) = \ldots$$

Prostor $\mathcal{K}_m(A, v)$ pro $m \geq d(A, v)$ je A-invariantní (zobrazení A ho zobrazuje do sebe). Stupeň d udává maximální dimenzi Krylovova postoru, který lze sestrojit pro dané A a v.

5.2.1 Výpočet báze $\mathcal{K}_m(A, v)$

Předpoklad: $m \leq d(A, v)$.

- Krylovova báze $v, Av, \ldots, A^{m-1}v$ je numericky nevhodná. Vektory konvergují k dominantnímu vlastnímu vektoru, takže jsou numericky skoro lineárně závislé.
- Chceme ortonormální (ON) bázi. Nesmíme mocnit A^k.

Arnoldiho proces

- Speciálně upravený Gram-Schmidtův ortogonalizační proces.
- Existuje ve variantách dle GS (CGS – klasický, MGS – modifikovaný), lišících se stabilitou, možností paralelizace atd.
- Počítá ON bázi $\mathcal{K}_m(A, v)$.

Odvození Arnoldiho procesu: Postupujeme následovně. Mějme vektor v, jeho normalizací dostaneme v_1. Násobíme v_1 maticí A a vektor Av_1 ortogonalizujeme proti v_1. Tím získáme vektor $\tilde{v}_2$ a jeho normalizací v_2. Násobíme v_2 maticí A a ortogonalizujeme proti $[v_1, v_2]$, atd.

- Krok $k = 1$: $v_1 := v/\|v\|$, $\operatorname{span}\{v\} = \operatorname{span}\{v_1\}$.
- Krok $k = 2$:
$$\tilde{v}_2 := Av_1 - (v_1^* A v_1)v_1 \quad \text{(GS krok)}$$
$$v_2 := \tilde{v}_2/\|\tilde{v}_2\|.$$
Platí $\operatorname{span}\{v_1, Av_1\} = \operatorname{span}\{v_1, v_2\}$.
- Krok $k = 3$:
$$\tilde{v}_3 := (Av_2) - (v_1^* A v_2)v_1 - (v_2^* A v_2)v_2$$
$$v_3 := \tilde{v}_3/\|\tilde{v}_3\|.$$

Klíčová otázka: Proč mohu volit Av_2 místo A^2v_1? Platí

$$\operatorname{span}\{v_1, A^1v_1, A^2v_1\} = \operatorname{span}\{v_1, Av_1, Av_2\} = \operatorname{span}\{v_1, v_2, v_3\},$$

neboť $Av_1 \in \operatorname{span}\{v_1, v_2\} \implies A(Av_1) \in \operatorname{span}\{Av_1, Av_2\}$.

Obecný krok Arnoldiho procesu:

$$\tilde{v}_{k+1} := Av_k - \sum_{i=1}^{k} (v_i^* A v_k) v_i$$
$$v_{k+1} := \tilde{v}_{k+1}/\|\tilde{v}_{k+1}\|, \quad k = 1, 2, \ldots, m-1.$$

LEMMA 5.3: *Korektnost Arnoldiho procesu*

Vektory $v_1, \ldots, v_m$ generované Arnoldiho procesem jsou ortonormální a platí

$$\operatorname{span}\{v_1, \ldots, v_m\} = \mathcal{K}_m(A, v).$$

Důkaz. Plyne z konstrukce (Gram-Schmidt na bázi Krylovova prostoru). □

Algoritmus 7: Arnoldiho algoritmus (MGS varianta)

Input: A, v, m (vstup: matice A, startovní vektor v, dimenze m)
Output: $v_1, \ldots, v_m$ (výstup: ON báze $\mathcal{K}_m(A, v)$)

```
1  v_1 := v/‖v‖;
2  for k = 1, 2, ..., m − 1 do
3  |  w := A v_k;
4  |  for j = 1, ..., k do
5  |  |  h_jk := v_j^* w;
6  |  |  w := w − h_jk v_j;
7  |  end
8  |  h_{k+1,k} := ‖w‖;
9  |  if h_{k+1,k} = 0 then
10 |  |  STOP ;          // nastane pouze, pokud dosáhneme d = d(A, v)
11 |  end
12 |  else
13 |  |  v_{k+1} := w/h_{k+1,k};
14 |  end
15 end
```

Poznámka 5.5 (Vlastnosti Arnoldiho algoritmu)**.**

- **Dlouhé rekurence:** Nový vektor se ortogonalizuje proti *všem* předchozím (vnitřní cyklus $j = 1, \ldots, k$). To znamená, že ortogonalita se zachovává dobře (MGS je stabilnější než CGS).
- **Výpočetní náklady:** V každé iteraci $1\times$ násobení matice A (typicky nákladná operace), cena ortogonalizace (počet skalárních součinů) roste s k.
- **Paměť:** Musíme ukládat celou bázi $v_1, \ldots, v_m$.

Označení výsledků Arnoldiho algoritmu Označme matici bázových vektorů (ON sloupce):

$$V_k := [v_1, \ldots, v_k] \in \mathbf{C}^{n\times k}.$$

Koeficienty h_{jk} tvoří tzv. horní Hessenbergovu matici $H_k \in \mathbf{C}^{k\times k}$ (matice s nulami pod první poddiagonálou):

$$H_k = \begin{bmatrix} h_{11} & h_{12} & \ldots & h_{1,k-1} & h_{1k} \\ h_{21} & h_{22} & \ldots & h_{2,k-1} & h_{2k} \\ 0 & h_{32} & \ddots & \vdots & \vdots \\ \vdots & \ddots & \ddots & h_{k-1,k-1} & \vdots \\ 0 & \ldots & 0 & h_{k,k-1} & h_{k,k} \end{bmatrix}.$$

Rozšířenou matici značíme $H_{k+1,k} \in \mathbf{C}^{(k+1)\times k}$:

$$H_{k+1,k} := \begin{bmatrix} H_k \\ 0 \ldots 0\ h_{k+1,k} \end{bmatrix}, \quad k = 1, \ldots, m-1.$$

V horním trojúhelníku $H_k, H_{k+1,k}$ jsou ortogonalizační koeficienty, poddiagonální prvky jsou kladné normy. Prvek $h_{k+1,k}$ je norma posledního vektoru w po ortogonalizaci (pokud algoritmus neskončil, je $h_{k+1,k} > 0$).

Lemma 5.4: *Vztahy v Arnoldiho procesu*

Nechť $m \leq d(A, v)$. Pak pro matice $V_k, H_k, H_{k+1,k}$ $(k = 1, \ldots, m-1)$ generované Arnoldiho procesem platí:

1. $AV_k = V_{k+1}H_{k+1,k}$,
2. $AV_k = V_kH_k + h_{k+1,k}v_{k+1}e_k^T$,
3. $V_k^*AV_k = H_k$.

Důkaz. 1. Z konstrukce Arnoldiho procesu (Gram-Schmidt) plyne:

$$h_{k+1,k}v_{k+1} = Av_k - \sum_{i=1}^{k} h_{i,k}v_i \implies Av_k = \sum_{i=1}^{k} h_{i,k}v_i + h_{k+1,k}v_{k+1}.$$

Toto platí pro každý sloupec, tedy maticově:

$$AV_k = V_{k+1}H_{k+1,k}.$$

2. Rozepíšeme-li matici $H_{k+1,k}$ na blok H_k a poslední řádek $(0, \ldots, 0, h_{k+1,k})$, dostaneme:

$$V_{k+1}H_{k+1,k} = V_kH_k + v_{k+1}h_{k+1,k}e_k^T.$$

Dosazením do bodu 1 získáme dokazovaný vztah.

3. Vynásobíme vztah 2 zleva maticí V_k^*:

$$V_k^*AV_k = V_k^*V_kH_k + h_{k+1,k}\underbrace{V_k^*v_{k+1}}_{=0}e_k^T = I_kH_k = H_k.$$

(Využili jsme ortonormality báze V_{k+1}, tedy $V_k^*V_k = I$ a $V_k^*v_{k+1} = 0$.)

□

Poznámka 5.6 (Zúžení operátoru)**.** Řekneme, že H_k je *zúžení operátoru* A na prostor $\mathcal{K}_k(A, v)$. Zatímco $A : \mathbf{C}^n \to \mathbf{C}^n$, tak $H_k : \mathbf{C}^k \to \mathbf{C}^k$.

5.2.2 Lanczosova tridiagonalizace

Jedná se o zjednodušený Arnoldiho proces pro případ, kdy je matice A Hermitovská (tj. $A = A^*$).

Věta 5.5: *Arnoldiho proces pro Hermitovskou matici*

Nechť A je Hermitovská, $k < d(A, v)$. Pak matice H_k generovaná Arnoldiho procesem je symetrická, třídiagonální s kladnými poddiagonálními prvky (tzv. Jacobiho matice).

Důkaz. Z lemmatu víme, že $H_k = V_k^* A V_k$. Použijeme-li $A = A^*$, pak:

$$H_k^* = (V_k^* A V_k)^* = V_k^* A^* (V_k^*)^* = V_k^* A V_k = H_k.$$

Matice H_k je tedy Hermitovská a jako taková má reálné diagonální prvky. Zároveň je H_k horní Hessenbergova (z definice Arnoldiho procesu). Matice, která je Hermitovská a horní Hessenbergova, musí být nutně třídiagonální. Protože poddiagonání prvky matice H_k jsou kladné normy, je H_k dokonce reálná symetrická. □

Matice H_k má tedy tvar:

$$H_k = \begin{bmatrix} h_{11} & h_{21} & 0 & \dots & 0 \\ h_{21} & h_{22} & h_{23} & \ddots & \vdots \\ 0 & h_{32} & h_{33} & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & h_{k,k-1} \\ 0 & \dots & 0 & h_{k,k-1} & h_{k,k} \end{bmatrix}$$

Důsledek 5.2 (Tříčlenná rekurence)**.** Jelikož je H_k třídiagonální, plyne z toho, že ortonormální bázi $\mathcal{K}_m(A, v)$ pro A Hermitovskou lze počítat tzv. **tříčlennou rekurencí**. To znamená, že každý nový vektor Av_k ortogonalizujeme pouze proti dvěma předchozím vektorům. Navíc jeden koeficient známe z předchozího kroku (norma) díky symetrii.

Algoritmus 8: Lanczosův algoritmus (krátké rekurence)

Input: A, v, m
Output: V_m, T_m

$v_1 := v/\|v\|$;
$\beta_1 := 0, v_0 := 0$;
for $k = 1$ **to** m **do**
  $w := Av_k - \beta_k v_{k-1}$; `// pouze 3-členná rekurence`
  $\alpha_k := v_k^* w$;
  $w := w - \alpha_k v_k$;
  $\beta_{k+1} := \|w\|$;
  $v_{k+1} := w/\beta_{k+1}$;
end

Označení Označme

$$V_k = [v_1, \dots, v_k], \quad T_k = \begin{bmatrix} \alpha_1 & \beta_2 & 0 & \dots \\ \beta_2 & \alpha_2 & \beta_3 & \dots \\ 0 & \beta_3 & \alpha_3 & \ddots \\ \vdots & \vdots & \ddots & \ddots \end{bmatrix} \in \mathbf{R}^{k\times k}, \quad T_{k+1,k} = \begin{bmatrix} T_k \\ 0 \dots \beta_{k+1} \end{bmatrix} \in \mathbf{R}^{(k+1)\times k}.$$

Matice T_k, $T_{k+1,k}$ jsou speciálním případem H_k, $H_{k+1,k}$. Platí pro ně tedy analogické vztahy.

VĚTA 5.6: *Vztahy v Lanczosově procesu*

Platí:

$$AV_k = V_{k+1} T_{k+1,k} = V_k T_k + \beta_{k+1} v_{k+1} e_k^T$$
$$V_k^* A V_k = T_k, \quad k = 1, \dots, m-1.$$

Poznámka 5.7 (Vlastnosti Lanczosova procesu)**.** Krátké rekurence jsou výpočetně méně náročné (konstantní cena na iteraci), ale ortogonalita (OG) se vlivem zaokrouhlovacích chyb rychle ztrácí.

5.3 Ritzova čísla

Nyní využijeme Krylovovy prostory k aproximaci vlastních párů. Připomeňme vztah pro výstupy Arnoldiho procesu (platný pro $k \leq d$):

$$AV_k = V_k H_k + h_{k+1,k} v_{k+1} e_k^T.$$

Nechť $(\mu_i^{(k)}, y_i^{(k)})$, $i = 1, \ldots, \tilde{k}$ jsou vlastní páry matice H_k, tj.

$$H_k y_i^{(k)} = \mu_i^{(k)} y_i^{(k)}.$$

Zde $\tilde{k} \leq k$ značí počet vlastních párů H_k. Přenásobíme-li vztah nahoře vektorem $y_i^{(k)}$ zprava, dostaneme:

$$AV_k y_i^{(k)} = V_k H_k y_i^{(k)} + h_{k+1,k} v_{k+1} e_k^T y_i^{(k)}.$$

Dosadíme $H_k y_i^{(k)} = \mu_i^{(k)} y_i^{(k)}$ a označíme $x_i^{(k)} := V_k y_i^{(k)}$:

$$Ax_i^{(k)} = \mu_i^{(k)} x_i^{(k)} + h_{k+1,k}(e_k^T y_i^{(k)}) v_{k+1}.$$

Označíme-li $r_i^{(k)} := h_{k+1,k}(e_k^T y_i^{(k)}) v_{k+1}$, můžeme psát:

$$Ax_i^{(k)} = \mu_i^{(k)} x_i^{(k)} + r_i^{(k)},$$

kde $r_i^{(k)}$ představuje chybu aproximace vlastního páru.

DEFINICE 5.4: *Ritzův pár*

Nechť $1 \leq k \leq d$ a $(\mu_i^{(k)}, y_i^{(k)})$ je vlastní pár matice H_k. Dvojici $(\mu_i^{(k)}, x_i^{(k)})$, kde $x_i^{(k)} = V_k y_i^{(k)}$, nazveme **Ritzův pár** matice A. Vektor

$$r_i^{(k)} = Ax_i^{(k)} - \mu_i^{(k)} x_i^{(k)} = h_{k+1,k}(e_k^T y_i^{(k)}) v_{k+1}$$

nazveme **chyba Ritzova páru** (nebo Ritzovo reziduum).

Otázka: Jak dobře aproximují Ritzovy páry vlastní páry matice A?

Je-li dosaženo maximální dimenze Krylovova prostoru, tj. $k = d$, pak je tento prostor A-invariantní. Proto také $h_{k+1,k} = 0$ a Ritzův pár je přesně vlastní pár matice A.

Takový počet iterací může ale být velký a navíc v FPA nemusíme A-invariantního prostoru vůbec dosáhnout (zaokrouhlovací chyby). Proto chceme získat rozumnou aproximaci pro $k < d$.

LEMMA 5.7: *Vlastnosti Ritzova rezidua*

Chyba Ritzova páru $r_i^{(k)}$ splňuje:

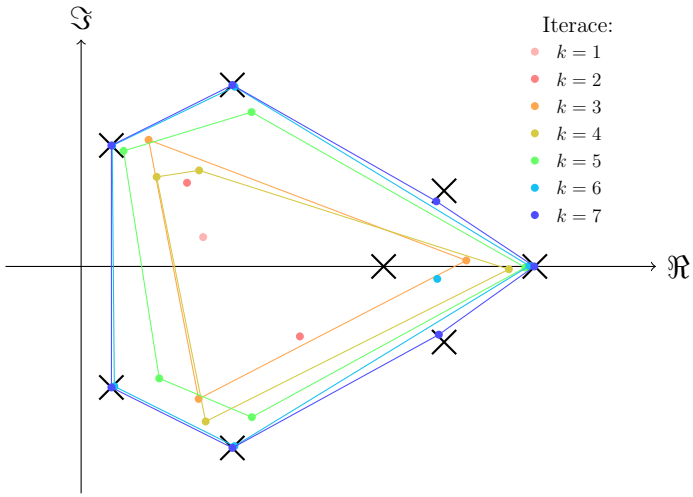
1. $r_i^{(k)} \perp \mathcal{K}_k(A, v)$,
2. $\|r_i^{(k)}\| = h_{k+1,k} |e_k^T y_i^{(k)}|$ (za předpokladu $\|y_i^{(k)}\| = 1$).

Důkaz. 1. Platí $r_i^{(k)} = \underbrace{h_{k+1,k}(e_k^T y_i^{(k)})}_{\in \mathbf{C}} v_{k+1}$. Vektor v_{k+1} je z konstrukce Arnoldiho procesu kolmý na $\mathcal{K}_k(A, v) = \operatorname{span}\{v_1, \ldots, v_k\}$. Tedy i jeho násobek $r_i^{(k)}$ je kolmý na $\mathcal{K}_k(A, v)$.

2. Počítáme normu:
$$\|r_i^{(k)}\| = |h_{k+1,k}(e_k^T y_i^{(k)})| \cdot \|v_{k+1}\|.$$
Protože v_{k+1} je normovaný ($\|v_{k+1}\| = 1$) a $h_{k+1,k} \geq 0$ (norma), dostáváme dokazovanou rovnost. Poznamenejme, že $\|x_i^{(k)}\| = \|V_k y_i^{(k)}\| = \|y_i^{(k)}\| = 1$, protože sloupce V_k jsou ortonormální.

□

Poznámka 5.8 (Vztah Ritzova residua a kvality aproximace spektra)**.** Pokud je norma chyby $\|r_i^{(k)}\|$ malá, pak doufáme, že $(\mu_i^{(k)}, x_i^{(k)})$ je dobrou aproximací vlastního páru matice A. Numerická zkušenost ukazuje, že Ritzova čísla dobře aproximují **okraj** spektra $\operatorname{sp}(A)$ (tj. největší a nejmenší vlastní čísla).



Obrázek 16: Konvergence Ritzových čísel k vlastním číslům matice A. Okraj spektra se aproximuje nejlépe. Vlastní číslo uprostřed spektra se aproximuje špatně.

5.3.1 Shrnutí a speciální případy

Algoritmus 9: Arnoldiho metoda aproximace vlastních párů

Input: $A, v \neq 0, m$
Output: Ritzovy páry
for $k = 1, \ldots, m$ **do**
  proveď 1 přídavný krok Arnoldiho procesu (rozšíření $V_k, H_{k-1} \to V_{k+1}, H_k$);
  spočti vlastní páry $(\mu_i^{(k)}, y_i^{(k)})$ matice H_k pro $i = 1, \ldots, \tilde{k}$;
  spočti Ritzovy vektory $x_i^{(k)} := V_k y_i^{(k)}$ pro $i = 1, \ldots, \tilde{k}$;
end

Speciální případ: A je hermitovská ($A = A^*$) V tomto případě Arnoldiho proces nahradíme Lanczosovou tridiagonalizací, která vede na Lanczosovu metodu aproximace vlastních párů.

Výhody:

- Nižší výpočetní náklady (krátké rekurence).
- Lze získat lepší odhady chyby.

Z Lanczosovy rekurence $AV_k = V_k T_k + \beta_{k+1} v_{k+1} e_k^T$ plyne vztah pro chybu:

$$\|Ax_i^{(k)} - \mu_i^{(k)} x_i^{(k)}\| = \beta_{k+1} |e_k^T y_i^{(k)}|,$$

kde $(\mu_i^{(k)}, y_i^{(k)})$ jsou vlastní páry třídiagonální matice T_k.

LEMMA 5.8: *Vlastnosti Ritzova páru v Lanczosově metodě*

Nechť $A \in \mathbf{C}^{n\times n}$, $A = A^*$. Pak Ritzův pár $(\mu_i^{(k)}, x_i^{(k)})$ spočtený Lanczosovou metodou splňuje, že

$$\operatorname{dist}(\mu_i^{(k)}, \operatorname{sp}(A)) = \min_{\lambda\in\operatorname{sp}(A)} |\lambda - \mu_i^{(k)}| \le \beta_{k+1} \cdot |e_k^T y_i^{(k)}|.$$

Důkaz. Jelikož $A = A^*$, existuje spektrální rozklad $A = UDU^*$, kde $D = \operatorname{diag}(\lambda_1, \ldots, \lambda_n)$ a U je unitární matice. Vyjádříme chybu Ritzova páru:

$$r_i^{(k)} = Ax_i^{(k)} - \mu_i^{(k)} x_i^{(k)} = UDU^* x_i^{(k)} - \mu_i^{(k)} UU^* x_i^{(k)} = U(D - \mu_i^{(k)} I)U^* x_i^{(k)}.$$

Odhadneme normu rezidua (použijeme fakt, že U i U^* zachovávají normu):

$$\|r_i^{(k)}\| = \|U(D - \mu_i^{(k)} I)(U^* x_i^{(k)})\| = \|(D - \mu_i^{(k)} I)x_i^{(k)}\| \ge \min_{j=1,\ldots,n} |\lambda_j - \mu_i^{(k)}| \cdot \|x_i^{(k)}\|.$$

Protože U je unitární a vektory $y_i^{(k)}$ jsou normované, platí:

$$\|U^* x_i^{(k)}\| = \|x_i^{(k)}\| = \|V_k y_i^{(k)}\| = \|y_i^{(k)}\| = 1.$$

Použitím vztahu pro chybu v Lanczosově metodě $\|r_i^{(k)}\| = \beta_{k+1} |e_k^T y_i^{(k)}|$ dostáváme:

$$\min_{\lambda\in\operatorname{sp}(A)} |\lambda - \mu_i^{(k)}| = \min_{j=1,\ldots,n} |\lambda_j - \mu_i^{(k)}| \le \|r_i^{(k)}\| = \beta_{k+1} |e_k^T y_i^{(k)}|.$$

□

Důsledek 5.3 (Ritzovo residuum v Lanczosově metodě)**.** Je-li hodnota $\beta_{k+1}|e_k^T y_i^{(k)}|$ malá, pak existuje vlastní číslo $\lambda \in \operatorname{sp}(A)$, které je dobře aproximováno Ritzovým číslem $\mu_i^{(k)}$. Číslo $\beta_{k+1}|e_k^T y_i^{(k)}|$ udává horní odhad vzdálenosti Ritzova čísla od nejbližšího vlastního čísla matice A.

6 Úplný problém vlastních čísel

Úloha: $A \in \mathbf{C}^{n\times n}$, najdi aproximaci všech vlastních čísel a vlastních vektorů A.

Chceme převést A do tvaru, ze kterého budou vlastní čísla viditelná. Nesmíme však porušit spektrum matice. Jaké transformace zachovávají spektrum?

LEMMA 6.1: *Podobnostní transformace a spektrum*

Nechť $A \in \mathbf{C}^{n\times n}$ a $S \in \mathbf{C}^{n\times n}$ je regulární. Nechť $B = S^{-1}AS$, tj. B vznikne *podobnostní transformací* matice A. Pak A a B mají stejná vlastní čísla.

Důkaz. $A = VJ_AV^{-1}$, kde J_A je Jordanův tvar $A \implies$

$$B = S^{-1}(VJ_AV^{-1})S = (S^{-1}V)J_A(S^{-1}V)^{-1} \implies J_A \text{ je Jord. tvar matice } B.$$

□

Poznámka 6.1 (Vlastní vektor podobné matice)**.** Vlastní vektory se transformují násobením S^{-1}. Pokud v je vlastní vektor A, $S^{-1}v$ je vlastní vektor B.

Problém: $A = \hat{A} + E_A$ ... pochází z praktické úlohy, tedy obsahuje chyby

- $\hat{A}$ ... přesná data
- E_A ... chyby měření

Jak ovlivní transformace S chybu:

$$S^{-1}AS = S^{-1}\hat{A}S + S^{-1}E_AS \implies B = \hat{B} + E_B,$$

kde $E_B = S^{-1}E_AS$ je chyba v B po podobnostní transformaci. Dostaneme odhad

$$\|E_B\| \le \|S^{-1}\| \cdot \|S\| \cdot \|E_A\| = \varkappa(S) \cdot \|E_A\|.$$

Tedy je-li $\varkappa(S) \gg 1$, pak transformace S může velmi zvětšit chybu v datech.

Důsledek: Chci *unitární* podobnostní transformaci A, která vyjeví spektrum.

6.1 Schurova věta

VĚTA 6.2: *Schurova*

Nechť $A \in \mathbf{C}^{n\times n}$. Pak existuje $U \in \mathbf{C}^{n\times n}$ unitární taková, že $R := U^*AU$ je horní trojúhelníková matice. Navíc R má na diagonále vlastní čísla A v libovolném předem předepsaném pořadí.

Důkaz. Indukcí dle n. $n = 1$: triviální.

$n \to n+1$: $A \in \mathbf{C}^{(n+1)\times(n+1)}$. $\lambda :=$ první vlastní číslo v požadovaném pořadí $\implies \exists\, x$: $Ax = \lambda x, \|x\| = 1$. Zvolíme $X \in \mathbf{C}^{(n+1)\times n}$ tak, že $H := [x, X] \in \mathbf{C}^{(n+1)\times(n+1)}$ je unitární (doplníme x na unitární bázi). Pak

$$H^*AH = \begin{bmatrix} x^* \\ X^* \end{bmatrix} A[x, X] = \begin{bmatrix} x^*Ax & x^*AX \\ X^*Ax & X^*AX \end{bmatrix}.$$

Víme, že $Ax = \lambda x$. Tedy:

$$x^*Ax = x^*(\lambda x) = \lambda x^*x = \lambda$$
$$X^*Ax = X^*(\lambda x) = \lambda X^*x = 0 \quad \text{(sloupce } X \text{ jsou kolmé na } x)$$

$$\implies H^*AH = \begin{bmatrix} \lambda & b^* \\ 0 & C \end{bmatrix}, \quad \text{kde } b^* = x^*AX, C = X^*AX.$$

Protože H^*AH je blokově horní trojúhelníková, platí

$$\mathrm{sp}(A) = \mathrm{sp}(H^*AH) = \{\lambda\} \cup \mathrm{sp}(C).$$

Pro $C \in \mathbf{C}^{n\times n}$ dle indukčního předpokladu existuje unitární V taková, že $V^*CV = \tilde{R}$ je horní trojúhelníková (a má na diagonále zbývající vlastní čísla A). Položíme

$$U := [x, XV] = H \cdot \begin{bmatrix} 1 & 0 \\ 0 & V \end{bmatrix}.$$

Matice U je součinem unitárních matic, tedy je unitární. Pak

$$R = U^*AU = \begin{bmatrix} 1 & 0 \\ 0 & V^* \end{bmatrix} H^*AH \begin{bmatrix} 1 & 0 \\ 0 & V \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & V^* \end{bmatrix} \begin{bmatrix} \lambda & b^* \\ 0 & C \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & V \end{bmatrix}$$

$$= \begin{bmatrix} 1 & 0 \\ 0 & V^* \end{bmatrix} \begin{bmatrix} \lambda & b^*V \\ 0 & CV \end{bmatrix} = \begin{bmatrix} \lambda & b^*V \\ 0 & V^*CV \end{bmatrix} = \begin{bmatrix} \lambda & b^*V \\ 0 & \tilde{R} \end{bmatrix}.$$

Výsledná matice je horní trojúhelníková. □

Definice 6.1: *Schurův rozklad*

Rozklad $A = URU^*$ nazveme *Schurův rozklad* matice A.

Poznámka 6.2 (Nejednoznačnost Schurova rozkladu)**.** Schurův rozklad není určen jednoznačně. Například $A = URU^* = (UD)D^*RD(D^*U^*)$ pro libovolnou diagonální unitární matici D (třeba $D = \mathrm{diag}(e^{i\phi_k})$).

Důsledkem Schurovy věty je věta o *unitární diagonalizovatelnosti* (bylo v LA).

Věta 6.3: *Unitární diagonalizovatelnost normálních matic*

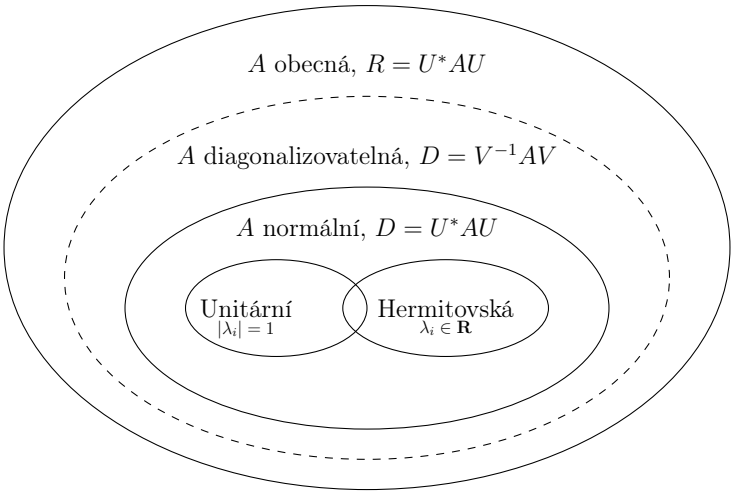
Matice $A \in \mathbf{C}^{n\times n}$ je *normální*, tj. $A^*A = AA^*$, právě tehdy, když existuje $U \in \mathbf{C}^{n\times n}$ unitární taková, že $D = U^*AU$ je diagonální matice. Pak $U = [u_1, \ldots, u_n]$, kde u_i jsou vlastní vektory.

Pro diagonalizovatelnou matici A platí $A = VDV^{-1}$, kde V je báze vlastních vektorů. Schurův rozklad dává $A = URU^*$ s U unitární a R horní trojúhelníkovou.

Mezi V a U existuje následující vztah. Provedeme QR rozklad $V = QR_V$ s unitárním Q a regulární horní trojúhelníkovou R_V. Pak

$$A = VDV^{-1} = (QR_V)D(R_V^{-1}Q^*) = Q(R_VDR_V^{-1})Q^*,$$

tj. $U = Q$, $R = R_VDR_V^{-1}$ je rovněž horní trojúhelníková. Tedy U souvisí s V přes unitární část QR rozkladu.



Obrázek 17: Vztahy mezi třídami matic a jejich rozklady

Schurova věta má teoreticky zajímavý důsledek.

VĚTA 6.4: *Hustota diagonalizovatelných matic*

Množina všech (diagonalizovatelných) matic s navzájem různými vlastními čísly je hustá v $\mathbf{C}^{n\times n}$.

Důkaz. Nechť $A \in \mathbf{C}^{n\times n}$. Pokud A není diagonalizovatelná, pak existuje alespoň jedno vícenásobné vlastní číslo A. Nechť $A = URU^*$ je Schurův rozklad matice A. Zvolme $D_\varepsilon \in \mathbf{C}^{n\times n}$ diagonální tak, aby $\|D_\varepsilon\| < \varepsilon$ a $R_\varepsilon := R + D_\varepsilon$ měla na diagonále navzájem různá čísla. Pak

$$\|A - \underbrace{UR_\varepsilon U^*}_{=:A_\varepsilon}\| = \|UD_\varepsilon U^*\| = \|D_\varepsilon\| < \varepsilon.$$

Tedy pro všechna $\varepsilon > 0$ dost malá existuje diagonalizovatelná matice A_ε taková, že $\|A - A_\varepsilon\| < \varepsilon$. □

Poznámka 6.3 (Perturbace může měnit vlastnosti)**.** Z věty neplyne, že se při návrhu a analýze metod lze omezit na diagonalizovatelné matice, neboť perturbace $A \to A_\varepsilon$ mění vlastnosti A!

Reálný Schurův rozklad Nechť $A \in \mathbf{R}^{n\times n}$.

- Je-li A symetrická, pak existuje její *reálný* Schurův rozklad $D = U^TAU$.
- Obecně nelze uvažovat reálný Schurův rozklad, neboť A může mít komplexní vlastní čísla. Existuje ale rozklad $T = U^TAU$, kde T je blokově horní trojúhelníková:

$$T = \begin{bmatrix} T_{11} & T_{12} & \dots & T_{1k} \\ 0 & T_{22} & \dots & T_{2k} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & T_{kk} \end{bmatrix},$$

  kde reálné diagonální bloky T_{ii} jsou velikosti 1×1 (pro reálná vlastní čísla) nebo 2×2 (jejich vlastní čísla jsou komplexně sdružené páry vlastních čísel A).

VĚTA 6.5: *Abel & Galois & Ruffini*

Kořeny polynomu stupně $n \geq 5$ nelze vyjádřit v radikálech (tj. pomocí operací sčítání, odčítání, násobení, dělení a odmocnin libovolného stupně).

Důsledkem věty je, že kořeny polynomů a tedy ani vlastní čísla obecných matic nelze počítat konečným algoritmem (vlastní čísla jsou kořeny charakteristického polynomu). Proto nemůže existovat metoda pro přesný výpočet Schurova rozkladu a lze jej jen iteračně aproximovat.

6.2 QR algoritmus

= řeší úplný problém vlastních čísel pomocí iterační aproximace Schurova rozkladu.

BÚNO: A je regulární (pro zjednodušení důkazů)

Krok 1: Preprocessing

Cíl: přímou unitární *podobnostní* transformací převést A do tvaru blízkého hornímu trojúhelníkovému (tj. do horního Hessenbergova tvaru).

1. Mějme $A \in \mathbf{C}^{n\times n}$.
2. Zvolme $H_1 = \begin{bmatrix} 1 & 0 \\ 0 & \tilde{H}_1 \end{bmatrix} \in \mathbf{C}^{n\times n}$, kde $\tilde{H}_1$ je Householderova reflexe, která vynuluje prvky v prvním sloupci A pod prvkem a_{21}.
3. $H_1 A$ má nuly v prvním sloupci kromě prvních dvou prvků.
4. Pro zachování podobnosti, aplikujeme H_1^* zprava (dokonce $H_1 = H_1^*$). Pak $H_1 A H_1^*$ má stále požadované nuly v prvním sloupci, protože násobení zprava nemění první sloupec matice (resp. lineární kombinace sloupců 2 až n neovlivní nuly v prvním sloupci vytvořené reflexí $\tilde{H}_1$, která pracuje na souřadnicích 2 až n).

Opakováním tohoto postupu pro další sloupce (H_2 pro druhý sloupec, atd., až do H_{n-2}) dostaneme postupně hledanou transformaci:

$$H_{n-2} \dots H_1 A H_1^* \dots H_{n-2}^* = \begin{bmatrix} \times & \times & \dots & \times & \times \\ \times & \times & \dots & \times & \times \\ 0 & \times & \ddots & \vdots & \times \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & \dots & 0 & \times & \times \end{bmatrix} =: A_0.$$

Máme $A_0 = H^* A H$, kde $H = H_1^* \dots H_{n-2}^*$ je unitární. Matice A_0 je v *horním Hessenbergově tvaru*. Platí $\mathrm{sp}(A_0) = \mathrm{sp}(A)$.

Vlastnosti:

- Výpočetní náročnost: $O(n^3)$.
- Výpočetní stabilita (díky ortogonálním transformacím).

Zbývá se zbavit poddiagonály v matici A_0, což lze pouze iteračně.

Poznámka 6.4 (Hermitovská matice v horním Hessenbergově tvaru)**.** Je-li A hermitovská, pak A_0 vyjde dokonce třídiagonální.

Krok 2: Iterace

Máme $A_0 \in \mathbf{C}^{n\times n}$ v horním Hessenbergově tvaru. → uvažujme posloupnost matic konstruovaných QR iteracemi:

$$\begin{aligned} A_k &= Q_k R_k \quad \text{(spočteme QR rozklad)} \\ A_{k+1} &= R_k Q_k \quad \text{(přenásobíme matice v opačném pořadí)} \end{aligned}$$

pro $k = 0, 1, \ldots$.

LEMMA 6.6: *Vlastnosti QR iterací*

Nechť A je regulární. Pak pro $k = 0, 1, \ldots$:

1. A_k jsou v horním Hessenbergově tvaru,
2. A_k jsou si unitárně podobné, přičemž
$$A_{k+1} = Q_k^* A_k Q_k = P_k^* A_0 P_k,$$
kde $P_k = Q_0 \ldots Q_k$ je unitární matice,
3. A_k mají stejné spektrum.

Důkaz. 1. A_0 regulární $\implies$ A_k, Q_k, R_k regulární pro $k = 0, 1, \ldots$. Indukcí: $k = 0$ z konstrukce. $k \to k+1$:

$$A_k = Q_k R_k \implies Q_k = A_k R_k^{-1}.$$

A_k je horní Hessenbergova, R_k^{-1} je horní trojúhelníková (inverze horní $\triangle$ je horní $\triangle$). Součin horní Hessenbergovy a horní trojúhelníkové matice je horní Hessenbergova matice (rozmyslete si strukturu nul). Pak

$$A_{k+1} = R_k Q_k$$

je součin horní trojúhelníkové a horní Hessenbergovy matice, což je opět horní Hessenbergova matice.

2. Začneme podobností A_k a A_{k+1}. Z iterací

$$A_k = Q_k R_k \implies R_k = Q_k^* A_k,$$

$$A_{k+1} = R_k Q_k = (Q_k^* A_k) Q_k = Q_k^* A_k Q_k.$$

Dále rekurzivně:

$$A_{k+1} = Q_k^* (Q_{k-1}^* A_{k-1} Q_{k-1}) Q_k = \cdots = \underbrace{Q_k^* \ldots Q_0^*}_{P_k^*} A_0 \underbrace{Q_0 \ldots Q_k}_{P_k}.$$

Zde P_k je součin unitárních matic, tedy je unitární, tj. $P_k^* P_k = P_k P_k^* = I$.

3. Plyne z bodu 2 (podobnostní transformace zachovává spektrum).

□

Důsledek 6.1 (Vztahy matic v QR iteracích)**.** Zjevně $A_{k+1} = (HP_k)^* A (HP_k)$, kde HP_k je unitární. Platí $\mathrm{sp}(A_{k+1}) = \mathrm{sp}(A)$ pro $k = 0, 1, \ldots$.

Větu o konvergenci uvedeme bez důkazu. Později dokážeme alespoň slabší tvrzení.

VĚTA 6.7: *Konvergence QR algoritmu*

Nechť $A \in \mathbf{C}^{n\times n}$ a pro její vlastní čísla platí $|\lambda_1| > |\lambda_2| > \cdots > |\lambda_n| > 0$. Pak $A_k \xrightarrow{k\to\infty} R$, kde R je horní trojúhelníková matice. Zároveň $HP_k \xrightarrow{k\to\infty} U$, kde $R = U^* A U$ je Schurův rozklad matice A.

Poznámka 6.5 (Rychlost konvergence QR algoritmu)**.** Dokonce lze dokázat, že poddiagonální prvky $|a^{(k)}_{i+1,i}| \xrightarrow{k\to\infty} 0$ rychlostí $\left|\frac{\lambda_{i+1}}{\lambda_i}\right|^k < 1$ pro $i = 1, \ldots, n-1$. Prvky pod diagonálou konvergují tedy k nule různě rychle.

Algoritmus 10: QR algoritmus

Input: $A \in \mathbf{C}^{n\times n}$, m = max počet iterací
Output: $A_{m+1} \approx R$ (ze Schurova rozkladu)

$A_0 \leftarrow H^* A H$; `// Preprocessing Householderovými reflexemi (`$O(n^3)$`)`
for $k = 0, 1, \ldots, m$ **do**
  $A_k = Q_k R_k$; `// QR rozklad (Givensovy rotace,` $O(n^2)$`)`
  $A_{k+1} = R_k Q_k$; `// Efektivní násobení (`$O(n^2)$`)`

Výpočet QR rozkladu A_k (v horním Hessenbergově tvaru): Pro nulování poddiagonály stačí $n-1$ Givensových rotací.

$$A_k = \begin{bmatrix} \times & \times & \ldots \\ \times & \times & \ldots \\ 0 & \times & \ldots \end{bmatrix} \xrightarrow{(n-1)\text{ Giv. rotací}} \Gamma_k \cdot A_k = \begin{bmatrix} \times & \times & \ldots \\ 0 & \times & \ldots \\ 0 & 0 & \ldots \end{bmatrix} = R_k$$

kde Γ_k je složená rotace. Celková náročnost iterace je tedy jen $O(n^2)$ operací.

Poznámka 6.6 (Výpočetní vlastních vektorů)**.** Pokud chceme aproximovat jen vlastní čísla A, nemusíme ukládat H a Q_k, stačí přepisovat matici A na A_k. V opačném případě budou paměťové a výpočetní náklady vyšší, než je uvedeno v algoritmu.

Celková výpočetní náročnost: Obvykle je třeba cca n iterací k dosažení dostatečně kvalitní aproximace. Proto celkové náklady QR algoritmu odhadujeme jako $O(n^3)$ s větší konstantou u n^3. Algoritmus je tedy výpočetně poměrně nákladný.

Poznámka 6.7 (Výpočetní úspora QR algoritmu pro hermitovské matice)**.** Je-li A hermitovská, pak $A_k \to D$ (diagonální matice) a náklady jsou o řád menší (matice jsou třídiagonální).

Poznámka 6.8 (Vylepšení QR algoritmu)**.** Algoritmus lze vylepšit, aby konvergoval pro všechny $A \in \mathbf{C}^{n\times n}$ (pomocí technik jako shifty, rekurze, deflace).

Proč QR algoritmus konverguje?

- Důkaz konvergenční věty je těžký.
- Vysvětlíme, že za algoritmem se skrývá *mocninná metoda* nad celým prostorem $\mathbf{C}^n$.

LEMMA 6.8: *Mocninná metoda v QR iteracích*

Označme $P_k = Q_0 \dots Q_k$ a $S_k = R_k \dots R_0$. Pak $A_0^{k+1} = P_k S_k$ je QR rozklad matice A_0^{k+1}, tj. P_k je unitární a S_k je horní trojúhelníková.

Důkaz. P_k je součin unitárních matic, tedy je unitární. S_k je součin horních trojúhelníkových matic, tedy je horní trojúhelníková a nutně $S_k = R$. Nyní postupujeme indukcí dle k:

$k = 0$: $A_0 = P_0 S_0 = Q_0 R_0$ (QR rozklad A_0).

$k \to k+1$: $P_k S_k = P_{k-1} Q_k R_k S_{k-1} = P_{k-1} A_k S_{k-1}$. Podle lemmatu 6.6 je $A_k = P_{k-1}^* A_0 P_{k-1}$. Dosadíme

$$P_k S_k = P_{k-1} P_{k-1}^* A_0 P_{k-1} S_{k-1} = I A_0 A_0^k = A_0^{k+1},$$

což dokončuje indukční krok. □

Uvažujme Schurův rozklad $U^* A U = R$ takový, že prvek $a_{11} = \lambda_1$ je dominantní vlastní číslo. Označme $U = [u_1, \dots, u_n]$, pak $A u_1 = \lambda_1 u_1$, tj. první sloupec U je vlastní vektor příslušný k λ_1. (Pozor, ostatní sloupce U obecně nejsou vlastní vektory.)

Místo obecné věty o konvergenci QR algoritmu dokážeme slabší tvrzení, které ukazuje souvislost s mocninnou metodou.

LEMMA 6.9: *Konvergence prvního sloupce*

Nechť platí předpoklady Věty 6.7 (tj. $|\lambda_1| > |\lambda_2| > \dots$) a nechť (λ_1, u_1) je dominantní vlastní pár matice A. Pak platí:

1. $a_{11}^{(k)} := e_1^* A_k e_1 \xrightarrow{k\to\infty} \lambda_1$,
2. $H P_k e_1 \xrightarrow{k\to\infty} u_1$,

v obou případech lineárně s rychlostí $\left|\frac{\lambda_2}{\lambda_1}\right|$.

Důkaz. Z Lemmatu 6.8 víme, že $A_0^{k+1} = P_k S_k$. Podívejme se na aplikaci této rovnosti na první bázový vektor e_1:

$$A_0^{k+1} e_1 = P_k S_k e_1.$$

Matice S_k je horní trojúhelníková, proto $S_k e_1 = s_{11}^{(k)} e_1$, kde $s_{11}^{(k)} = e_1^* S_k e_1 \in \mathbf{C}$ je prvek na diagonále. Tedy

$$A_0^{k+1} e_1 = P_k (s_{11}^{(k)} e_1) = s_{11}^{(k)} P_k e_1 = s_{11}^{(k)} p_1^{(k)},$$

kde $p_1^{(k)} := P_k e_1 \in \mathbf{C}^n$ je první sloupec unitární matice P_k. Protože P_k je unitární, je $\|p_1^{(k)}\| = 1$. Z regularity A plyne regularita R_k i S_k, tedy $s_{11}^{(k)} \neq 0$. Můžeme psát

$$p_1^{(k)} = \frac{1}{s_{11}^{(k)}} A_0^{k+1} e_1.$$

Vidíme, že $p_1^{(k)}$ je normalizovaný vektor, který bychom získali po $k+1$ krocích *mocninné metody* aplikované na matici A_0 se startovacím vektorem e_1.

V kroku 1 (Preprocessing) jsme dostali $A = HA_0H^*$, tedy $A_0 = H^*AH$. Pro mocniny platí $A^{k+1} = HA_0^{k+1}H^*$. Zkoumejme vektor $Hp_1^{(k)}$:

$$Hp_1^{(k)} = \frac{1}{s_{11}^{(k)}} HA_0^{k+1}e_1 = \frac{1}{s_{11}^{(k)}} HA_0^{k+1} \underbrace{H^*H}_{I} e_1 = \frac{1}{s_{11}^{(k)}} A^{k+1}(He_1).$$

Vektor $Hp_1^{(k)}$ tedy odpovídá $k+1$ iteracím mocninné metody na matici A se startovacím vektorem He_1. Protože $|\lambda_1| > |\lambda_2|$, mocninná metoda konverguje k dominantnímu vlastnímu vektoru u_1. Tedy $Hp_1^{(k)} \to u_1$ lineárně s rychlostí $|\lambda_2/\lambda_1|$.

Dále pro první prvek A_k (což je aproximace vlastního čísla) platí:

$$a_{11}^{(k)} = e_1^*A_ke_1 = e_1^*(P_k^*A_0P_k)e_1 = (P_ke_1)^*A_0(P_ke_1) = (p_1^{(k)})^*A_0p_1^{(k)}.$$

Jde o Rayleighův podíl odpovídající vektoru $p_1^{(k)}$ a matici A_0. Protože $p_1^{(k)}$ konverguje k dominantnímu vlastnímu vektoru A_0, konverguje hodnota $a_{11}^{(k)}$ k dominantnímu vlastnímu číslu λ_1. □

Poznámka 6.9 (Rayleighův podíl aproximující dominantní vlastní číslo)**.** Rayleighův podíl odpovídající vektoru $Hp_1^{(k)}$ a matici A je:

$$(Hp_1^{(k)})^*A(Hp_1^{(k)}) = (p_1^{(k)})^*H^*AHp_1^{(k)} = (p_1^{(k)})^*A_0p_1^{(k)} = a_{11}^{(k)}.$$

Tedy $a_{11}^{(k)} \xrightarrow{k\to\infty} \lambda_1$ dle věty o konvergenci mocninné metody lineárně s rychlostí $\left|\frac{\lambda_2}{\lambda_1}\right|$.

6.2.1 Implementační aspekty

Jak víme, poddiagonální prvky A_k konvergují v průběhu iterací k nule různě rychle. Proto může být nějaký prvek $a_{i+1,i}^{(k)}$ velmi blízký nule ($a_{i+1,i}^{(k)} \approx 0$), ale iterace ještě nechceme zastavit kvůli ostatní prvkům. Tuto situaci využíváme k zefektivnění výpočtu.

- **Redukce rozměru:** Pokud $a_{n,n-1}^{(k)} \approx 0$, pak zanedbáme poslední řádek a sloupec matice A_k (který již obsahuje konvergované vlastní číslo $a_{nn}^{(k)}$) a voláme QR algoritmus rekurzivně na menší hlavní podmatici $\tilde{A}_k \in \mathbf{C}^{(n-1)\times(n-1)}$.
- **Deflace:** Pokud $a_{i+1,i}^{(k)} \approx 0$ pro nějaké $i \in \{1, \ldots, n-2\}$, pak je matice A_k blokově horní trojúhelníková. Problém se tedy rozpadá na dva nezávislé podproblémy pro diagonální bloky. Voláme rekurzivně QR algoritmus na dvě podmatice $\tilde{A}_k$ (horní levý blok) a $\hat{A}_k$ (dolní pravý blok).

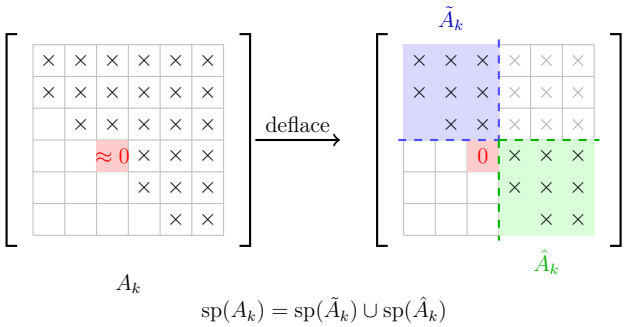
Tento postup vede ke zvýšení efektivity výpočtu.

6.3 QR algoritmus pro SVD

QR algoritmus lze upravit pro aproximaci singulárního rozkladu (SVD).

Úloha: Nechť $A \in \mathbf{C}^{n\times m}$. Hledáme rozklad $A = U\Sigma V^*$, kde U, V jsou unitární a $\Sigma = [\mathrm{diag}(\sigma_1, \ldots, \sigma_r, 0, \ldots, 0)] \in \mathbf{R}^{n\times m}$. Singulární čísla splňují $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0$, kde $r = \mathrm{rank}(A)$.

BÚNO: předpokládejme $n \geq m$. (Jinak matici transponujeme.)



Obrázek 18: Ilustrace deflace v QR algoritmu. Nulový prvek na poddiagonále umožňuje rozdělit problém na menší podproblémy.

Krok 1: Preprocessing

Cíl: převést A do jednoduššího tvaru pomocí unitárních transformací zleva a zprava (nemusí být stejné, protože nepotřebujeme podobnostní transformaci!).

Postupujeme pomocí Householderových reflexí:

1. Zleva aplikujeme H_1, která nuluje prvky v 1. sloupci pod diagonálou.
2. Zprava aplikujeme $\hat{H}_1$, která nuluje prvky v 1. řádku za superdiagonálou (tj. od 3. sloupce dál).
3. Opakujeme pro další sloupce a řádky.

Celkově dostaneme transformaci

$$H_m \dots H_1 A \hat{H}_1 \dots \hat{H}_{m-2} \equiv HA\hat{H} \equiv B \in \mathbf{R}^{n \times m},$$

kde výsledná matice B je v horním bidiagonálním tvaru (má nenulové prvky jen na diagonále a na první superdiagonále). Díky vhodně voleným transformacím je B reálná, protože obsahuje normy redukovaných podvektorů (nulované prvky řádků a sloupců). Protože $H, \hat{H}$ jsou unitární matice (součiny Householderových reflexí), mají A a B stejná singulární čísla.

Vlastnosti:

- Výpočetní náklady: $O(mn^2)$.
- Zpětně stabilní (díky unitárním transformacím).

Krok 2: Iterace

Cíl: iteračně dopočítat singulární čísla matice B. To odpovídá nalezení vlastních čísel matice $B^T B \in \mathbf{R}^{m \times m}$, která je SPD (symetrickou pozitivně definitní) a třídiagonální,

$$B^T B = \begin{bmatrix} \times & \times & 0 & \dots \\ \times & \times & \times & \dots \\ 0 & \times & \times & \dots \\ \vdots & \vdots & \vdots & \ddots \end{bmatrix}.$$

Na tuto matici $A_0 := B^T B$ můžeme aplikovat velmi efektivní symetrický QR algoritmus (již bez preprocessingu). Podle vět o konvergenci QR algoritmu pak

$$A_k \xrightarrow{k\to\infty} \operatorname{diag}(\sigma_1^2, \ldots, \sigma_m^2).$$

Posloupnost A_k konverguje k diagonální matici s vlastními čísly $B^T B$, což jsou druhé mocniny singulárních čísel A.

Vlastnosti:

- Výpočetní náklady iterace: $O(m^2)$ (díky třídiagonálnímu tvaru).
- Zpětná stabilita.

Poznámka 6.10 (Vylepšení QR algoritmu pro SVD)**.** V praxi se nepočítá součin $B^T B$ explicitně, ale pracuje se přímo s maticí B (tzv. Golub-Kahan SVD step). Tím se vyhneme ztrátě přesnosti při výpočtu.

Poznámka 6.11 (Singulární vektory)**.** Singulární vektory (matice U, V) lze dopočítat z transformací $H, \hat{H}$ (z preprocessingu) a z unitárních matic Q_k generovaných v průběhu QR algoritmu.

7 Kořeny nelineárních algebraických rovnic

Úloha Dána funkce $f : [a, b] \to \mathbf{R}$, spojitá. Hledám kořen $x_* \in \mathbf{R}$, tedy bod splňující $f(x_*) = 0$.

Příklad 7.1 ('Neřešitelné' rovnice)**.** Je známé, že polynomiální rovnice jsou řešitelné pomocí radikálů až do stupně 4. Takové vzorce ovšem neexistují od stupně 5 dále. Např. jediný reálný kořen rovnice

$$x^5 - x + 1 = 0$$

nelze vyjádřit konečnou kombinací operací $+, -, *, /, \sqrt[n]{}$. Kořeny obecných polynomiálních rovnic lze sice vyjádřit extrémně složitými speciálními funkcemi (Siegelovy modulární funkce), ale takové vzorce jsou naprosto nepraktické a nepoužitelné. Hromaděním dalších a dalších speciálních funkcí není rozumný přístup k řešení problémů.

7.1 Metoda bisekce (půlení intervalu)

Metoda je založena na větě z Analýzy:

VĚTA 7.1: *Existence řešení*

Nechť f je spojitá a platí $f(a) \cdot f(b) \leq 0$. Potom $\exists x_* \in [a, b]$.

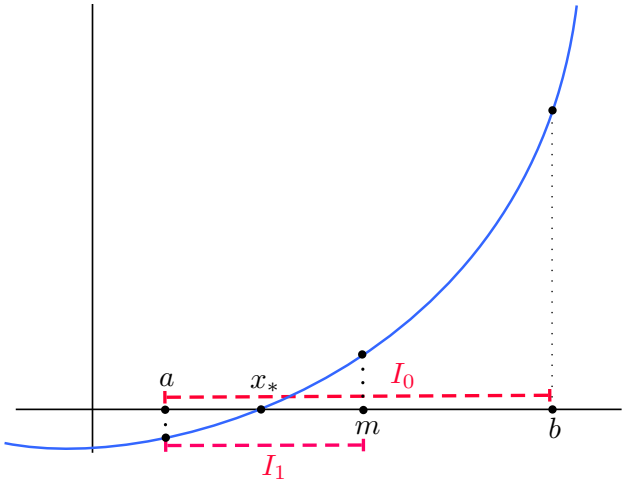
Tuto větu aplikujeme iterativně s tím, že vezmeme střed m aktuálního intervalu a z dvojic a, m a m, b vybereme tu, která splňuje předpoklad věty. Zmenšíme tak počáteční interval na polovinu.

Algoritmus 11: Metoda bisekce

```
Input: f, a, b, Tol
while |b − a| > Tol do
    m = (a+b)/2;
    if f(a) · f(m) < 0 then
        b := m;
    else
        a := m;
    end
end
```

- Generuje posloupnost intervalů $\{I_n\}_{n=0}^{\infty}$, $I_0 = [a, b]$, $I_{n+1} \subset I_n$.
- Platí $|I_{n+1}| = |I_n|/2 \implies |I_n| = \frac{|b-a|}{2^n}$.
- Platí $x_* \in I_n \quad \forall n$.
- Potřebuji $n \geq \log_2 \frac{|b-a|}{\text{Tol}}$ iterací, aby $\frac{|b-a|}{2^n} \leq \text{Tol}$.
- Vždy konverguje, ale pomalu.
- Nejde pro sudé kořeny jako $x^2 = 0$. (x_* musí být kořen liché násobnosti). ALE: kořen sudé násobnosti pro f je kořenem liché násobnosti pro f'.

- Existují zobecnění pro $\mathbf{R}^n$, ale jsou nepraktické. Např. pro $n = 100$ musím testovat $2^{100} \approx 1.26 \cdot 10^{30}$ podintervalů (krychlí v $\mathbf{R}^n$) v každé iteraci, která z nich obsahuje kořen.



Obrázek 19: Metoda půlení intervalu.

7.2 Metoda Regula falsi (false position)

Místo $m = \frac{a+b}{2}$ vezmu v bisekci

$$s = \frac{af(b) - bf(a)}{f(b) - f(a)} \quad \text{místo } m,$$

což je kořen lineární funkce určené body $[a, f(a)]$ a $[b, f(b)]$, viz Obr. 20.

- Typicky $a_n \to x_*$ nebo $b_n \to x_*$, nikoli $|I_n| \to 0$.
- Obecně je rychlejší než bisekce, ojediněle může být i výrazně pomalejší.
- Vždy konverguje.

7.3 Newtonova metoda

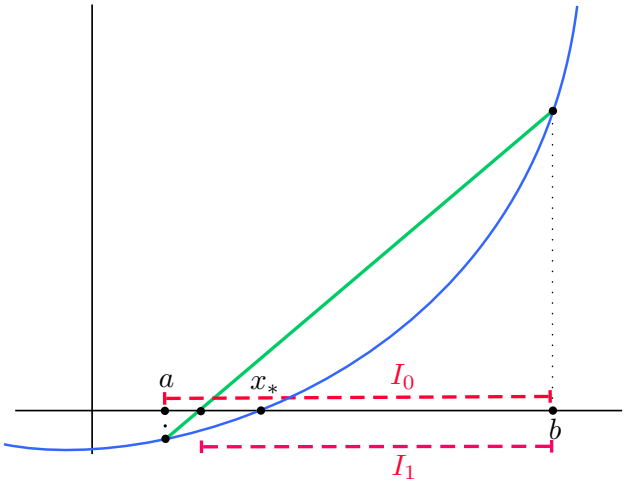
Newtonova metoda buduje posloupnost bodů $x_n \to x_*$. Začneme z počáteční aproximace $x_0 \approx x_*$. Pomocí Taylora (R je zbytek) dostaneme

$$0 = f(x_*) \overset{\text{Taylor}}{=} f(x_0) + f'(x_0)(x_* - x_0) + R$$

Zanedbáním zbytku R a vyjádřením x_* dostaneme

$$0 \approx f(x_0) + f'(x_0)(x_* - x_0) \implies x_* \approx x_0 - \frac{f(x_0)}{f'(x_0)} =: x_1.$$

Postup opakuj.

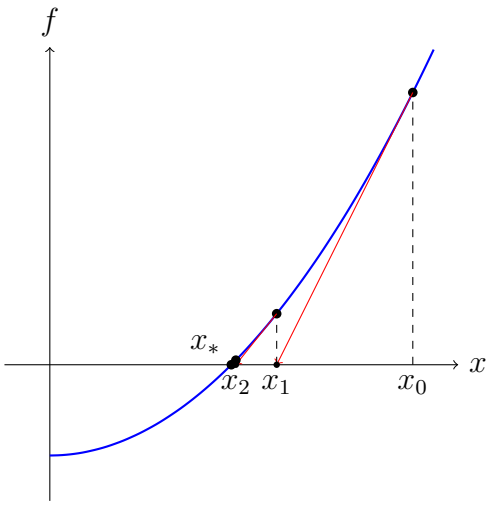


Obrázek 20: Metoda Regula falsi.

Definice 7.1: *Newtonova metoda*

Dáno $x_0 \approx x_*$. Pro $n = 0, 1, \ldots$ iteruj

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$



Obrázek 21: Newtonova metoda

Příklad 7.2 (Babylonská metoda pro výpočet odmocniny)**.** Výpočet $\sqrt{A}$. Řešíme $x^2 - A = 0$.

$$x_{n+1} = x_n - \frac{x_n^2 - A}{2x_n} = \frac{1}{2}\left(x_n + \frac{A}{x_n}\right)$$

Koverguje VELMI rychle, v každé iteraci se zhruba zdvojnásobí počet platných cifer výsledku.

7.3.1 Rychlost konvergence

Mějme posloupnost $\{x_n\}_{n=0}^{\infty}$, $x_n \to x_*$. Zajímá nás chyba v n-té iteraci: $e_n = x_n - x_*$. Platí $e_n \to 0$.

Definice 7.2: *Řád konvergence*

Nechť $\exists p \geq 1, C > 0$ takové, že

$$\lim_{n\to\infty} \frac{|e_{n+1}|}{|e_n|^p} = C,$$

pak řekneme, že $\{x_n\}_{n=0}^{\infty}$ má **řád konvergence** p. Metoda je řádu p, jestliže každá posloupnost získaná touto metodou má řád alespoň p a alespoň jedna má posloupnost má řád přesně p.

Nejčastější případy:

- $p = 1$: *lineární* konvergence (bisekce).
- $p > 1$: *superlineární* (metoda sečen).
- $p = 2$: *kvadratická* (Newton).

Poznámky

- Blízko kořene x_* platí $|e_{n+1}| \sim C|e_n|^p \implies$ pokud $|e_n| \sim 10^{-l} \implies |e_{n+1}| \sim 10^{-pl}$. Tj. v další iteraci mám zhruba p-krát víc platných cifer výsledku (viz Př. 7.2).
- Bisekce je lineárně konvergentní (beru $e_n = |I_n|$ v definici): $|I_{n+1}| = \frac{1}{2}|I_n| \implies \lim_{n\to\infty} \frac{|I_{n+1}|}{|I_n|} = \frac{1}{2}$.

Věta 7.2: *Lokální kvadratická konvergence Newtonovy metody*

Nechť $f \in \mathcal{C}^2[a,b]$, $\exists x_* \in (a,b)$ takové, že $f(x_*) = 0$ a nechť $f'(x_*) \neq 0$. Nechť x_0 je dostatečně blízko x_* a $\{x_n\}_{n=0}^{\infty}$ je posloupnost z Newtonovy metody. Pak $x_n \to x_*$ kvadraticky.

Důkaz. **1)** Nechť $x_n \to x_*$ (tj. $e_n \to 0$), dokážeme, že kvadraticky. Taylorujeme (Lagrangeův tvar zbytku):

$$0 = f(x_*) = f(x_n) + f'(x_n)(x_* - x_n) + \frac{1}{2}f''(\xi_n)(x_* - x_n)^2, \quad \xi_n \text{ mezi } x_n \text{ a } x_*.$$

Vydělíme $f'(x_n)$ a přeuspořádáme:

$$\underbrace{-\frac{f(x_n)}{f'(x_n)} + x_n}_{x_{n+1}} - x_* = \frac{1}{2}\frac{f''(\xi_n)}{f'(x_n)}(x_n - x_*)^2.$$

Všimněme si, že z definice metody je $-\frac{f(x_n)}{f'(x_n)} + x_n = x_{n+1}$ a také $x_{n+1} - x_* = e_{n+1}$. Tedy:

$$e_{n+1} = \frac{1}{2}\frac{f''(\xi_n)}{f'(x_n)} e_n^2 \quad \text{(chybová rovnice)}.$$

Vydělíme e_n^2 a vezmeme limitu:

$$\lim_{n\to\infty} \frac{|e_{n+1}|}{|e_n|^2} = \lim_{n\to\infty} \frac{1}{2}\left|\frac{f''(\xi_n)}{f'(x_n)}\right| = \frac{1}{2}\frac{|f''(x_*)|}{|f'(x_*)|} = C > 0.$$

2) Teď dokážeme konvergenci: $x_n \to x_*$.
Víme $|e_{n+1}| = \frac{1}{2}\left|\frac{f''(\xi_n)}{f'(x_n)}\right| |e_n|^2$. Dále

- $|f''|$ omezená na $[a,b]$ ($f \in \mathcal{C}^2$).
- $f'(x_*) \neq 0$, f' je spojitá $\implies$ $\exists \mathcal{U}$ okolí x_*, kde $|f'| \geq \beta > 0$ (tj. je odražená od nuly).

$\implies \exists M < \infty$ takové, že $\left|\frac{f''(x)}{f'(y)}\right| \leq M$ pro všechna $x, y \in \mathcal{U}$. Pak

$$|e_{n+1}| \leq \frac{1}{2}M|e_n|^2 = \left(\frac{1}{2}M|e_n|\right)|e_n|.$$

Pokud zvolíme $x_0 \in \mathcal{U}$ tak blízko, že $|e_0| = |x_0 - x_*| \leq \frac{1}{M}$, definujeme tím okolí $\mathcal{V}$ bodu x_*. Pak

$$|x_1 - x_*| = |e_1| \leq \frac{1}{2}\underbrace{\left|\frac{f''(\xi_0)}{f'(x_0)}\right|}_{\leq M}\underbrace{|e_0|}_{\leq 1/M}|e_0| \leq \frac{1}{2}|e_0| \implies x_1 \in \mathcal{V}.$$

Jelikož $x_1 \in \mathcal{V}$, máme

$$|x_2 - x_*| = |e_2| \leq \frac{1}{2}M|e_1||e_1| \leq \frac{1}{2}|e_1| \leq \left(\frac{1}{2}\right)^2 |e_0| \implies x_2 \in \mathcal{V}.$$

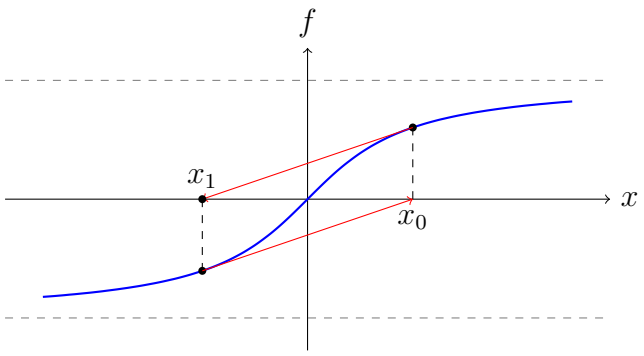
Obecně indukcí: $x_n \in \mathcal{V}$,

$$|e_{n+1}| \leq \frac{1}{2}|e_n| \leq \cdots \leq \left(\frac{1}{2}\right)^n |e_0| \xrightarrow{n\to\infty} 0.$$

□

Problémy s Newtonovou metodou

- **Jen lokální konvergence**: např. $\arctan x = 0$. $\exists x_c$ takové, že pro $x_0 \in (-x_c, x_c)$ metoda konverguje: $x_n \to x_*$. Pro $x_0 = x_c$ se metoda 'zacyklí'. Mimo tento interval diverguje.



- **Zacyklení**: $x^3 - 2x + 2 = 0$ (kořen ≈ -1.77). Zvolíme-li $x_0 = 0$, Newtonova metoda dá posloupnost $0, 1, 0, 1, \ldots$. Navíc existují okolí bodu 0 (a analogické okolí bodu 1) tak, že pokud x_0 je z tohoto okolí, pak x_n konverguje k cyklu {0,1}:

$$\lim_{n\to\infty} x_{2n} = 0, \quad \lim_{n\to\infty} x_{2n+1} = 1$$

  a tyto konvergence jsou dokonce kvadratické. Tedy Newton velmi rychle konverguje k cyklu, ne ke kořeni.

- Obecné zacyklení: Stačí už polynom stupně 4, aby se Newton zacyklil s libovolně velkou periodou:

VĚTA 7.3: *Li-Yorke*

Nechť $f : \mathbf{R} \to \mathbf{R}$ je polynom s aspoň 4 různými kořeny v $\mathbf{R}$. Pak $\forall p \in \mathbf{N}\ \exists x_0$ takové, že posloupnost $\{x_n\}$ generovaná Newtonovou metodou má periodu p.

Zastavovací kritéria Testujeme, kdy iteraci ukončit.

- **Absolutní kritérium (konvergence posloupnosti):** $|x_{n+1} - x_n| < \varepsilon$.
- **Relativní kritérium:** $\frac{|x_{n+1}-x_n|}{|x_n|} < \varepsilon$.
- **Reziduum (splnění rovnice):** $|f(x_n)| < \varepsilon$.

7.4 Modifikace Newtonovy metody

Někdy nemáme k dispozici derivaci $f'(x)$ (např. f dáno implementací a ne vzorcem) nebo je její výpočet drahý. Aproximujeme derivaci diferencí ($f'(x_n) \approx d_n$):

7.4.1 Newton s diferencemi

Zvol h_n dost malé, iteruj

$$d_n := \frac{f(x_n + h_n) - f(x_n)}{h_n}, \quad x_{n+1} := x_n - \frac{f(x_n)}{d_n}.$$

- Platí $|f'(x_n) - d_n| = O(h_n)$.
- Pokud $h_n = h$ (konstantní), dostáváme lineární konvergenci $x_n \to x_*$.
- Pokud $h_n \to 0$, dostaneme superlineární konvergenci.

7.4.2 Metoda sečen

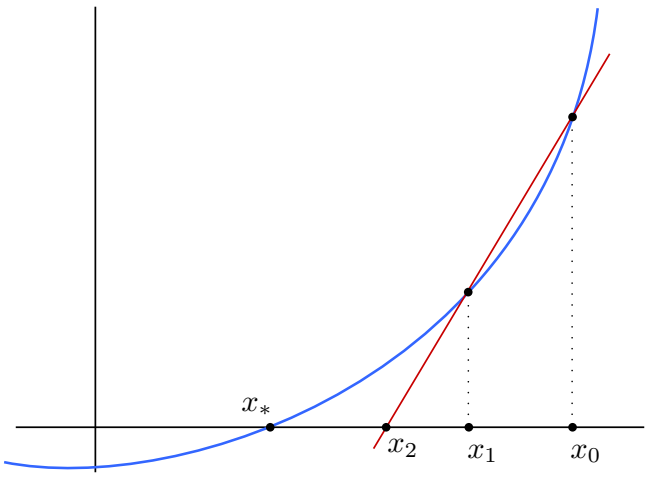
Volíme $h_n = x_n - x_{n-1}$. Derivaci tedy nahradíme směrnicí sečny body $[x_{n-1}, f(x_{n-1})]$ a $[x_n, f(x_n)]$:

$$d_n = \frac{f(x_n) - f(x_{n-1})}{x_n - x_{n-1}}.$$

Iterační předpis:

$$x_{n+1} = x_n - \frac{f(x_n)}{d_n} = x_n - f(x_n)\frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})} = \frac{x_{n-1}f(x_n) - x_n f(x_{n-1})}{f(x_n) - f(x_{n-1})}.$$

- Řád konvergence je $\frac{1+\sqrt{5}}{2} \approx 1.618034\ldots$ (zlatý řez).
- V každé iteraci počítáme jen $f(x_n)$, hodnotu $f(x_{n-1})$ si pamatujeme. Tedy sečny jsou dvakrát efektivnější než Newton či Newton s diferencemi, kde potřebuji dvě vyhodnocení f (resp. f a f') za iteraci.



Obrázek 22: Metoda sečen.

8 Iterace funkcí

Nyní zaujmeme poněkud obecnější pohled. Nechť $f(x_*) = 0$. Chceme najít kořen x_* pomocí iteračního procesu $x_{n+1} = g(x_n)$. Předpokládejme, že $x_n \to x_*$ a funkce g je spojitá. Pak platí:

$$x_* = \lim_{n\to\infty} x_{n+1} = \lim_{n\to\infty} g(x_n) \implies x_* = g(x_*)$$

Tedy pokud je x_* **kořenem** f, pak je **pevným bodem** funkce g. Tedy z hlediska iterace je přirozené dívat se na pevné body.

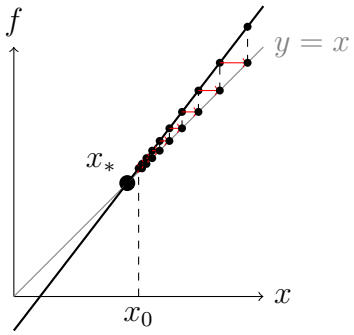
8.1 Metoda prosté iterace

Zvolme tedy funkci $g : \mathbf{R} \to \mathbf{R}$ (získanou např. Newtonovou metodou) a dané x_0 a iterujme:

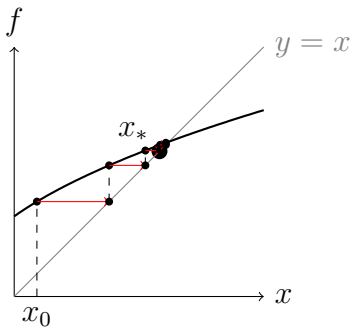
$$x_{n+1} = g(x_n).$$

Otázka je, kdy bude $x_n \to x_*$.

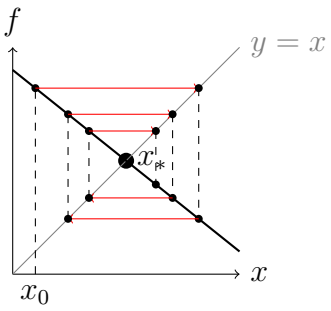
Příklad 8.1 (Iterace $\cos(x)$)**.** Pokud na kalkulačce zadáme libovolné číslo a opakovaně mačkáme funkci `cos`, generujeme posloupnost $x_{n+1} = \cos(x_n)$. Tato posloupnost konverguje k číslu $x_* \approx 0.739$, což je řešení rovnice $\cos x = x$.



Obrázek 23: Divergence ($|g'| > 1$)



Obrázek 24: Monotónní konvergence ($0 < g' < 1$)



Obrázek 25: Oscilující konvergence ($-1 < g' < 0$)

Obrázek 26: Pavučinové diagramy (Cobweb plots)

Příklad 8.2 (Výběr iterační funkce)**.** Řešíme rovnici $f(x) = e^x - 2x - 1 = 0$. Zřejmě $f(0) = 0$. Hledáme další kořen x_*. Jelikož $f(1) = e - 3 < 0$ a $f(2) = e^2 - 5 > 0$, existuje kořen $x_* \in (1, 2)$. Chceme převést $f(x) = 0$ na $x = g(x)$ tak, aby metoda konvergovala (např. pro $x_0 = 1.5$).

Je nekonečně možností, jak $f(x) = 0$ transformovat na $g(x) = x$. Uvažujme následující čtyři volby $g(x)$:

1. $e^x - x - 1 = x \implies g_1(x) = e^x - x - 1$. Derivace $g_1'(x) = e^x - 1$. Pro $x \in (1, 2)$ je $g_1'(x) > e - 1 \approx 1.7 > 1$. Metoda diverguje.

2. $\ln(2x + 1) = x \implies g_2(x) = \ln(2x + 1)$. Derivace $g_2'(x) = \frac{2}{2x+1}$. Pro $x \approx 1.25$ je $g_2'(x) \approx 0.57 < 1$. Metoda konverguje (lineárně).

3. $\frac{e^x-1}{2} = x \implies g_3(x) = \frac{e^x-1}{2}$. Derivace $g_3'(x) = \frac{e^x}{2}$. Pro $x > 1$ je $g_3'(x) > \frac{e}{2} \approx 1.35 > 1$. Diverguje.

4. $\frac{xe^x-e^x+1}{e^x-2} = x \implies g_3(x) = \frac{xe^x-e^x+1}{e^x-2}$. Metoda konverguje kvadraticky. Tato varianta je Newtonovou metodou pro $f(x) = 0$.

Pozorování: Pokud $g \in \mathcal{C}^1$ a $|g'(x)| < 1$ na okolí bodu x_*, pak iterace $x_n \to x_*$. Toto lze ospravedlnit teoreticky:

DEFINICE 8.1: *Kontrakce*

Funkce $g : [a, b] \to \mathbf{R}$ je **kontrakce** (kontrahující zobrazení), pokud existuje konstanta $L < 1$ taková, že:

$$\forall x, y \in [a, b] : |g(x) - g(y)| \leq L|x - y|$$

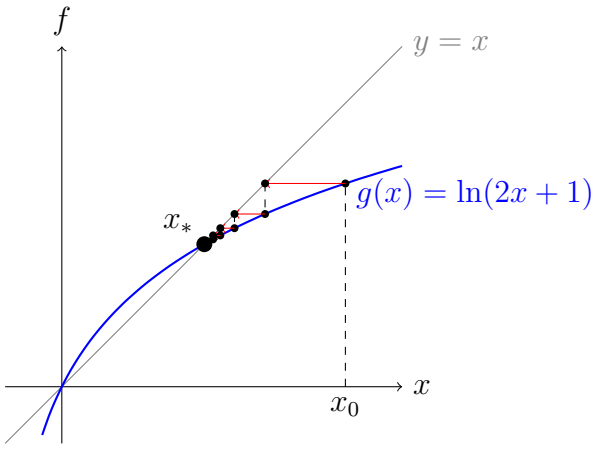
(Funkce g je lipschitzovská s konstantou $L < 1$.)

VĚTA 8.1: *Banachova věta o kontrakci*

Nechť $g : [a, b] \to [a, b]$ je kontrakce s konstantou lipschitzovskosti L<1. Pak existuje právě jeden pevný bod $x_* \in [a, b]$ (tj. $g(x_*) = x_*$). Dále pro libovolnou počáteční aproximaci $x_0 \in [a, b]$ platí, že posloupnost definovaná $x_{n+1} = g(x_n)$ konverguje k x_*, tj. $x_n \to x_*$. Platí odhad chyby:

$$|x_n - x_*| \leq \frac{L^n}{1 - L}|x_1 - x_0| \to 0 \quad \text{pro } n \to \infty$$

(Důkaz: viz přednáška z Matematické analýzy)

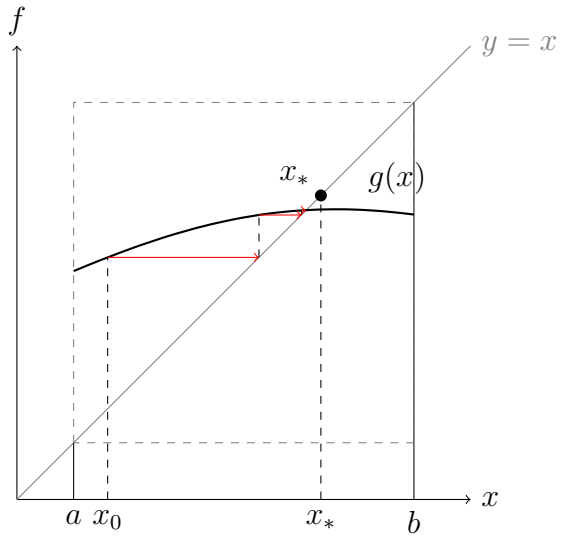


Obrázek 27: Iterace funkce $g(x) = \ln(2x + 1)$

Poznámka 8.1 (Postačující podmínka pro kontrakci)**.** Podmínka $|g'| < 1$ na intervalu $[a, b]$ zajišťuje, že g je kontrakce. Skutečně: Lagrangeova věta o střední hodnotě dá

$$|g(x) - g(y)| = |g'(\xi)| \cdot |x - y| \leq \big(\max_{\eta \in [a,b]} |g'(\eta)|\big)|x - y| = L|x - y|.$$

Pokud $L < 1$, jde o kontrakci.



Obrázek 28: Princip kontrakce a konvergence k pevnému bodu

Poznámka 8.2 (**Lineární konvergence** metody prosté iterace)**.** Nejlepší konvergence, kterou můžeme u metody prosté iterace obecně očekávat, je lineární:

LEMMA 8.2: *Lineární konvergence prosté iterace*

Nechť $g \in \mathcal{C}^1[a,b]$ a nechť $0 < |g'| < 1$ na $[a,b]$. Pak pro všechna $x_0 \in [a,b]$ konverguje $x_n \to x_*$ lineárně.

Důkaz. Dle předchozí poznámky je g kontrakce, tudíž $x_n \to x_*$. Z věty o střední hodnotě máme

$$|x_{n+1} - x_*| = |g(x_n) - g(x_*)| = |g'(\xi_n)(x_n - x_*)|,$$

tudíž

$$\lim_{n\to\infty} \frac{|x_{n+1} - x_*|}{|x_n - x_*|} = \lim_{n\to\infty} |g'(\xi_n)| = |g'(x_*)| > 0,$$

jelikož $\xi_n \to x_*$ dle 'lemmatu o dvou policajtech'. □

8.2 Newtonova metoda v $\mathbf{R}^N$

Konvergence a vztah k prosté iteraci Newtonovu metodu lze chápat jako metodu prosté iterace $x_{n+1} = g(x_n)$, kde iterační funkce g má tvar

$$g(x) = x - \frac{f(x)}{f'(x)}.$$

Newtonova metoda je vlastně návod, jak pro danou rovnici $f(x) = 0$ zkonstruovat funkci g, aby metoda rychle konvergovala. Spočtěme derivaci $g(x)$ (předpokládáme $f(x_*) = 0$):

$$g'(x) = 1 - \frac{(f'(x))^2 - f(x)f''(x)}{(f'(x))^2} = \frac{f(x)f''(x)}{(f'(x))^2}.$$

V kořeni x_* platí $f(x_*) = 0$, a tedy:

$$g'(x_*) = \frac{0 \cdot f''(x_*)}{(f'(x_*))^2} = 0$$

- Vidíme tedy, že $|g'(x)| < 1$ na okolí bodu x_*, takže metoda konverguje.
- Lze dokázat, že pokud $g \in \mathcal{C}^2[a,b]$ a $g'(x_*) = 0$, pak metoda konverguje **kvadraticky** (řád konvergence 2).

Díky vlastnosti $g'(x_*) = 0$ tedy Newtonova metoda (pokud konverguje) konverguje velmi rychle: Počet platných cifer se v každém kroku přibližně zdvojnásobuje.

Newtonova metoda pro soustavy rovnic Mějme soustavu n nelineárních rovnic o n neznámých:

$$F : \mathbf{R}^N \to \mathbf{R}^N, \quad x \in \mathbf{R}^N, \quad x = (x_1, \ldots, x_N)^T$$

$$F(x) = \begin{pmatrix} f_1(x_1, \ldots, x_N) \\ \vdots \\ f_n(x_1, \ldots, x_N) \end{pmatrix} = \begin{pmatrix} 0 \\ \vdots \\ 0 \end{pmatrix}$$

. Použijeme Taylorův rozvoj prvního řádu pro vektorovou funkci F v okolí aktuální aproximace x_n:

$$0 = F(x_*) \approx F(x_n) + J_F(x_n)(x_* - x_n), \tag{1}$$

kde $J_F(x_n)$ je **Jacobiho matice** funkce F v bodě x_n:

$$\big(J_F(x)\big)_{ij} = \frac{\partial f_i}{\partial x_j}(x).$$

Pokud je $J_F(x_n)$ invertovatelná, pak z lineární aproximace (1) vyjádříme hledaný kořen x_* a tuto aproximaci definujeme jako novou iteraci x_{n+1} v **Newtonově metodě**:

$$x_* \approx x_{n+1} = x_n - [J_F(x_n)]^{-1} F(x_n). \tag{2}$$

Poznámka 8.3 (Inverze Jacobiho matice v praxi)**.** Výpočet (dokonce i aproximace) inverzní matice je drahý. V praxi tedy přeformulujeme Newtonovu metodu (2) v každé iteraci jako řešení soustavy lineárních rovnic pro **korekci** δ_n:

$$J_F(x_n)\delta_n = -F(x_n) \implies x_{n+1} = x_n + \delta_n$$

Výpočetní náklady na jeden krok rostou s dimenzí prostoru N (např. $O(N^3)$ pro Gaussovu eliminaci). Pro rozsáhlé simulace (např. diskretizace parciálních diferenciálních rovnic), kde $N \sim 10^6$ či mnohem více, je nutné využívat speciálních vlastností matic (řídkost) a iterativních řešičů lineárních soustav.

9 Optimalizace

Úloha Hledám minimum funkce (funkcionálu), resp. chci bod, kde se minimum nabývá. Nechť $f : \mathbf{R}^N \to \mathbf{R}$. Hledáme $\bar{x} = \operatorname{argmin}_{x \in M} f(x)$, kde $M \subset \mathbf{R}^N$. Tj. hledáme bod $\bar{x} \in M$ takový, že

$$f(\bar{x}) \leq f(x) \quad \forall x \in M.$$

Rozlišujeme dva základní případy podle množiny M:

- $M = \mathbf{R}^N$: nepodmíněná optimalizace.
- $M \subset \mathbf{R}^N$: podmíněná optimalizace (s vazbami).

DEFINICE 9.1: *Lokální a globální minimum*

Bod $\bar{x} \in M$ se nazývá:

- **Lokální minimum**, pokud $f(\bar{x}) \leq f(x)$ pro všechna $x \in M \cap \mathcal{U}$, kde $\mathcal{U}$ je nějaké okolí bodu $\bar{x}$.
- **Globální minimum**, pokud $f(\bar{x}) \leq f(x)$ pro všechna $x \in M$.

Globální minimum je velmi těžké najít, často nám stačí (nebo najdeme) právě to lokální.

Příklady

- **Metoda nejmenších čtverců**:

$$\min_{x \in \mathbf{R}^N} \|Ax - b\|$$

  Použití: např. naladit parametry modelu.

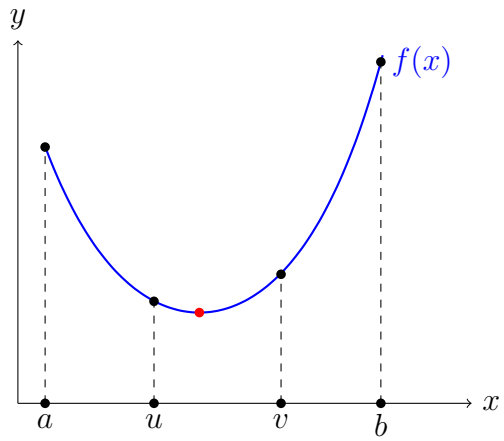
- **Machine Learning, AI, neuronové sítě**: Trénování sítě je optimalizační proces (minimalizace *cost function*, chybové funkce). Často se používají variace na metodu největšího spádu (gradient descent). Dimenze prostoru $\mathbf{R}^N$ je velmi velká, např. pro ChatPGT je řádově $N \sim 10^{11}$.

- Poznámka (historická): Yitang Zhang (2013) dokázal, že existuje nekonečně mnoho dvojic prvočísel p_{n+1}, p_n takových, že $p_{n+1} - p_n \leq K$ (pro $K = 7 \cdot 10^7$). Numerickou optimalizací 'mazané' funkce uvnitř důkazu se tato mez snížila na 246 (projekt Polymath). Jde o první významný pokrok v tzv. problému prvočíselných dvojčat.

9.1 Minimalizace v R - Metoda zlatého řezu

Metoda zlatého řezu (analogie bisekce, tj. levná, vždy konvergující, ale velmi pomalá metoda). Pro začátek předpokládejme, že funkce je *unimodální* na intervalu $[a, b]$ (tj. že má na intervalu právě jedno minimum, nalevo od něj je f klesající a napravo rostoucí.)

Máme body $a < u < v < b$. Pokud $f(u) < f(v)$, pak lze snadno nahlédnout, že minimum nutně leží v intervalu $[a, v]$, viz Obr. 29. V opačném případě ($f(u) > f(v)$) leží minimum v $[u, b]$.



Obrázek 29: Metoda zlatého řezu pro unimodální funkci

Obdobně jako v metodě půlení intervalu jsme schopni původní interval $[a, b]$ zmenšit na nový interval ($[a, v]$ nebo $[u, b]$), který stále obsahuje minimum, a to na základě porovnání funkčních hodnot $f(u)$ a $f(v)$. Tento postup můžeme iterativně opakovat:

Algoritmus 12: Zúžení intervalu obsahující minimum

Vstup : Funkce f, interval $[a_n, b_n]$, body $u_n, v_n \in (a_n, b_n)$ s $u_n < v_n$.
Výstup : Nový interval $[a_{n+1}, b_{n+1}]$ obsahující minimum.

```
if f(u_n) ≤ f(v_n) then
    a_{n+1} := a_n;
    b_{n+1} := v_n;
else
    a_{n+1} := u_n;
    b_{n+1} := b_n;
```

Algoritmus tedy generuje posloupnost do sebe vnořených zmenšujících se intervalů, stejně jako bisekce.

Poznámka 9.1 (Ne-unimodální funkce)**.** Pokud f **není unimodální**, pak algoritmus také funguje. V tomto případě má f více minim a uvedený algoritmus zkonverguje k jednomu z nich. Je to obdobné, jako bisekce aplikovaná na funkci s více kořeny: bisekce zkonverguje pouze k jednomu z nich.

Výběr dělících bodů pro zúžení intervalu Chtěli bychom, aby volba bodů u, v byla efektivní v tom smyslu, abychom museli f vyhodnocovat co nejméně krát. To znamená, že bychom ideálně chtěli v další iteraci znovu použít jeden z jichž napočítaných bodů a jeho již vypočtenou funkční hodnotu. Chceme tedy, aby relativní pozice bodů v intervalu zůstala zachována, viz Obr. 30.

Nechť jsou body u, v definovány pomocí parametru $\rho \in (0, 1/2)$ takto:

$$u := a + \rho(b - a),$$

$$v := b - \rho(b - a).$$

Po zúžení intervalu $[a, b]$ na $[a', b'] = [a, v]$ se starý bod u stane novým bodem v', viz Obr. 30. Takže chceme, aby u měl v $[a, v]$ stejnou relativní pozici, jako měl v v $[a, b]$. Tedy:

$$\frac{u - a}{v - a} = \frac{v - a}{b - a}$$

Po dosazení a úpravě dostáváme:

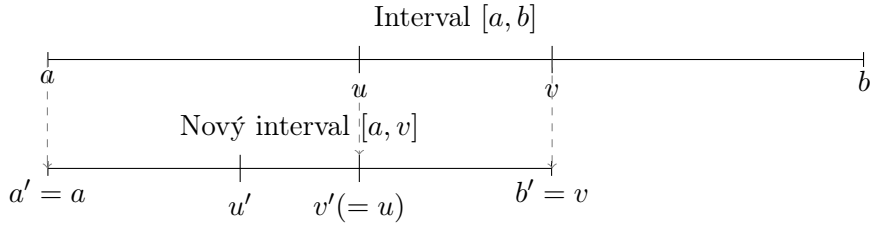
$$\frac{\rho(b-a)}{b-\rho(b-a)-a} = \frac{b-\rho(b-a)-a}{b-a}$$

$$\frac{\rho}{1-\rho} = 1-\rho \implies \rho = (1-\rho)^2$$

To vede na kvadratickou rovnici $\rho^2 - 3\rho + 1 = 0$, jejíž řešení v intervalu $(0,1)$ je

$$\rho = \frac{3-\sqrt{5}}{2} \approx 0.382.$$

Poměr $1-\rho = \frac{\sqrt{5}-1}{2}$ je známý jako **zlatý řez**. Pokud v Algoritmu 12 volíme u_n, v_n dle tohoto poměru ρ, dostáváme **metodu zlatého řezu**.



Obrázek 30: Opakované využití dělícího bodu při metodě zlatého řezu

9.2 Newtonova metoda pro minimalizaci

Metoda zlatého řezu je velmi pomalá (lineární konvergence). Výrazně rychlejší metodou je Newtonova metoda pro hledání lokálního minimum funkce. Je založena na využití více informací o f než jen funkčních hodnot, a sice derivací f.

Myšlenka Dáno $x_n \approx \bar{x}$. Aproximujeme funkci f v okolí bodu x_n kvadratickou funkcí p (Taylorův polynom 2. stupně) a místo minima f hledáme minimum p:

$$x_{n+1} := \operatorname*{argmin}_{x \in \mathbf{R}} p(x).$$

Předpokládejme, že $f \in \mathcal{C}^2(I)$. Taylorův rozvoj v bodě x_n je:

$$f(x) \approx p(x) = f(x_n) + f'(x_n)(x-x_n) + \frac{1}{2}f''(x_n)(x-x_n)^2.$$

Minimum paraboly $p(x)$ najdeme řešením rovnice $p'(x) = 0$:

$$p'(x) = f'(x_n) + f''(x_n)(x-x_n) = 0.$$

Řešení definujeme jako další iteraci x_{n+1}:

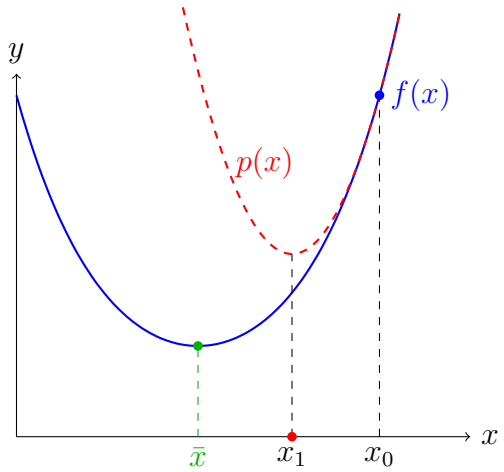
$$x_{n+1} = x_n - \frac{f'(x_n)}{f''(x_n)}.$$

Definice 9.2: *Newtonova metoda pro minimalizaci*

Dáno $x_0 \approx \bar{x}$. Pro $n = 0, 1, 2, \ldots$ iteruj

$$x_{n+1} = x_n - \frac{f'(x_n)}{f''(x_n)}.$$

Poznamenejme, že toto je ekvivalentní použití klasické Newtonovy metody pro hledání kořene rovnice $f'(x) = 0$, což je nutná podmínka pro extrém.



Obrázek 31: Jeden krok Newtonovy metody pro minimalizaci. Funkce f je aproximována parabolou p v bodě x_0. Minimum paraboly, x_1, je novou aproximací minima funkce f.

9.3 Optimalizace v $\mathbf{R}^N$

Nechť $f : \mathbf{R}^N \to \mathbf{R}$, opět hledáme bod minima $\bar{x}$.

Věta 9.1: *Podmínky pro extrém v* $\mathbf{R}^N$

- **Nutná podmínka:** Nechť $\bar{x}$ je bod lokálního minima funkce f a nechť v tomto bodě existuje gradient $\nabla f(\bar{x})$. Potom platí $\nabla f(\bar{x}) = \vec{0}$.
- **Postačující podmínka:** Nechť $f \in \mathcal{C}^2(M)$ na nějakém okolí M bodu $\bar{x}$. Pokud $\nabla f(\bar{x}) = \vec{0}$ a Hessova matice $H_f(\bar{x})$ je pozitivně definitní, pak $\bar{x}$ je bodem lokálního minima.

Připomeňme si definici gradientu a Hessovy matice:

$$\nabla f(x) = \begin{pmatrix} \frac{\partial f}{\partial x_1} \\ \vdots \\ \frac{\partial f}{\partial x_N} \end{pmatrix}, \quad H_f(x) = \begin{pmatrix} \frac{\partial^2 f}{\partial x_1^2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_N} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_N \partial x_1} & \cdots & \frac{\partial^2 f}{\partial x_N^2} \end{pmatrix}$$

9.3.1 Metody spádových směrů

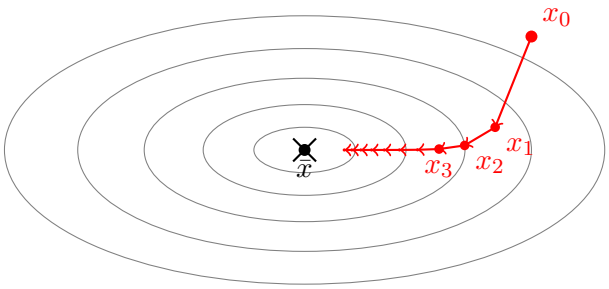
Zvolíme počáteční aproximaci $x_0 \in \mathbf{R}^N$ a konstruujeme posloupnost $\{x_n\}_{n \in \mathbf{N}}$ s cílem $x_n \to \bar{x}$, kde $\bar{x}$ je hledané minimum.

Definice 9.3: *Metoda spádového směru*

Dáno $x_0 \approx \bar{x}$. Pro $n = 0, 1, 2, \ldots$ iteruj

$$x_{n+1} = x_n + \alpha_n d_n,$$

kde $d_n \in \mathbf{R}^N$ je **směr sestupu** a $\alpha_n \in \mathbf{R}^+$ je **délka kroku**. Tyto parametry volíme tak, aby platilo $f(x_{n+1}) < f(x_n)$.



Obrázek 32: Iterační metoda postupující přes vrstevnice k minimu.

Definice 9.4: *Spádový směr*

Nechť $x \in \mathbf{R}^N$. Směr $d \in \mathbf{R}^N$ se nazývá *spádový* v bodě x, pokud existuje $\bar{\alpha} > 0$ takové, že

$$f(x + \alpha d) < f(x) \quad \forall \alpha \in (0, \bar{\alpha}].$$

Lemma 9.2: *Charakterizace spádového směru*

Nechť existuje $\nabla f(x)$. Pak d je spádový směr v bodě x právě tehdy, když $\nabla f(x) \cdot d < 0$.

Důkaz. Definujme pomocnou funkci jedné proměnné $g(\alpha) = f(x + \alpha d)$ pro $\alpha \geq 0$. Derivace g v bodě 0 odpovídá derivaci f ve směru d:

$$g'(0) = \lim_{\alpha \to 0^+} \frac{g(\alpha) - g(0)}{\alpha} = \lim_{\alpha \to 0^+} \frac{f(x + \alpha d) - f(x)}{\alpha} = \nabla f(x) \cdot d.$$

Pokud platí $\nabla f(x) \cdot d < 0$, je $g'(0) < 0$. To znamená, že funkce g je v pravém okolí bodu 0 klesající. Existuje tedy $\bar{\alpha} > 0$ tak, že pro všechna $\alpha \in (0, \bar{\alpha}]$ je $g(\alpha) < g(0)$, což je ekvivalentní podmínce $f(x + \alpha d) < f(x)$. □

Lemma 9.3: *Směr největšího spádu*

Směr největšího spádu funkce f v bodě x je směr $-\nabla f(x)$.

Důkaz. Hledáme směr d (BÚNO s normou $\|d\| = 1$) takový, aby derivace f ve směru d, tedy $\nabla f(x) \cdot d$, byla co nejvíce záporná. Z Cauchy-Schwarzovy nerovnosti plyne:

$$|\nabla f(x) \cdot d| \leq \|\nabla f(x)\| \cdot \|d\|.$$

Protože $\|d\| = 1$, máme tedy $-\|\nabla f(x)\| \leq \nabla f(x) \cdot d \leq \|\nabla f(x)\|$. Minimální hodnota $-\|\nabla f(x)\|$ se nabývá, právě když je směr d opačný ke směru $\nabla f(x)$, tedy

$$d = -\frac{\nabla f(x)}{\|\nabla f(x)\|}.$$

Pro tuto volbu totiž dostáváme:

$$\nabla f(x) \cdot d = \nabla f(x) \cdot \left(-\frac{\nabla f(x)}{\|\nabla f(x)\|}\right) = -\frac{\|\nabla f(x)\|^2}{\|\nabla f(x)\|} = -\|\nabla f(x)\|.$$

□

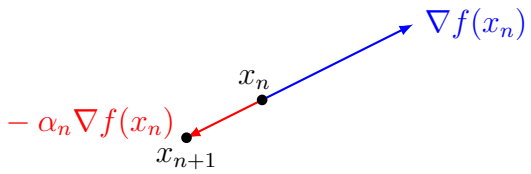
Metoda největšího spádu Jedná se o nejjednodušší metodu spádových směrů, kde za směr sestupu d_n volíme směr největšího spádu, tedy $d_n = -\nabla f(x_n)$.

DEFINICE 9.5: *Metoda největšího spádu*

Dáno $x_0 \approx \bar{x}$. Pro $n = 0, 1, 2, \ldots$ iteruj

$$x_{n+1} = x_n - \alpha_n \nabla f(x_n),$$

kde $\alpha_n \in \mathbf{R}^+$ je takové, aby platilo $f(x_{n+1}) < f(x_n)$.



Obrázek 33: Jeden krok metody největšího spádu.

Volba délky kroku α_n Klíčovým prvkem metody je volba délky kroku α_n. Existuje mnoho strategií, zde je několik příkladů:

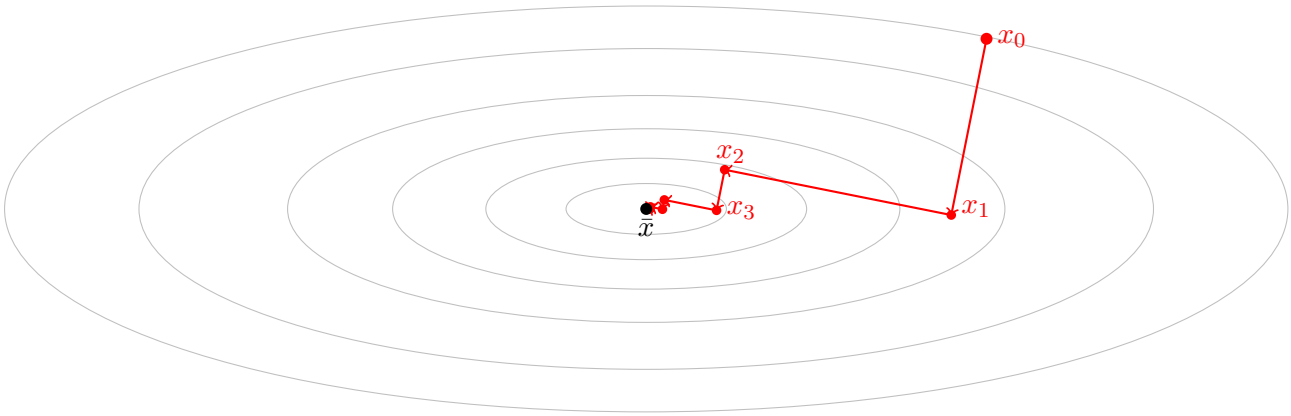
- **Konstantní krok:** $\alpha_n = \alpha$ pro dostatečně malé zafixované $\alpha > 0$. Pokud je α příliš velké, metoda může divergovat.
- **Zmenšující se krok:** Například $\alpha_n \to 0$ pro $n \to \infty$. Aby metoda konvergovala k minimu, musí kroky klesat dostatečně pomalu: lze dokázat, že pro konvergenci musí nutně platit $\sum_{n=0}^{\infty} \alpha_n = \infty$ (metoda má šanci 'dojít' libovolně daleko). Zároveň se předpokládá $\sum_{n=0}^{\infty} \alpha_n^2 < \infty$ (kroky se zkracují dostatečně rychle, čímž se uhlídá akumulovaná chyba). Vhodnou volbou je tedy např. $\alpha_n = 1/(n+1)$.
- **Line search:** Krok hledáme jako (přibližné) minimum funkce jedné proměnné $g(\alpha) = f(x_n - \alpha \nabla f(x_n))$. To může být výpočetně náročné.
  - **Přesný line search (exact line search):** Volíme α_n tak, aby minimalizovalo funkci $g(\alpha)$:

$$\alpha_n = \operatorname*{argmin}_{\alpha > 0} g(\alpha).$$

To znamená, že $g'(\alpha_n) = \nabla f(x_n - \alpha_n \nabla f(x_n)) \cdot (-\nabla f(x_n)) = 0$. Gradient v novém bodě x_{n+1} je tedy kolmý na směr předchozího kroku $\nabla f(x_n)$.

Lemma 9.4: *Zig-zag efekt*

Při použití metody největšího spádu s přesným line search jsou směry sestupu v po sobě jdoucích krocích na sebe kolmé, tedy $\nabla f(x_n) \cdot \nabla f(x_{n+1}) = 0$. Důsledkem je, že pro špatně podmíněné úlohy (kde vrstevnice jsou protažené elipsy) může metoda konvergovat velmi pomalu a postupovat charakteristickým 'cik-cak' způsobem.



Obrázek 34: Zig-zag efekt metody největšího spádu u špatně podmíněné úlohy (úzké eliptické vrstevnice). Po sobě jdoucí kroky jsou na sebe kolmé.

Věta 9.5: *Konvergence metody největšího spádu*

Nechť $f : \mathbf{R}^N \to \mathbf{R}$ je $\mathcal{C}^2$ a existuje konstanta $L > 0$ taková, že $\|\nabla f(x) - \nabla f(y)\| \leq L\|x - y\|$ pro všechna $x, y \in \mathbf{R}^N$ (tj. gradient je Lipschitzovský). Při volbě konstantního kroku $\alpha_n = \alpha \in (0, 2/L)$ metoda největšího spádu konverguje ke stacionárnímu bodu, tj. $\nabla f(\bar{x}) = \nabla f(\lim_{n\to\infty} x_n) = \lim_{n\to\infty} \nabla f(x_n) = \vec{0}$. Konvergence je lineární.

Věta 9.6: *Konvergence pro konvexní funkce*

Nechť f je ryze konvexní a $\mathcal{C}^2$. Pak posloupnost $\{x_n\}$ generovaná metodou největšího spádu (s vhodnou volbou α_n, např. přesný line search nebo dostatečně malý konstantní krok) konverguje lineárně k jedinému globálnímu minimu $\bar{x}$.

9.4 Newtonova metoda v $\mathbf{R}^N$

Zatímco metoda největšího spádu využívá pouze první derivace (gradient), Newtonova metoda využívá i druhé derivace (Hessovu matici) k lepší aproximaci účelové funkce f.

Předpokládejme, že v okolí bodu x_n lze funkci f aproximovat kvadratickým polynomem $p(x)$ (Taylorův polynom 2. řádu):

$$p(x) = f(x_n) + \nabla f(x_n)^T (x - x_n) + \frac{1}{2}(x - x_n)^T H_f(x_n)(x - x_n).$$

Novou iteraci x_{n+1} hledáme jako minimum tohoto polynomu $p(x)$, tedy bod, kde $\nabla p(x) = \vec{0}$. Gradient $p(x)$ je:

$$\nabla p(x) = \nabla f(x_n) + H_f(x_n)(x - x_n).$$

Položíme-li $\nabla p(x) = \vec{0}$, dostáváme soustavu lineárních rovnic pro x:

$$H_f(x_n)(x - x_n) = -\nabla f(x_n).$$

Pokud je Hessova matice regulární, můžeme vyjádřit řešení x (které označíme jako x_{n+1}):

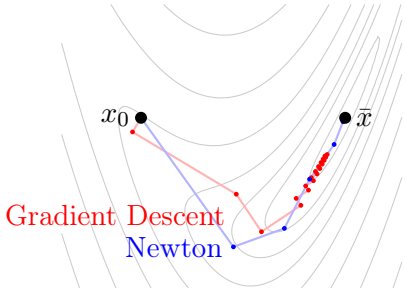
Definice 9.6: *Newtonova metoda pro minimalizaci*

Dáno $x_0 \approx \bar{x}$. Pro $n = 0, 1, 2, \ldots$ iteruj

$$x_{n+1} = x_n - (H_f(x_n))^{-1} \nabla f(x_n).$$

Vektor $d_n = -(\nabla^2 f(x_n))^{-1} \nabla f(x_n)$ se nazývá **Newtonův směr**.

- Tento postup je zobecněním Newtonovy metody pro hledání kořenů funkce jedné proměnné ($f'(x) = 0$), kde jsme měli $x_{n+1} = x_n - f'(x_n)/f''(x_n)$. Zde řešíme soustavu $\nabla f(x) = \vec{0}$.
- Pokud je původní funkce f kvadratická (tj. $f(x) = \frac{1}{2} x^T Q x + b^T x + c$ s Q pozitivně definitní), pak Newtonova metoda nalezne přesné minimum v jediné iteraci pro libovolnou počáteční volbu x_0. Pro obecné nelineární funkce konverguje Newtonova metoda typicky mnohem rychleji (kvadraticky) než metoda největšího spádu (pouze lineárně), i když může vyžadovat více kroků než začne skutečně konvergovat.
- Aby bylo nalezené x_{n+1} minimem (a ne maximem nebo sedlovým bodem) kvadratické aproximace, musí být Hessova matice $H_f(x_n)$ pozitivně definitní. To odpovídá podmínce $f''(x_n) > 0$ v jednorozměrném případě (funkce je konvexní).



Obrázek 35: Srovnání Newtonovy metody a metody největšího spádu na nekvadratické funkci (Rosenbrockova funkce). Newtonova metoda (modře) konverguje k minimu v několika málo krocích, zatímco gradientní metoda (červeně) osciluje a konverguje pomalu.

Vlastnosti Newtonova směru

Lemma 9.7: *O spádovém směru*

Nechť $\nabla f(x) \neq \vec{0}$. Nechť B je pozitivně definitní matice. Pak směr $d = -B \nabla f(x)$ je spádový směr funkce f v bodě x.

Důkaz. Ověříme podmínku spádového směru $\nabla f(x)^T d < 0$:

$$\nabla f(x)^T d = \nabla f(x)^T(-B\nabla f(x)) = -\nabla f(x)^T B \nabla f(x).$$

Protože B je pozitivně definitní a $\nabla f(x) \neq \vec{0}$, platí $\nabla f(x)^T B \nabla f(x) > 0$. Tedy celkově je skalární součin záporný, což znamená, že d je spádový směr. □

Důsledek 9.1 (Newtonův směr je spádový)**.** Pokud je Hessova matice $H_f(x_n)$ pozitivně definitní, pak je i její inverze $H_f(x_n))^{-1}$ pozitivně definitní. Z lemmatu pak plyne, že Newtonův směr $d_n = -(H_f(x_n))^{-1}\nabla f(x_n)$ je spádový směr.

9.4.1 Obecný algoritmus a modifikace

Obecný iterační krok námi uvažovaných optimalizačních metod lze zapsat ve tvaru:

$$x_{n+1} = x_n - \alpha_n M_n \nabla f(x_n),$$

kde M_n je pozitivně definitní matice a α_n je délka kroku (určená např. pomocí line search).

Různé volby matice M_n vedou na různé metody, například:

- $M_n = I$ (jednotková matice): **Metoda největšího spádu**.
- $M_n = (H_f(x_n))^{-1}$: **Newtonova metoda** (pro $\alpha_n = 1$), resp. *Newtonova metoda s line search* pro α_n určené pomocí line search.
- $M_n = (\mathrm{diag}(H_f(x_n)))^{-1}$: Aproximace Newtonovy metody (pouze diagonála Hessovy matice), výhoda je snadná invertovatelnost.
- $M_n = (\beta_n I + \nabla^2 f(x_n))^{-1}$: **Levenberg-Marquardtova metoda**.

Levenberg-Marquardtova metoda Tato metoda je kompromisem mezi metodou největšího spádu a Newtonovou metodou řešícím problém potenciální indefinitnosti $H_f(x_n)$. Matice M_n je inverzí k 'regularizované' Hessově matici.

- Pokud $\beta_n \to 0$, metoda se blíží Newtonově metodě.
- Pokud $\beta_n \to \infty$, převládá člen $\beta_n I$, a krok se blíží směru $-\frac{1}{\beta_n}\nabla f(x_n)$, což je (krátký) krok metody největšího spádu.

Volbou β_n tedy můžeme 'ladit' výsledný algoritmus, aby měl lepší z obou světů: robustnost metody největšího spádu a rychlost konvergence Newtonovy metody. Tato modifikace je užitečná zejména v případech, kdy Hessova matice $H_f(x_n)$ není pozitivně definitní (nebo je blízko singulární). Volbou dostatečně velkého $\beta_n > 0$ zajistíme, že matice $(\beta_n I + \nabla^2 f(x_n))$ bude pozitivně definitní, a tedy vygenerovaný směr bude spádový. Existují jednoduché, snadno implementovatelné algoritmy, jak volit β_n.

10 Ortogonální polynomy

Ortogonální (OG) polynomy jsou pomocné funkce mající šikovné vlastnosti. Jde o posloupnosti polynomů vzájemně ortogonálních vzhledem k danému skalárnímu součinu na (a, b).

Definice 10.1: *Skalární součin s vahou*

Dána váhová funkce $w : (a, b) \to \mathbf{R}, w \in L^\infty(a, b)$, $w > 0$ skoro všude na (a, b). Pro $f, g : (a, b) \to \mathbf{R}, f, g \in L^2(a, b)$ definujeme skalární součin

$$(f, g) = \int_a^b w(x) f(x) g(x)\,\mathrm{d}x,$$

Pozn.:

$$(f, f) = \int_a^b w(x) f^2(x)\,\mathrm{d}x \geq 0, \quad (f, f) = 0 \iff f \equiv 0.$$

Definice 10.2: *Vektorový prostor polynomů*

Vektorový prostor polynomů stupně nejvýše n na intervalu (a, b) označíme $\mathcal{P}_n(a, b)$.

Definice 10.3: *Ortogonální polynomy*

Posloupnost ortogonálních polynomů $\{\varphi_n\}_{n=0}^\infty$: pro všechna n je φ_n polynom stupně přesně n a

$$(\varphi_i, \varphi_j) = 0 \quad \forall i \neq j.$$

Konstrukce Gram-Schmidtem: Za bázi prostoru všech polynomů vezmeme množinu $\{c_n x^n\}_{n=0}^\infty$, kde $c_n \in \mathbf{R} \setminus \{0\}$ lze brát libovolně. Vezmeme $\varphi_0 := c_0$.

Dále nechť již mám zkonstruováno $\varphi_0, \ldots, \varphi_n$. Hledám φ_{n+1} ve tvaru

$$\varphi_{n+1}(x) = c_{n+1} x^{n+1} + a_0 \varphi_0(x) + \ldots + a_n \varphi_n(x).$$

Skalárním součinem s $\varphi_j, j = 0, \ldots, n$ dostaneme (pozor, $(\varphi_i, \varphi_j) = 0$ pro $i \neq j$):

$$0 = c_{n+1}(x^{n+1}, \varphi_j) + a_j(\varphi_j, \varphi_j),$$

tedy

$$a_j = -\frac{c_{n+1}(x^{n+1}, \varphi_j)}{(\varphi_j, \varphi_j)}, \quad j = 0, \ldots, n.$$

Algoritmus 13: Konstrukce ortogonální báze (Gram-Schmidt)

Data: Váhová funkce w na (a, b) a báze $\{c_n x^n\}_{n=0}^{\infty}$, $c_n \in \mathbf{R} \setminus \{0\}$.
Result: Ortogonální polynomy $\{\varphi_n\}_{n=0}^{\infty}$ vůči $(\cdot, \cdot)$.

$\varphi_0 := c_0$;
for $n = 0, 1, 2, \ldots$ **do**
  $\varphi_{n+1}(x) := c_{n+1} x^{n+1}$;
  for $j = 0$ **to** n **do**
    $a_j := -\dfrac{c_{n+1} \int_a^b w(x) x^{n+1} \varphi_j(x)\,\mathrm{d}x}{\int_a^b w(x) \varphi_j^2(x)\,\mathrm{d}x}$;
    $\varphi_{n+1}(x) := \varphi_{n+1}(x) + a_j \varphi_j(x)$;
  end
  // $\varphi_{n+1} \perp \varphi_j$ pro $j = 0, \ldots, n$
end

Poznámka 10.1 (Vlastnosti ortogonálních polynomů)**.**

- Ortogonalita: $(\varphi_i, \varphi_j) = 0$ pro $i \neq j$.
- $\{\varphi_j\}_{j=0}^{n}$ je OG báze $\mathcal{P}_n$.
- $\varphi_n \perp \mathcal{P}_{n-1}$, tj. $(\varphi_n, p) = 0,\ \forall p \in \mathcal{P}_{n-1}$.

Normalizace: OG polynomy lze normalizovat více způsoby (historické důvody), nejčastěji tak, aby φ_n byly

- **Monické:** koeficient u x^n ve $\varphi_n(x)$ je 1. Dosáhnu volbou $c_n = 1$.
- **Normalizované v koncovém bodě:** $\varphi_n(b) = 1$.
- **Ortonormální:** $(\varphi_n, \varphi_n) = 1$. Dosáhnu následnou normalizací $\varphi_n := \frac{1}{\sqrt{(\varphi_n, \varphi_n)}} \varphi_n$.

Pro OG polynomy platí jednoduché rekurence. Uvedeme si příklad pro monické OG polynomy. Pro ortonormální polynomy platí analogická verze.

VĚTA 10.1: *3-členná rekurence*

Existují $A_n, B_n \in \mathbf{R}$ taková, že pro monické ortogonální polynomy platí:

$$\varphi_{n+1}(x) = (x + A_n)\varphi_n(x) + B_n \varphi_{n-1}(x).$$

Důkaz. $x\varphi_n(x) \in \mathcal{P}_{n+1}$, tudíž

$$x\varphi_n(x) = \underbrace{\alpha_{n+1}}_{=1} \varphi_{n+1}(x) + \alpha_n \varphi_n + \cdots + \alpha_0 \varphi_0, \tag{3}$$

kde $\alpha_{n+1} = 1$, abychom dostali monický polynom. Skalární součin s φ_k, $k < n - 1$ dává

- Levá strana:

  $$(x\varphi_n, \varphi_k) = \int_a^b w(x) x \varphi_n(x) \varphi_k(x)\,\mathrm{d}x = \int_a^b w(x) \varphi_n(x) \big(x\varphi_k(x)\big)\,\mathrm{d}x = (\varphi_n, x\varphi_k) = 0,$$

  neboť $x\varphi_k \in \mathcal{P}_{k+1} \subseteq \mathcal{P}_{n-1}$ (protože $k < n - 1$) a tudíž $(\varphi_n, x\varphi_k) = 0$ neboť $\varphi_n \perp \mathcal{P}_{n-1}$.

- Pravá strana: zbyde jenom člen $\alpha_k(\varphi_k, \varphi_k)$, všechny ostatní členy jsou nulové ze vzájemné ortogonality funkcí φ_j.

Celkově tedy dostaneme $0 = \alpha_k(\varphi_k, \varphi_k) \implies \alpha_k = 0, \forall k < n-1$. Tudíž z (3) zůstane

$$x\varphi_n(x) = \varphi_{n+1} + \alpha_n\varphi_n + \alpha_{n-1}\varphi_{n-1}.$$

Přeuspořádám:

$$\varphi_{n+1}(x) = (x - \alpha_n)\varphi_n(x) - \alpha_{n-1}\varphi_{n-1}(x).$$

□

Věta 10.2: *Kořeny ortogonálních polynomů*

Polynom φ_n má n různých kořenů v intervalu (a, b).

Důkaz. Nechť φ_n má k kořenů *liché násobnosti* v (a, b) a nechť je $k < n$ (což přivedeme ke sporu). Označme tyto liché kořeny $\alpha_1, \dots \alpha_k$. Pak polynom

$$(x - \alpha_1)(x - \alpha_2) \dots (x - \alpha_k)\, \varphi_n(x) \tag{4}$$

má jen kořeny sudé násobnosti, tudíž nemění znaménko v (a, b). Tedy

$$0 \neq \int_a^b w(x) \underbrace{(x - \alpha_1) \dots (x - \alpha_k)}_{=p_k(x)} \varphi_n(x)\, \mathrm{d}x = (p_k, \varphi_n) = 0, \tag{5}$$

neboť stupeň polynomu p_k je $k < n$. Tím jsme dostali spor $0 \neq 0$. □

10.1 Legendreovy polynomy

Definice 10.4: *Legendreovy polynomy*

Ortogonální polynomy $\{\mathcal{L}_n\}_{n=0}^{\infty}$ na intervalu $(a, b) = (-1, 1)$ s váhovou funkcí $w(x) = 1$. Normalizace: $\mathcal{L}_n(1) = 1$. Prvních pár polynomů:

- $\mathcal{L}_0(x) = 1$
- $\mathcal{L}_1(x) = x$
- $\mathcal{L}_2(x) = \frac{1}{2}(3x^2 - 1)$

Rekurence:

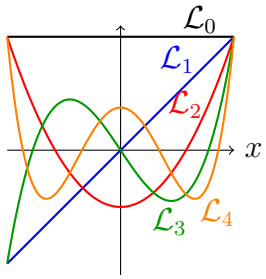
$$(n+1)\mathcal{L}_{n+1}(x) = (2n+1)x\mathcal{L}_n(x) - n\mathcal{L}_{n-1}(x).$$

10.2 Čebyševovy polynomy

Definice 10.5: *Čebyševovy polynomy*

Ortogonální polynomy $\{T_n\}_{n=0}^{\infty}$ na intervalu $(a, b) = (-1, 1)$ s váhovou funkcí $w(x) = \frac{1}{\sqrt{1-x^2}}$. Definice:

$$T_n(x) = \cos(n \arccos x), \quad x \in [-1, 1].$$



Obrázek 36: Legendreovy polynomy (ortogonální na $[-1,1]$ s vahou $w(x)=1$).

Poznámka 10.2 (Příklady Čebyševových polynomů)**.** Substituce $x=\cos\theta, \theta\in[0,\pi]$. Pak $T_n(\cos\theta)=\cos(n\theta)$. Příklady:

- $T_0(x)=1$
- $T_1(x)=x$
- $T_2(x)=2x^2-1$ (ze vzorce $\cos(2\theta)=2\cos^2\theta-1$)

Věta 10.3: *Rekurence pro Čebyševovy polynomy*

$$T_{n+1}(x)=2xT_n(x)-T_{n-1}(x).$$

Důkaz. Viz cvičení. □

Věta 10.4: *Ortogonalita Čebyševových polynomů*

$$(T_n,T_m)=0,\quad \forall n\neq m.$$

Důkaz.

$$(T_n,T_m)=\int_{-1}^{1}\frac{\cos(n\arccos x)\cos(m\arccos x)}{\sqrt{1-x^2}}\,\mathrm{d}x.$$

Substituce $x=\cos\theta, \mathrm{d}x=-\sin\theta\,\mathrm{d}\theta$. Meze $[-1,1]\to[\pi,0]$:

$$(T_n,T_m)=\int_0^{\pi}\frac{\cos(n\theta)\cos(m\theta)}{\sin\theta}\sin\theta\,\mathrm{d}\theta=\int_0^{\pi}\cos(n\theta)\cos(m\theta)\,\mathrm{d}\theta=0,\quad \text{pro } n\neq m.$$

□

Věta 10.5: *Monické Čebyševovy polynomy*

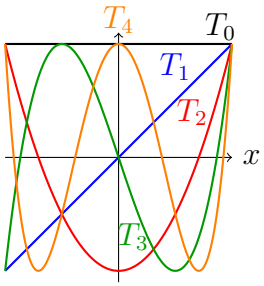
Polynomy $\tilde{T}_n(x)=\frac{1}{2^{n-1}}T_n(x)$ pro $n\geq 1$ (a $\tilde{T}_0=T_0$) jsou monické (koeficient u x^n je 1).

Důkaz. Plyne indukcí z Věty 10.3. □

Poznámka 10.3 (Kořeny Čebyševových polynomů)**.** Čebyševův polynom $T_n(x)=\cos(n\arccos x)$ má kořeny

$$x_k=\cos\left(\frac{2k-1}{2n}\pi\right),\quad k=1,\dots,n.$$

Extrémy T_n na $[-1,1]$ jsou v bodech $x_k=\cos(\frac{k\pi}{n})$, hodnota je $(-1)^k$.



Obrázek 37: Čebyševovy polynomy (ortogonální na $[-1,1]$ s vahou $w(x) = \frac{1}{\sqrt{1-x^2}}$)

Min-Max vlastnost

Věta 10.6: *Min-Max vlastnost*

Nechť p_n je libovolný monický polynom stupně n. Pak

$$\max_{x\in[-1,1]} |p_n(x)| \geq \max_{x\in[-1,1]} |\tilde{T}_n(x)| = \frac{1}{2^{n-1}}.$$

Tedy $\tilde{T}_n$ je monický polynom s nejmenší maximální normou na $[-1,1]$.

Důkaz. 1. $|\tilde{T}_n|$ nabývá maxima v bodech $y_k = \cos\frac{k\pi}{n}$, $k = 0,\dots,n$. Platí:

$$\tilde{T}_n(y_k) = \frac{1}{2^{n-1}} T_n(y_k) = \frac{(-1)^k}{2^{n-1}}.$$

Hodnoty střídají znaménka.

2. Sporem: Nechť existuje monický p_n takový, že $\max_{[-1,1]} |p_n| < \frac{1}{2^{n-1}}$. Definujme $r = \tilde{T}_n - p_n$. Jde o rozdíl dvou monických polynomů stupně n, tedy $\deg r \leq n-1$. Podívejme se na body y_k:

$$r(y_k) = \tilde{T}_n(y_k) - p_n(y_k) = \frac{(-1)^k}{2^{n-1}} - p_n(y_k).$$

Protože $|p_n(y_k)| < \frac{1}{2^{n-1}}$, znaménko $r(y_k)$ je určeno znaménkem $\tilde{T}_n(y_k)$. Tedy

$$\operatorname{sgn} r(y_k) = \operatorname{sgn}((-1)^k) = (-1)^k,$$

tedy $r(y_k) > 0$ pro k sudé, $r(y_k) < 0$ pro k liché. Čili r mění znaménko mezi y_k a y_{k+1} pro $k = 0,\dots,n-1$. Tedy r má alespoň n různých kořenů. Ale $\deg r \leq n-1$. Jediný polynom stupně $\leq n-1$ s n kořeny je nulový polynom. Tedy $r \equiv 0 \Rightarrow p_n = \tilde{T}_n$. Spor s předpokladem ostré nerovnosti $\max|p_n| < \max|\tilde{T}_n|$.

□

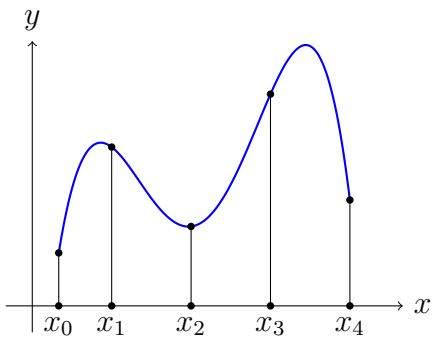
11 Interpolace funkcí

Úloha: Aproximovat složité funkce jednoduššími, s danou přesností. Nechť $f : \mathbf{R} \to \mathbf{R}$ je daná funkce, resp. její hodnoty v zadaných bodech $x_0, \ldots, x_n$. Z těchto funkčních hodnot chci zkonstruovat polynom (resp. po částech polynom) dobře aproximující f.

- $f \in \mathcal{C}[a, b]$ je de facto určena hodnotami v ∞-mnoho bodech, nahradím něčím konečným – polynomy.
- Taylorův polynom – obecně špatná aproximace dál od bodu, kde Tayloruji, vyžaduje vyšší derivace.
- **My:** f znám v konečně mnoha bodech (uzly) $x_0, \ldots, x_n \longrightarrow$ polynom.

Příklad 11.1 (Příklady využití)**.**

- Výpočet funkcí moc složitý: $\Gamma(x) = \int_0^\infty t^{x-1} e^{-t} dt$, $\Gamma(n) = (n-1)!$.
- f dáno tabulkou, měřením, experimentem.
- Mapy – interpolace vrstevnic; doplnění chybějících dat; rekonstrukce chybějících částí obrázku; digital $\to$ analog; atd.



Obrázek 38: Proložení bodů grafem polynomu

11.1 Lagrangeova interpolace

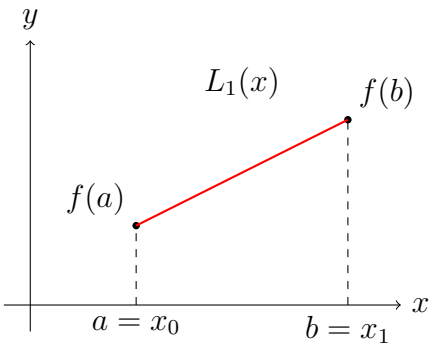
DEFINICE 11.1: *Lagrangeova interpolace*

Dáno $f : [a, b] \to \mathbf{R}$, uzly $a \leq x_0 < x_1 < x_2 < \cdots < x_n \leq b$. Polynom $L_n \in \mathcal{P}_n$ takový, že

$$L_n(x_i) = f(x_i), \quad i = 0, \ldots, n,$$

nazveme **Lagrangeovou interpolací** f v uzlech $\{x_i\}_{i=0}^n$.

L_n je určeno $n+1$ koeficienty, máme $n+1$ podmínek. Každá podmínka $L_n(x_i) = f(x_i)$ dá jednu lineární rovnici. Celkově máme tedy čtvercovou soustavu lineárních rovnic pro koeficienty L_n. Takhle to ale není praktické konstruovat.



Obrázek 39: Lineární interpolace

Příklad 11.2 (Lineární interpolace)**.** Lagrangeova interpolace mezi dvěma body dává polynom stupně 1, tedy afinní funkci.

$$L_1(x) = \frac{x-b}{a-b} f(a) + \frac{x-a}{b-a} f(b).$$

Jde o lineární kombinaci $f(a), f(b)$. Koeficienty této lineární kombinace jsou funkce (např. $\frac{x-b}{a-b}$), které mají tu zásadní vlastnost, že v zadaných uzlech a, b nabývají hodnot 0 nebo 1. To je i recept na obecnou konstrukci.

Obecně: Chci $l_i \in \mathcal{P}_n$ (bázové funkce) splňující

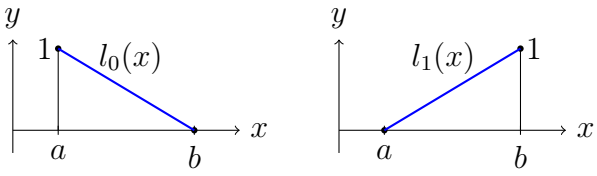
$$l_i(x_j) = \delta_{ij} = \begin{cases} 1, & i = j, \\ 0, & i \neq j. \end{cases}$$

DEFINICE 11.2: *Lagrangeovy bázové polynomy*

Nechť $x_0 < x_1 < \cdots < x_{n-1} < x_n \in \mathbf{R}$ jsou uzly interpolace. Polynomy

$$l_i(x) = \prod_{\substack{k=0 \\ k \neq i}}^{n} \frac{x - x_k}{x_i - x_k}, \quad i = 0, \ldots, n$$

nazveme **Lagrangeovy bázové polynomy**.



Obrázek 40: Lagrangeovy bázové polynomy pro $n = 1$

LEMMA 11.1: *Vlastnost bázových polynomů*

Platí $l_i(x_j) = \delta_{ij}$.

Důkaz.

$$l_i(x_i) = \prod_{\substack{k=0 \\ k\neq i}}^{n} \frac{x_i - x_k}{x_i - x_k} = 1.$$

Pro $j \neq i$:

$$l_i(x_j) = \prod_{\substack{k=0 \\ k\neq i}}^{n} \frac{x_j - x_k}{x_i - x_k} = 0,$$

protože v součinu se vyskytuje činitel pro $k = j$ (což je povoleno, neboť $k \neq i$), tedy čitatel obsahuje $(x_j - x_j) = 0$. □

Věta 11.2: *Existence a jednoznačnost interpolačního polynomu*

Nechť je dána funkce f a uzly $\{x_i\}_{i=0}^n$ jako výše. Pak existuje právě jeden $L_n \in \mathcal{P}_n$ interpolující f v uzlech $\{x_i\}_{i=0}^n$, a platí

$$L_n(x) = \sum_{i=0}^{n} l_i(x) f(x_i).$$

Důkaz. **Existence:** Ověříme, že definovaný L_n splňuje interpolační podmínky.

$$L_n(x_j) = \sum_{i=0}^{n} l_i(x_j) f(x_i) = \sum_{i=0}^{n} \delta_{ij} f(x_i) = f(x_j).$$

Jelikož $l_i \in \mathcal{P}_n$, je i jejich lineární kombinace polynom stupně nejvýše n.

Jednoznačnost: Sporem. Nechť existují dva různé polynomy $L_n \neq \tilde{L}_n$, $L_n, \tilde{L}_n \in \mathcal{P}_n$, oba interpolující f v uzlech $\{x_i\}_{i=0}^n$. Pak

$$L_n(x_i) = \tilde{L}_n(x_i) = f(x_i), \quad i = 0, \ldots, n.$$

Polynom $Q(x) = L_n(x) - \tilde{L}_n(x)$ je stupně nejvýše n (rozdíl dvou polynomů z $\mathcal{P}_n$). Zároveň má $n+1$ různých kořenů $x_0, \ldots, x_n$, protože $Q(x_i) = L_n(x_i) - \tilde{L}_n(x_i) = 0$. Polynom stupně $\leq n$ mající $n+1$ kořenů musí být identicky nulový (viz algebra, nenulový polynom st. n má max n kořenů). Tedy $Q(x) \equiv 0 \implies L_n(x) \equiv \tilde{L}_n(x)$, což je spor s předpokladem, že jsou různé. □

Věta 11.3: *Odhad chyby (Cauchy)*

Nechť $f \in \mathcal{C}^{n+1}[a,b]$, $a \leq x_0 < x_1 < \cdots < x_n \leq b$ jsou uzly a L_n je příslušný Lagrangeův interpolační polynom. Pak pro každé $x \in [a,b]$ existuje $\xi_x \in [a,b]$ takové, že

$$f(x) - L_n(x) = \frac{f^{(n+1)}(\xi_x)}{(n+1)!} \prod_{i=0}^{n} (x - x_i).$$

Důkaz. 1. Pokud $x = x_i$ pro nějaké i, pak levá strana je $f(x_i) - L_n(x_i) = 0$ a pravá strana obsahuje činitel $(x_i - x_i) = 0$, takže rovnost platí $(0 = 0)$.

2. Nechť $x \in [a,b]$ a $x \neq x_i$ pro všechna i. Zafixujme toto x. Definujme pomocnou funkci $g(t)$ proměnné t:

$$g(t) = f(t) - L_n(t) - (f(x) - L_n(x)) \prod_{i=0}^{n} \frac{t - x_i}{x - x_i}.$$

Zkoumejme kořeny $g(t)$:

- Pro $t = x_k$ $(k = 0, \ldots, n)$:

$$g(x_k) = \underbrace{f(x_k) - L_n(x_k)}_{=0} - (f(x) - L_n(x)) \prod_{i=0}^{n} \underbrace{\frac{x_k - x_i}{x - x_i}}_{=0 \text{ pro } i=k} = 0.$$

- Pro $t = x$:

$$g(x) = f(x) - L_n(t) - (f(x) - L_n(x)) \prod_{i=0}^{n} \underbrace{\frac{x - x_i}{x - x_i}}_{=1,\ \forall i} = 0.$$

Funkce $g(t)$ má tedy $n+2$ různých kořenů v $[a,b]$ (body $x_0, \ldots, x_n$ a bod x). Navíc $g \in \mathcal{C}^{n+1}[a,b]$, protože f je z této třídy a ostatní členy jsou polynomy.

Použijeme Rolleovu větu opakovaně:

- g má $n+2$ různých kořenů $\implies$ g' má alespoň $n+1$ různých kořenů (mezi každými dvěma sousedními kořeny g).
- g' má $n+1$ různých kořenů $\implies$ g'' má alespoň n různých kořenů.
- $\ldots$
- $g^{(n+1)}$ má alespoň 1 kořen v $[a,b]$. Označme jej ξ_x.

Tedy $\exists \xi_x \in [a,b] : g^{(n+1)}(\xi_x) = 0$.

Nyní spočtěme $g^{(n+1)}(t)$:

$$g^{(n+1)}(t) = f^{(n+1)}(t) - \underbrace{L_n^{(n+1)}(t)}_{=0} - \frac{f(x) - L_n(x)}{\prod_{i=0}^{n}(x - x_i)} \frac{\mathrm{d}^{n+1}}{\mathrm{d}t^{n+1}} \left(\underbrace{\prod_{i=0}^{n}(t - x_i)}_{(*)} \right).$$

Protože $L_n \in \mathcal{P}_n$, jeho $(n+1)$-ní derivace je 0. Všimněme si dále, že výraz $(*) = t^{n+1} + p_n$, kde $p_n \in \mathcal{P}_n$. Tudíž

$$\frac{\mathrm{d}^{n+1}}{\mathrm{d}t^{n+1}} \left(\prod_{i=0}^{n}(t - x_i) \right) = \frac{\mathrm{d}^{n+1}}{\mathrm{d}t^{n+1}} t^{n+1} = (n+1)!$$

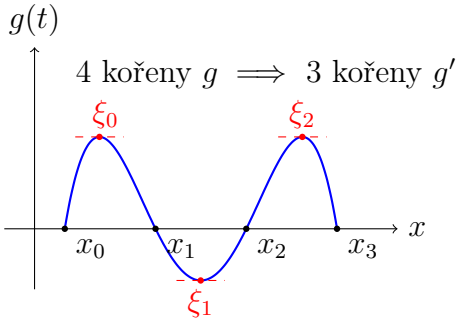
Dosadíme do $g^{(n+1)}(\xi_x) = 0$:

$$0 = f^{(n+1)}(\xi_x) - (f(x) - L_n(x)) \frac{(n+1)!}{\prod_{i=0}^{n}(x - x_i)}.$$

Přeuspořádáním dostaneme tvrzení věty:

$$f(x) - L_n(x) = \frac{f^{(n+1)}(\xi_x)}{(n+1)!} \prod_{i=0}^{n}(x - x_i).$$

□



Obrázek 41: Ilustrace Rolleovy věty pro chybu interpolace

Poznámka 11.1 (Analogie s Taylorovým polynomem)**.** Lagrangeův tvar zbytku Taylorova polynomu je podobný chybě interpolace:

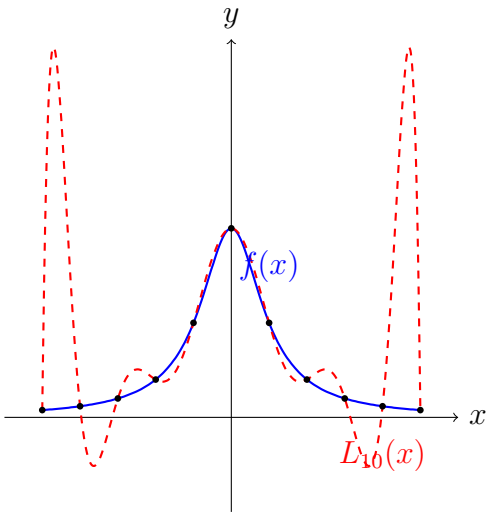
$$R_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!}(x-a)^{n+1} \quad \text{vs} \quad E_n(x) = \frac{f^{(n+1)}(\xi_x)}{(n+1)!}\prod_{i=0}^{n}(x-x_i).$$

Taylorův polynom je v jistém smyslu limitním případem interpolace, kde všechny uzly konvergují k bodu a.

11.2 Konvergence interpolace

Otázka: Platí $L_n \to f$ bodově či stejnoměrně pro $n \to \infty$?

Příklad 11.3 (Rungeho jev)**.** Uvažujme funkci $f(x) = \frac{1}{1+25x^2}$ na intervalu $[-1,1]$. Při rovnoměrném rozložení uzlů $\{x_i\}_{i=0}^n$ v intervalu $[a,b]$ posloupnost interpolačních polynomů L_n **diverguje** (a osciluje u krajů intervalu), i když $f \in \mathcal{C}^\infty[-1,1]$.



Obrázek 42: Rungeho jev: funkce $f(x)$ a interpolační polynom $L_{10}(x)$ na rovnoměrném dělení

VĚTA 11.4: *Analýza chyby u Rungeho jevu*

Označme $M_n = \max_{x\in[a,b]} |f^{(n+1)}(x)|$. Věta o odhadu chyby dává

$$|L_n(x) - f(x)| \leq \frac{1}{(n+1)!} M_n \max_{x\in[a,b]} \left| \prod_{i=0}^{n} (x - x_i) \right|.$$

Pro Rungeho funkci $f(x) = \frac{1}{1+25x^2}$ má f póly v komplexní rovině v bodech $\pm\frac{i}{5}$. Derivace M_n proto rostou pro tuto funkci tak rychle, že ani faktoriál $(n+1)!$ ve jmenovateli nestačí k potlačení růstu, a chyba diverguje. Detailní analýza je velice technická a delikátní.

Horní odhad z Věty 11.3 máme jedinou možnost ovlivnit, a sice volbou uzlů $\{x_i\}_{i=0}^n$. Ty totiž určují člen $\omega_n(x) = \prod_{i=0}^n (x - x_i)$. Právě tímto členem můžeme zmírnit Rungeho jev. Tento člen osciluje a pro ekvidistantní uzly nabývá největších hodnot právě u krajů intervalu.

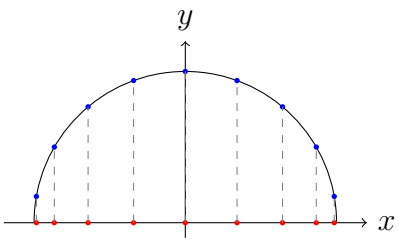
Lepší volba uzlů: Chceme minimalizovat člen $\prod_{i=0}^n (x - x_i)$, což je monický polynom. Z Věty 10.6 víme, že nejmenší monický polynom (v maximové normě) je Čebyševův polynom. Pokud za uzly x_i vezmeme kořeny Čebyševova polynomu $\tilde{T}_n$, pak $\prod_{i=0}^n (x - x_i)$ bude příslušným Čebyševovým polynomem. To je v jistém smyslu optimální volba uzlů.

DEFINICE 11.3: *Čebyševova interpolace*

Optimální volbou uzlů na intervalu $[-1, 1]$ jsou kořeny Čebyševova polynomu (prvního druhu) stupně $n+1$:

$$x_i = \cos\left(\frac{2i+1}{2n+2}\pi\right), \quad i = 0, \ldots, n.$$

(Pro obecný interval $[a, b]$ použijeme lineární transformaci). Příslušná Lagrangeova interpolace se nazývá **Čebyševovou interpolací**.
Pro tyto uzly je hodnota $\max_{[-1,1]} \prod_{i=0}^n |x - x_i|$ nejmenší možná mezi všemi monickými polynomy stupně $n+1$.



Obrázek 43: Kořeny Čebyševova polynomu pro $n = 9$

VĚTA 11.5: *Konvergence pro Čebyševovy uzly*

Pokud $f \in \mathcal{C}[a,b]$ je *lipschitzovská*, pak pro posloupnost Lagrangeových interpolačních polynomů L_n s Čebyševovými uzly platí

$$L_n \rightrightarrows f \quad \text{na } [a,b] \quad \text{(stejnoměrná konvergence)}.$$

O moc víc to nejde zlepšit, protože platí:

VĚTA 11.6: *Faberova věta (o divergenci)*

Pro libovolnou posloupnost uzlů (trojúhelníkové schéma) existuje funkce $f \in \mathcal{C}[a,b]$, pro kterou posloupnost interpolačních polynomů L_n diverguje skoro všude.

Dodatky

Open problémy

- **Lebesgueovy konstanty:** Otázky konvergence jsou spojeny s odhady tzv. Lebesgueových konstant
$$\max_{x\in[a,b]} \sum_{i=0}^{n} |l_i(x)|,$$
kdy je potřeba odhadnout jejich růst pro $n \to \infty$ pro dané dělení. Významné výsledky zde dokázal např. Paul Erdős (jinak známý z teorie čísel).

- Paul Erdős dále dokázal, že pokud jsou uzly kořeny ortogonálních polynomů s váhou w, pak
$$\int_a^b (f - L_n)^2 w \, dx \to 0,$$
tj. konvergence ve vážené L^2-normě.

Extrapolace

- Interpoluji na $[a,b]$, chci odhad $f(x) \approx L_n(x)$ pro $x \notin [x_0, \ldots, x_n]$.
- Chyba $f(x) - L_n(x)$ roste velmi rychle do nekonečna pro $x \notin [a,b]$.
- Lze dokázat, že Čebyševův polynom je nejrychleji rostoucí monický polynom mimo $[-1,1]$. Tedy Čebyševova interpolace (nejlepší možná na $[-1,1]$) je v jistém smyslu nejhorší možná mimo $[-1,1]$.

12 Spline funkce

Interpolace pomocí jedné globální funkce (polynomu velmi vysokého stupně) $L_n(x)$ má nevýhody: vyžaduje hodně operací při vyhodnocení v x a může trpět oscilacemi (Rungeho jev). Jiný přístup: dělení intervalu, po částech polynom. Pro jednoduchost budeme uvažovat rovnoměrná dělení, ale snadno se vše zobecní na nerovnoměrné dělení.

12.1 Lineární interpolační spliny

Mějme funkci $f : [a, b] \to \mathbf{R}$ a ekvidistantní uzly $a = x_0 < x_1 < \cdots < x_n = b$, kde $x_{i+1} - x_i = h$ pro všechna i. Definujme $I_i = [x_i, x_{i+1}]$ a pro jednoduchost označme $f_i = f(x_i)$.

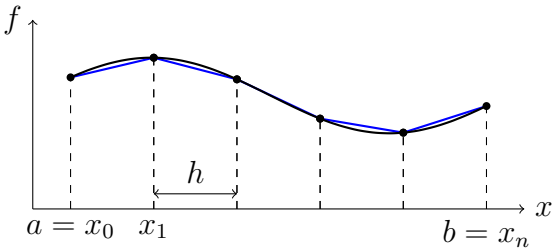
Definice 12.1: *Lineární interpolační spline*

Funkci S_L nazveme **lineární interpolační spline**, pokud

1. $S_L \in \mathcal{C}[a, b]$ (funkce je spojitá na celém intervalu),
2. $S_L|_{I_i}$ je lineární funkce pro všechna $i = 0, \ldots, n-1$,
3. $S_L(x_i) = f_i$ pro všechna $i = 0, \ldots, n$ (interpolační podmínky).

Lineární spline S_L lze snadno zkonstruovat na každém intervalu zvlášť. Na intervalu I_i je totiž S_L totožný s Lagrangeovou lineární interpolací L_1 na tomto intervalu:

$$S_L(x)|_{I_i} = \frac{x_{i+1} - x}{h} f_i + \frac{x - x_i}{h} f_{i+1}.$$



Obrázek 44: Lineární interpolační spline S_L na intervalech I_i.

Věta 12.1: *Odhad chyby lineárního splinu*

Nechť $f \in \mathcal{C}^2[a, b]$. Pak

$$\max_{x \in [a,b]} |f(x) - S_L(x)| \leq \frac{1}{8} h^2 \max_{x \in [a,b]} |f''(x)|.$$

Důkaz. Uvažujme chybu Lagrangeovy interpolace L_1 na intervalu I_i. Pro $x \in I_i$ platí podle věty o chybě interpolace (Věta 11.3):

$$f(x) - S_L(x) = \frac{1}{2} f''(\xi_x)(x - x_i)(x - x_{i+1}), \quad \text{kde } \xi_x \in I_i.$$

Odhadneme členy na pravé straně:

$$|f''(\xi)| \leq \max_{t\in[a,b]} |f''(t)|.$$

Maximum výrazu $|(x-x_i)(x-x_{i+1})|$ na intervalu délky h nastává uprostřed (parabola s kořeny x_i, x_{i+1}) a je rovno $\frac{h^2}{4}$. Tedy

$$|f(x) - S_L(x)| \leq \frac{1}{2} \cdot \max_{x\in[a,b]} |f''(x)| \cdot \frac{h^2}{4} = \frac{1}{8} h^2 \max_{x\in[a,b]} |f''(x)|.$$

□

Vidíme tedy, že pro $f \in \mathcal{C}^2[a,b]$ budou při zjemňování dělení interpolace lineárními spliny bodově konvergovat k f jako h^2. Snadno se dá dokázat, že interpolace budou konvergovat i pro $f \in \mathcal{C}[a,b]$. Srovnejme to s Lagrangeovou interpolací na rovnoměrném dělení, kde není zaručena konvergence ani pro $f \in \mathcal{C}^\infty[a,b]$.

12.2 Kubické interpolační spliny

Funcke S_L je přeci jen poněkud 'hranatá'. Pro vyšší hladkost aproximace zavádíme kubické spline funkce.

Definice 12.2: *Kubický interpolační spline*

Funkci S nazveme **kubický interpolační spline**, pokud

1. $S \in \mathcal{C}^2[a,b]$ (funkce je dvakrát spojitě diferencovatelná),
2. $S|_{I_i}$ je kubická funkce, tj. $S|_{I_i} \in \mathcal{P}_3$ pro všechna $i = 0, \ldots, n-1$,
3. $S(x_i) = f_i$ pro všechna $i = 0, \ldots, n$ (interpolační podmínky).

Počet neznámých a podmínek

Vychází to? Na každém intervalu I_i je $S|_{I_i}$ kubický polynom, což znamená 4 neznámé koeficienty. Máme n intervalů, tedy celkem $4n$ neznámých.

Podmínky, které musí S splňovat:

1. **Interpolační podmínky** na krajích intervalů I_i:

   $$S|_{I_i}(x_i) = f_i, \quad S|_{I_i}(x_{i+1}) = f_{i+1}.$$

   To jsou 2 podmínky na každém z n intervalů $\implies$ $2n$ podmínek.

2. **Spojitost 1. derivace** ve vnitřních uzlech $x_1, \ldots, x_{n-1}$:

   $$S'(x_i-) = S'(x_i+), \quad i = 1, \ldots, n-1.$$

   To je $n-1$ podmínek.

3. **Spojitost 2. derivace** ve vnitřních uzlech $x_1, \ldots, x_{n-1}$:

   $$S''(x_i-) = S''(x_i+), \quad i = 1, \ldots, n-1.$$

   To je $n-1$ podmínek.

Celkový počet podmínek je:

$$2n + (n-1) + (n-1) = 4n - 2 \text{ podmínek.}$$

Do plného určení $4n$ neznámých nám tedy chybí 2 podmínky. Tyto volné parametry se obvykle určují pomocí okrajových podmínek v a, b.

12.2.1 Okrajové podmínky a konstrukce

Doplníme 2 okrajové podmínky. Možné volby jsou například:

1. $S'(a) = f'(a), S'(b) = f'(b)$,
2. $S''(a) = f''(a), S''(b) = f''(b)$ (**náš případ**),
3. $S''(a) = S''(b) = 0$ (*přirozený kubický spline*).

Při konstrukci nemůžeme postupovat lokálně interval po intervalu (jako u lineárního splinu), protože podmínky spojitosti derivací propojují sousední intervaly.

Definice 12.3: *Momenty splinu*

Momenty splinu M_i definujeme jako hodnoty druhé derivace splinu v uzlových bodech:

$$M_i := S''(x_i), \quad i = 0, \ldots, n.$$

Pro momenty odvodíme soustavu lineárních rovnic, z jejíhož řešení pak zkonstruujeme S.

Lemma 12.2: *Soustava lineárních rovnic pro momenty M_i*

Pro $i = 1, \ldots, n-1$ platí:

$$M_{i-1} + 4M_i + M_{i+1} = \frac{6}{h^2}(f_{i+1} - 2f_i + f_{i-1}).$$

Z okrajových podmínek navíc platí $M_0 = f''(a)$ a $M_n = f''(b)$.
Matice soustavy A je ostře diagonálně dominantní, tedy regulární. Existuje proto právě jedno řešení $M = (M_1, \ldots, M_{n-1})^T$, a tedy právě jeden kubický spline S.

Důkaz. **1.** Nejprve Taylorujeme S doprava a doleva od x_i (pozor, S''' už je nespojitá a čtvrtáá derivace je již nulová):

$$S(x_{i+1}) = S(x_i) + S'(x_i)h + \tfrac{1}{2}S''(x_i)h^2 + \tfrac{1}{6}S'''(x_i+)h^3,$$
$$S(x_{i-1}) = S(x_i) - S'(x_i)h + \tfrac{1}{2}S''(x_i)h^2 - \tfrac{1}{6}S'''(x_i-)h^3.$$

Obě rovnice sečteme s tím, že $S(x_i) = f_i$, $S''(x_i) = M_i$, apod., čímž dostaneme

$$f_{i+1} + f_{i-1} = 2f_i + M_i h^2 + \tfrac{1}{6}h^3 \underbrace{(S'''(x_i+) - S'''(x_i-))}_{(*)}. \tag{6}$$

Nyní zbývá vyjádřit člen $(*)$.

2. Taylorujeme S'' doprava a doleva od x_i (S'' je lineární funkce):

$$S''(x_{i+1}) = S''(x_i) + S'''(x_i+)h,$$
$$S''(x_{i-1}) = S''(x_i) - S'''(x_i-)h.$$

Sečtením a vydělením h dostaneme výraz pro $(*)$:

$$(*) = S'''(x_i+) - S'''(x_i-) = \frac{1}{h}(M_{i+1} - 2M_i + M_{i-1}).$$

Dosazením do (6) a přeuspořádáním dostaneme kýžený výsledek. □

Soustavu rovnic pro neznámé momenty $\{M_i\}_{i=1}^{n-1}$ lze zapsat maticově:

$$\begin{pmatrix} 4 & 1 & & & \\ 1 & 4 & 1 & & \\ & \ddots & \ddots & \ddots & \\ & & 1 & 4 & 1 \\ & & & 1 & 4 \end{pmatrix} \begin{pmatrix} M_1 \\ \vdots \\ M_{n-1} \end{pmatrix} = \begin{pmatrix} g_1 - f''(a) \\ g_2 \\ \vdots \\ g_{n-2} \\ g_{n-1} - f''(b) \end{pmatrix}$$

kde $g_i = \frac{6}{h^2}(f_{i+1} - 2f_i + f_{i-1})$. Matice soustavy A je ostře diagonálně dominantní (v každém řádku je součet absolutních hodnot prvků mimo diagonálu roven 2, což je menší než diagonální prvek 4). Matice je tedy regulární a soustava má pro libovolné okrajové podmínky M_0, M_n právě jedno řešení.

Konstrukce $S|_{I_i}$

Na intervalu $I_i = [x_i, x_{i+1}]$ hledáme polynom $S \in \mathcal{P}_3$, který splňuje předepsané funkční hodnoty f_i, f_{i+1} a momenty (druhé derivace) M_i, M_{i+1}:

$$S(x_i) = f_i, \quad S(x_{i+1}) = f_{i+1}, \quad S''(x_i) = M_i, \quad S''(x_{i+1}) = M_{i+1}.$$

Protože S je kubický polynom, jeho druhá derivace S'' je lineární funkce. Můžeme ji vyjádřit jako lineární interpolaci hodnot M_i a M_{i+1}:

$$S''(x) = M_i\frac{x_{i+1} - x}{h} + M_{i+1}\frac{x - x_i}{h}.$$

Dvojnásobnou integrací získáme vyjádření pro $S(x)$. První integrací dostaneme:

$$S'(x) = -M_i\frac{(x_{i+1} - x)^2}{2h} + M_{i+1}\frac{(x - x_i)^2}{2h} + A_i,$$

kde A_i je integrační konstanta. Druhou integrací získáme:

$$S(x) = M_i\frac{(x_{i+1} - x)^3}{6h} + M_{i+1}\frac{(x - x_i)^3}{6h} + A_i(x - x_i) + B_i.$$

Zde jsme integrační konstantu z druhé integrace zvolili ve tvaru $A_i(x - x_i) + B_i$ (posunutí argumentu u A_i je jen přečíslování konstanty, které zjednoduší dosazování).

Neznámé konstanty A_i a B_i určíme z interpolačních podmínek:

1. Dosazení bodu x_i:

$$S(x_i) = M_i\frac{h^3}{6h} + 0 + 0 + B_i = M_i\frac{h^2}{6} + B_i = f_i \implies B_i = f_i - \frac{h^2}{6}M_i.$$

2. Dosazení bodu x_{i+1}:

$$S(x_{i+1}) = 0 + M_{i+1}\frac{h^3}{6h} + A_i h + B_i = f_{i+1}.$$

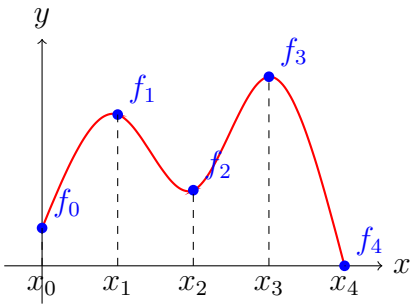
Dosadíme za B_i:

$$M_{i+1}\frac{h^2}{6} + A_i h + \left(f_i - \frac{h^2}{6}M_i\right) = f_{i+1}.$$

Odtud vyjádříme A_i:

$$A_i h = f_{i+1} - f_i - \frac{h^2}{6}(M_{i+1} - M_i) \implies A_i = \frac{f_{i+1} - f_i}{h} - \frac{h}{6}(M_{i+1} - M_i).$$

Tím je konstrukce splinu na intervalu I_i hotova (momenty M známe z řešení soustavy).



Obrázek 45: Ukázka přirozeného kubického splinu interpolujícího množinu bodů.

12.2.2 Konvergence

Věta 12.3: *Odhad chyby kubického splinu pro $f \in \mathcal{C}^4$*

Nechť $f \in \mathcal{C}^4[a,b]$. Pak existují konstanty C_0, C_1, C_2, C_3 nezávislé na f a h takové, že pro $j = 0, 1, 2, 3$ platí:

$$\max_{x\in[a,b]} |f^{(j)}(x) - S^{(j)}(x)| \leq C_j \max_{x\in[a,b]} |f^{(4)}(x)| \cdot h^{4-j}.$$

Poznámka 12.1 (Konstanty pro odhad chyby kubického splinu)**.** Konstanty pro odhad chyby kubického splinu jsou $C_0 = \frac{5}{384}, C_1 = \frac{1}{24}, C_2 = \frac{3}{8}$.

Poznámka 12.2 (Odhad chyby kubického splinu pro $f \in \mathcal{C}^2$)**.** Nechť $f \in \mathcal{C}^2[a,b]$. Pak existuje konstanta $C > 0$ nezávislá na f a h taková, že

$$\max_{x\in[a,b]} |f(x) - S(x)| \leq Ch^2 \max_{x\in[a,b]} |f''(x)|.$$

Věta 12.4: *Konvergence pro spojité funkce*

Mějme posloupnost rovnoměrných dělení $\{D_n\}_{n=1}^{\infty}$ intervalu $[a,b]$. Nechť S_n je přirozený kubický spline na dělení D_n. Pak pro libovolnou funkci $f \in \mathcal{C}^0[a,b]$ platí $S_n \rightrightarrows f$ na $[a,b]$.

Poznámka 12.3 (Spline zachovává monotonii a konvexitu)**.** Spline S zachovává kvalitativní vlastnosti funkce f:

- Pokud je f konvexní/konkávní a h je dostatečně malé, pak je i S konvexní/konkávní.
- Pokud je f monotónní a h je dostatečně malé, pak je i S monotónní.

13 Kvadratura

Úloha: Dána funkce $f : [a, b] \to \mathbf{R}$, aproximuj integrál $I(f) := \int_a^b f(x)\,\mathrm{d}x$.

Motivace

- Pro některé funkce nelze primitivní funkci vyjádřit konečnou kombinací elementárních funkcí (Liouville 1833). Příklady: e^{-x^2}, $\frac{\sin x}{x}$, x^x.
- Pro některé funkce je primitivní funkce neprakticky složitá. Příklad: $\frac{x}{1+20\cos x}$.

Aproximace Hledáme aproximaci ve tvaru kvadraturního vzorce:

$$I(f) \approx Q(f) := \sum_{i=0}^{n} w_i f(x_i),$$

kde:

- $x_0 < x_1 < \cdots < x_n$ jsou uzly (obvykle $\in [a, b]$, není nutné).
- $w_i \in \mathbf{R}$ jsou váhy.

DEFINICE 13.1: *Obecný kvadraturní vzorec*

Nechť $\{x_i\}_{i=0}^n$ jsou uzly a $\{w_i\}_{i=0}^n$ jsou váhy. Pak

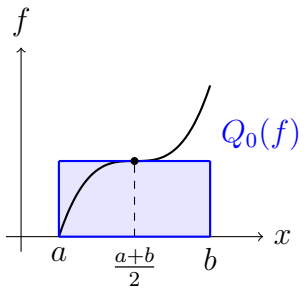
$$Q(f) = \sum_{i=0}^{n} w_i f(x_i),$$

nazýváme **obecný kvadraturní vzorec**.

Příklad 13.1 (**Obdélníkové pravidlo** - Midpoint rule)**.** Aproximujeme plochu pod grafem obdélníkem o výšce $f\left(\frac{a+b}{2}\right)$.

$$I(f) \approx (b-a) f\left(\frac{a+b}{2}\right) =: Q_0(f).$$

Toto pravidlo je přesné pro $f \in \mathcal{P}_1$ (lineární funkce), ale už ne pro $f \in \mathcal{P}_2$.



Obrázek 46: Obdélníkové pravidlo

Obecná konstrukce Funkci f aproximujeme Lagrangeovým interpolačním polynomem L_n na uzlech $\{x_i\}_{i=0}^n$ a ten zintegrujeme.

$$f(x) \approx L_n(x) = \sum_{i=0}^{n} f(x_i) l_i(x), \quad \text{kde } l_i(x) = \prod_{\substack{k=0 \\ k \neq i}}^{n} \frac{x - x_k}{x_i - x_k}.$$

Pak aproximujeme

$$\int_a^b f(x)\,\mathrm{d}x \approx \int_a^b L_n(x)\,\mathrm{d}x = \int_a^b \sum_{i=0}^{n} f(x_i) l_i(x)\,\mathrm{d}x = \sum_{i=0}^{n} f(x_i) \underbrace{\int_a^b l_i(x)\,\mathrm{d}x}_{=:w_i} = \sum_{i=0}^{n} w_i f(x_i),$$

kde takto definované váhy již nezávisí na f, pouze na pozici uzlů a a, b.

DEFINICE 13.2: *Kvadraturní vzorec určený uzly Lagrangeovy interpolace*

Nechť $\{x_i\}_{i=0}^n$ jsou uzly. Pak

$$Q(f) := \int_a^b L_n(x)\,\mathrm{d}x = \sum_{i=0}^{n} w_i f(x_i)$$

nazýváme **kvadraturní vzorec určený uzly** $\{x_i\}_{i=0}^n$. Váhy jsou určené lagrangeovskými bázovými funkcemi: $w_i = \int_a^b l_i(x)\,\mathrm{d}x$.

Poznámka 13.1 (Kvadratura je přesná pro $\mathcal{P}_n$)**.** Takto konstruovaný kvadraturní vzorec je automaticky přesný pro všechny polynomy stupně nejvýše n (protože pro $f \in \mathcal{P}_n$ platí $f = L_n$):

$$I(p) = Q(p), \quad \forall p \in \mathcal{P}_n.$$

13.1 Newton-Cotesovy vzorce

Uzly volíme rovnoměrně na intervalu $[a, b]$:

$$x_i = a + \frac{i}{n}(b - a), \quad i = 0, \ldots, n.$$

(Tedy $x_0 = a, x_n = b$ pro uzavřené Newton-Cotesovy vzorce).

Příklad 13.2 (**Lichoběžníkové pravidlo** ($n = 1$))**.** Uzly $x_0 = a, x_1 = b$. Váhy:

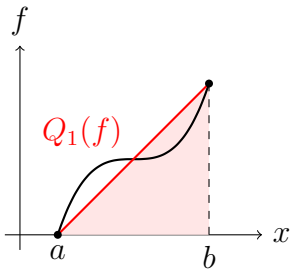
$$w_0 = \int_a^b l_0(x)\,\mathrm{d}x = \int_a^b \frac{x - b}{a - b}\,\mathrm{d}x = \frac{1}{a - b}\left[\frac{1}{2}(x - b)^2\right]_a^b = \frac{1}{a - b}\left(0 - \frac{1}{2}(a - b)^2\right) = \frac{b - a}{2}$$

$$w_1 = \int_a^b l_1(x)\,\mathrm{d}x = \cdots = \frac{b - a}{2} = w_0$$

(Ze symetrie nebo výpočtu). Vzorec:

$$I(f) \approx Q_1(f) = \frac{b - a}{2}(f(a) + f(b)).$$

Geometricky odpovídá obsahu lichoběžníku pod sečnou grafu. Přesné pro $f \in \mathcal{P}_1$, ne pro $\mathcal{P}_2$.



Obrázek 47: Lichoběžníkové pravidlo

Příklad 13.3 (**Simpsonovo pravidlo** ($n = 2$))**.** Uzly: $x_0 = a$, $x_1 = \frac{a+b}{2}$, $x_2 = b$. Váhy:

$$w_0 = \int_a^b \frac{(x - x_1)(x - x_2)}{(x_0 - x_1)(x_0 - x_2)}\,\mathrm{d}x = \cdots = \frac{b-a}{6}, \quad \text{atd.}$$

Analogicky ze symetrie $w_2 = w_0 = \frac{b-a}{6}$ a $w_1 = \frac{4(b-a)}{6}$. Vzorec:

$$I(f) \approx Q_S(f) = \frac{b-a}{6}\left(f(a) + 4f\left(\frac{a+b}{2}\right) + f(b)\right).$$

Toto pravidlo je přesné pro polynomy stupně 2 ($\mathcal{P}_2$), ale překvapivě i pro $\mathcal{P}_3$!

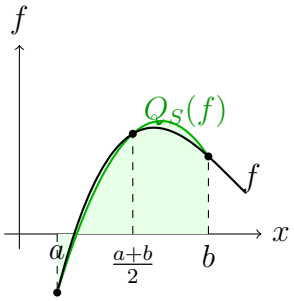
VĚTA 13.1: *Simpsonovo pravidlo je přesné pro* $\mathcal{P}_3$

$$Q_S(p) = I(p), \quad \forall p \in \mathcal{P}_3.$$

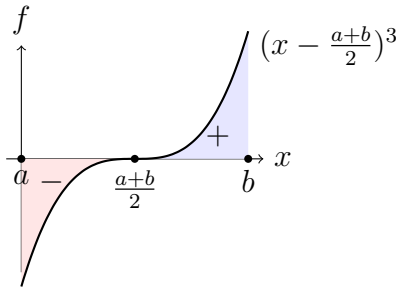
Důkaz. Stačí ukázat, že chyba je nulová pro speciální kubickou funkci $f(x) = (x - \frac{a+b}{2})^3$.

- Integrál: $\int_a^b (x - \frac{a+b}{2})^3\,\mathrm{d}x = 0$ (lichá funkce integrovaná přes symetrický interval).
- Kvadratura: $Q_S(f) = \frac{b-a}{6}\,(f(a) + 4 \cdot 0 + f(b)) = 0$, protože $f(a) = -f(b)$.

Libovolnou funkci $p \in \mathcal{P}_3$ lze vyjádřit jako $p(x) = \alpha(x - \frac{a+b}{2})^3 + \beta x^2 + \gamma x + \delta$. Každý z těchto členů Q_S integruje přesně (kvadratické a nižší z konstrukce, kubický viz výše). □



(a) Simpsonovo pravidlo



(b) Lichá funkce na symetrickém intervalu

Obrázek 48: Simpsonovo pravidlo a ilustrace liché funkce na symetrickém intervalu.

Další pravidla

- $n = 3$: Simpsonovo 3/8 pravidlo. Přesné pro $\mathcal{P}_3$, ne pro $\mathcal{P}_4$.
- $n = 4$: Booleovo pravidlo. Přesné pro $\mathcal{P}_5$.

13.2 Chyba kvadraturních vzorců

DEFINICE 13.3: *Algebraický řád kvadratury*

Řekneme, že kvadraturní vzorec Q má (algebraický) **řád** $N \in \mathbf{N}_0$, pokud N je největší takové, že $Q(p) = I(p)$ pro všechny $p \in \mathcal{P}_N$.

VĚTA 13.2: *Řád Newton-Cotesových vzorců*

Newton-Cotesův vzorec s uzly $x_0, \ldots, x_n$ má řád:

- n, pokud je n liché.
- $n + 1$, pokud je n sudé.

Důkaz. Pro n sudé (uzly symetrické, $w_i = w_{n-i}$):

$$\int_a^b \left(x - \frac{a+b}{2}\right)^{n+1} \mathrm{d}x = 0 = Q(f)$$

Integrál je 0 (lichá funkce), kvadratura je 0 (symetrické váhy a liché hodnoty v uzlech se odečtou, symetrie vah - viz cvičení). □

Poznámka 13.2 (Minimální algebraický řád)**.** Poznámku 13.2 můžeme formulovat tak, že řád kvadratury (založené na Lagrangeově interpolaci L_n) má řád $\geq n$.

VĚTA 13.3: *Odhad chyby obecné kvadratury*

Nechť Q je kvadratura s obecnými uzly $x_0, \ldots, x_n$. Nechť $f \in \mathcal{C}^{n+1}[a, b]$. Pak

$$|I(f) - Q(f)| \leq \frac{1}{(n+1)!} \max_{x \in [a,b]} |f^{(n+1)}(x)| \int_a^b \left| \prod_{i=0}^{n} (x - x_i) \right| \mathrm{d}x.$$

Důkaz.

$$|I(f) - Q(f)| = |I(f) - I(L_n)| = \left| \int_a^b (f(x) - L_n(x))\,\mathrm{d}x \right| \leq \int_a^b |f(x) - L_n(x)|\,\mathrm{d}x$$

Použijeme odhad chyby interpolace (Věta 11.3):

$$\begin{aligned} \int_a^b |f(x) - L_n(x)| \mathrm{d}x &\leq \int_a^b \frac{1}{(n+1)!} |f^{(n+1)}(\xi_x)| \left| \prod_{i=0}^{n} (x - x_i) \right| \mathrm{d}x \\ &\leq \frac{1}{(n+1)!} \max_{x \in [a,b]} |f^{(n+1)}(x)| \int_a^b \left| \prod_{i=0}^{n} (x - x_i) \right| \mathrm{d}x. \end{aligned}$$

□

Důsledek 13.1 (Hrubý odhad chyby)**.** Ve Větě 13.3 můžeme odhadnout $|x - x_i| \leq b - a$, dále $|\int_a^b (b-a)^{n+1} \mathrm{d}x| \leq (b-a)^{n+2}$, čímž dostaneme hrubý odhad

$$|I(f) - Q(f)| \leq \frac{(b-a)^{n+2}}{(n+1)!} \max_{x \in [a,b]} |f^{(n+1)}(x)|.$$

Důsledek 13.2 (Odhad chyby lichoběžníkového pravidla)**.** Pro lichoběžníkové pravidlo $(n = 1)$ můžeme spočíst $|\int_a^b (x-a)(b-x)\mathrm{d}x| \leq \frac{1}{12}(b-a)^3$, takže z Věty 13.3 dostaneme

$$|I(f) - Q_L(f)| \leq \frac{1}{12}(b-a)^3 \max_{x \in [a,b]} |f''(x)|.$$

Důsledek 13.3 (Odhad chyby Simpsonova pravidla)**.** Simpsonovo pravidlo na uzlech $a, \frac{a+b}{2}, b$ integruje přesně funkce z $\mathcal{P}_3$. Standardní odhad chyby pro $n = 2$ (založený na interpolaci $\mathcal{P}_2$) by vedl na chybu řádu $(b-a)^4$ závislou na f'''. Jelikož je však Q_S přesné i pro $\mathcal{P}_3$, lze dokázat lepší odhad chyby:

$$|I(f) - Q_S(f)| \leq \frac{1}{2880}(b-a)^5 \max_{x \in [a,b]} |f^{(4)}(x)|$$

kde $Q_S(f)$ je Simpsonova kvadratura a $I(f)$ je přesný integrál.

Poznámka 13.3 (Odhad pro vyšší řády)**.** Odhad z Věty 13.3 lze zlepšit, pokud má kvadratura řád N>n (viz předchozí poznámka). Pak se dá dokázat:

$$|I(f) - Q(f)| \leq \frac{(b-a)^{N+2}}{(N+1)!} \max_{x \in [a,b]} |f^{(N+1)}(x)|.$$

Speciální případ věty tohoto typu si dokážeme pro Gaussovu kvadraturu (Věta 13.5).

13.3 Gaussova kvadratura

Chceme najít uzly $\{x_i\}_{i=0}^n$ a váhy $\{w_i\}_{i=0}^n$ tak, aby kvadraturní formule

$$Q(f) = \sum_{i=0}^{n} w_i f(x_i)$$

měla co nejvyšší řád přesnosti. Celkem máme $2(n+1)$ neznámých (uzly a váhy), takže můžeme očekávat řád přesnosti $2n+1$ (polynom tohoto stupně je také určen $2(n+1)$ parametry). Pro jednoduchost uvažujme interval $[-1, 1]$, jinak uděláme lineární substituci.

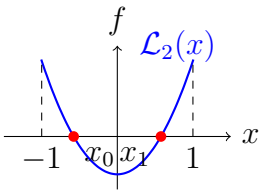
Příklad 13.4 (Gaussova kvadratura pro $n = 1$)**.** Pro $n = 1$ hledáme formuli

$$\int_{-1}^{1} f(x)\,\mathrm{d}x \approx w_0 f(x_0) + w_1 f(x_1).$$

Chceme, aby integrovala přesně bázové polynomy $1, x, x^2, x^3$ (tímpádem integruji přesně obecný polynom 3. stupně). To vede na soustavu nelineárních rovnic pro uzly a váhy. Řešením jsou váhy $w_0 = w_1 = 1$ a uzly $x_{0,1} = \pm\frac{\sqrt{3}}{3}$. Formule má tvar:

$$Q(f) = f\left(-\frac{\sqrt{3}}{3}\right) + f\left(\frac{\sqrt{3}}{3}\right),$$

což je kvadratura řádu 3.



Obrázek 49: Legendreův polynom $\mathcal{L}_2(x)$ a jeho kořeny, které tvoří uzly Gaussovy kvadratury pro $n = 1$.

Definice 13.4: *Gaussova kvadratura*

Kvadraturní formuli s uzly $\{x_i\}_{i=0}^n$ a vahami $\{w_i\}_{i=0}^n$ nazveme Gaussovou kvadraturou, pokud má nejvyšší možný řád $2n + 1$.

Uzly a váhy můžeme získat buď řešením příslušných nelineárních soustav rovnic (náročné a nepraktické), anebo použijeme následující větu.

Věta 13.4: *Gaussova kvadratura a ortogonální polynomy*

Nechť $\{x_i\}_{i=0}^n$ jsou kořeny Legendreova polynomu $\mathcal{L}_{n+1}$ (tj. polynomu ortogonálního na $[-1, 1]$ s vahou 1 ke všem polynomům stupně nejvýše n). Pak příslušná kvadraturní formule Q je Gaussova, tj. má řád $2n + 1$.

Důkaz. Nechť $p \in \mathcal{P}_{2n+1}$. Chceme ukázat, že $Q(p) = I(p)$. Vydělíme polynom p polynomem $\mathcal{L}_{n+1}$ se zbytkem:

$$p(x) = \mathcal{L}_{n+1}(x)q_n(x) + r_n(x),$$

kde $q_n, r_n \in \mathcal{P}_n$. Pak pro přesný integrál platí:

$$I(p) = \underbrace{\int_{-1}^{1} \mathcal{L}_{n+1}(x)q_n(x)\,\mathrm{d}x}_{=0,\text{ neboť } \mathcal{L}_{n+1}\perp\mathcal{P}_n} + \int_{-1}^{1} r_n(x)\,\mathrm{d}x = \int_{-1}^{1} r_n(x)\,\mathrm{d}x = I(r_n),$$

kde jsme využili ortogonalitu $\mathcal{L}_{n+1}$ na polynomy nižších stupňů.

Pro kvadraturní formuli platí:

$$Q(p) = \sum_{i=0}^{n} w_i p(x_i) = \sum_{i=0}^{n} w_i (\underbrace{\mathcal{L}_{n+1}(x_i)}_{0} q_n(x_i) + r_n(x_i)) = \sum_{i=0}^{n} w_i r_n(x_i) = Q(r_n).$$

Protože Q má řád alespoň n (Pozn. 13.2), platí $Q(r_n) = I(r_n)$. Celkem tedy

$$Q(p) = Q(r_n) = I(r_n) = I(p).$$

□

Poznámka 13.4 (Kořeny Legendreova polynomu)**.** Pro $n = 1$ platí $\mathcal{L}_2(x) = \frac{1}{2}(3x^2 - 1)$. Kořeny jsou $3x^2 - 1 = 0 \iff x = \pm\frac{1}{\sqrt{3}} = \pm\frac{\sqrt{3}}{3}$, což odpovídá výsledku z Př. 13.4. Pro vyšší řády neexistují rozumně použitelné vzorce pro kořeny $\mathcal{L}_{n+1}$. Existují ale extrémně efektivní dedikované numerické metody pro jejich výpočet.

VĚTA 13.5: *Odhad chyby Gaussovy kvadratury*

Nechť $f \in \mathcal{C}^{2n+2}[a,b]$. Pak pro chybu Gaussovy kvadratury na intervalu $[a,b]$ platí:

$$|I(f) - Q(f)| \leq \frac{1}{(2n+2)!}(b-a)^{2n+3} \max_{x\in[a,b]} |f^{(2n+2)}(x)|.$$

Důkaz. **1)** K uzlům $x_0, \ldots, x_n$ přidáme libovolné další uzly $x_{n+1}, \ldots, x_{2n+1} \in (a,b)$, které jsou navzájem různé a různé od $x_0, \ldots, x_n$.

2) Označme L_n Lagrangeův interpolační polynom funkce f stupně n na uzlech $x_0, \ldots, x_n$ a L_{2n+1} Lagrangeův interpolační polynom funkce f stupně $2n+1$ na uzlech $x_0, \ldots, x_{2n+1}$.

3) Platí $L_n(x_i) = L_{2n+1}(x_i) = f(x_i)$ pro všechna $i = 0, \ldots, n$. Tedy rozdíl $L_n - L_{2n+1}$ má kořeny $x_0, \ldots, x_n$ a můžeme jej psát ve tvaru:

$$L_n(x) - L_{2n+1}(x) = (x-x_0)\cdots(x-x_n)p_n(x), \quad p_n \in \mathcal{P}_n.$$

Výraz $(x-x_0)\cdots(x-x_n)$ je až na konstantu roven ortogonálnímu polynomu $\mathcal{L}_{n+1}$ přeškálovanému na $[a,b]$ ($x_0, \ldots, x_n$ jsou kořeny $\mathcal{L}_{n+1}$), který je kolmý na $\mathcal{P}_n$. Tedy:

$$\int_a^b (L_n(x) - L_{2n+1}(x))\,\mathrm{d}x = \int_a^b \underbrace{(x-x_0)\cdots(x-x_n)}_{\propto \mathcal{L}_{n+1}} p_n(x)\,\mathrm{d}x = 0.$$

Z toho plyne klíčová rovnost:

$$Q(f) = \int_a^b L_n(x)\,\mathrm{d}x = \int_a^b L_{2n+1}(x)\,\mathrm{d}x.$$

4) Chybu kvadratury pak můžeme vyjádřit jako chybu interpolace polynomem stupně $2n+1$:

$$|I(f) - Q(f)| = \left| \int_a^b f(x)\,\mathrm{d}x - \int_a^b L_{2n+1}(x)\,\mathrm{d}x \right| \leq \int_a^b |f(x) - L_{2n+1}(x)|\,\mathrm{d}x.$$

Použijeme Větu 11.3 zároveň s nerovností $\left|\prod_{i=0}^{2n+1}(x-x_i)\right| = \prod_{i=0}^{2n+1}|x-x_i| \leq (b-a)^{2n+2}$:

$$\int_a^b |f(x) - L_{2n+1}(x)|\,\mathrm{d}x \leq \int_a^b \frac{1}{(2n+2)!} \max_{t\in[a,b]} |f^{(2n+2)}(t)|(b-a)^{2n+2}\,\mathrm{d}x,$$

kde jsme odstranili všechny výskyty x z integrálu, a dostáváme tvrzení věty.

$$|I(f) - Q(f)| \leq \frac{1}{(2n+2)!}(b-a)^{2n+3} \max_{x\in[a,b]} |f^{(2n+2)}(x)|.$$

□

Poznámka 13.5 (Vlastnosti vah a stabilita)**.** Pro Gaussovu kvadraturu platí, že všechny váhy jsou kladné:

$$w_i > 0 \quad \forall i.$$

Díky tomu je součet absolutních hodnot vah omezený:

$$\sum_{i=0}^{n} |w_i| = \sum_{i=0}^{n} w_i = \int_a^b 1\,\mathrm{d}x = b - a.$$

To zajišťuje numerickou stabilitu metody i pro vysoká n. Naproti tomu pro Newton-Cotesovy vzorce (na ekvidistantních uzlech) platí, že váhy můžou být kladné i záporné a

$$\sum_{i=0}^{n} |w_i| \to \infty \quad \text{pro } n \to \infty.$$

Vysoké řády Newton-Cotesových vzorců jsou proto numericky nestabilní.

Poznámka 13.6 (Clenshaw-Curtisova kvadratura)**.** Alternativou ke Gaussově kvadratuře je *Clenshaw-Curtisova kvadratura*. Ta využívá uzly v kořenech Čebyševova polynomu. Řád je stejný jako u Newton-Cotese (n, resp. $n+1$), ale má optimální chybu interpolace (viz. definice 11.3). Výhodou je, že pro uzly existují explicitní vzorce (Pozn. 10.3). Pro $n = 1$ dostáváme $Q(f) = f(\frac{-\sqrt{2}}{2}) + f(\frac{\sqrt{2}}{2})$ na intervalu $[-1, 1]$. Pro některé funkce je lepší Gauss, pro jiné Clenshaw-Curtis.

13.4 Složené kvadraturní vzorce

Motivace Interpolace polynomy na celém intervalu $[a, b]$ nemusí fungovat dobře pro $n \to \infty$ (Rungeho jev). Na druhou stranu, v odhadech chyby kvadratury se vyskytuje člen $(b-a)^{N+2}$, kde N je řád kvadratury. Kdyby $b - a$ bylo malé číslo, bude tento člen velmi malým. Idea je využít aditivity integrály vzjledem k mezím. Rozdělíme proto interval $[a, b]$ na menší podintervaly:

$$a = x_0 < x_1 < \cdots < x_m = b.$$

Pro jednoduchost uvažujeme rovnoměrné dělení s krokem $h = \frac{b-a}{m}$, tj. $x_i = a + ih$. Pak platí:

$$\int_a^b f(x)\,\mathrm{d}x = \sum_{i=1}^{m} \int_{x_{i-1}}^{x_i} f(x)\,\mathrm{d}x.$$

Na každém podintervalu $[x_{i-1}, x_i]$ aproximujeme funkci oblíbenou kvadraturou $Q_{[x_{i-1},x_i]}(f)$.

Složené lichoběžníkové pravidlo Na každém intervalu použijeme lichoběžníkové pravidlo:

$$\int_{x_{i-1}}^{x_i} f(x)\,\mathrm{d}x \approx \frac{1}{2}h\big(f(x_{i-1}) + f(x_i)\big).$$

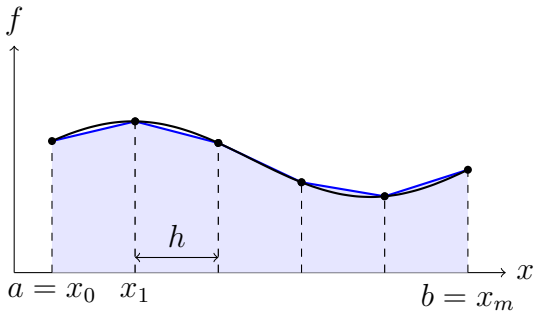
Sečtením dostáváme:

$$\int_a^b f(x)\,\mathrm{d}x \approx h\left[\frac{1}{2}f(a) + \sum_{i=1}^{m-1} f(a+ih) + \frac{1}{2}f(b)\right].$$

VĚTA 13.6: *Chyba složeného lichoběžníkového pravidla*

Nechť $f \in \mathcal{C}^2[a, b]$ a Q je složené lichoběžníkové pravidlo s krokem h. Pak

$$|I(f) - Q(f)| \le \frac{1}{12}h^2(b-a) \max_{x\in[a,b]} |f''(x)|.$$



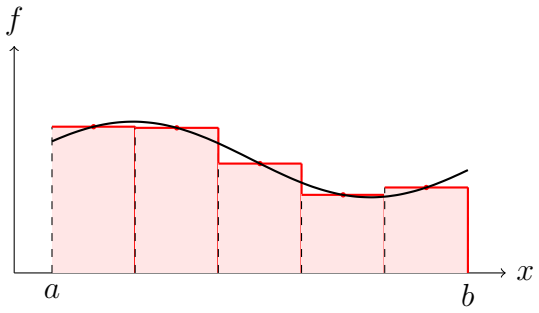
Obrázek 50: Složené lichoběžníkové pravidlo: aproximace funkce lomenou čarou.

Důkaz. Chyba na jednom podintervalu $[x_{i-1}, x_i]$ je (Důsledek 13.2) omezena hodnotou $\frac{1}{12}h^3 \max_{[x_{i-1},x_i]} |f''|$. Celková chyba je součtem chyb na podintervalech:

$$|I(f) - Q(f)| = \left|\sum_{i=1}^{m}(I_i - Q_i)\right| \leq \sum_{i=1}^{m} \frac{1}{12}h^3 \max_{[x_{i-1},x_i]} |f''| \leq \frac{1}{12}h^3 m \max_{[a,b]} |f''|.$$

Protože $mh = b - a$, platí $h^3 m = h^2(mh) = h^2(b-a)$, čímž dostáváme tvrzení. □

Složené obdélníkové pravidlo Složené obdélníkové pravidlo (s uzlem ve středu intervalu) odpovídá Riemannovu součtu.



Obrázek 51: Složené obdélníkové pravidlo.

Složené Simpsonovo pravidlo Uvažujme dělení s krokem h, ale pro aplikaci Simpsonova pravidla potřebujeme interval délky $2h$ (tři uzly). Na intervalu $[x_{2i}, x_{2i+2}]$:

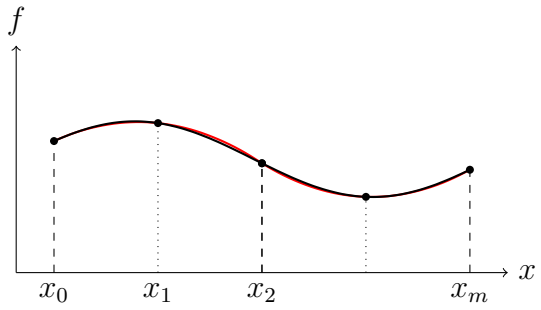
$$\int_{x_{2i}}^{x_{2i+2}} f(x)\,\mathrm{d}x \approx \frac{2h}{6}\left[f(x_{2i}) + 4f(x_{2i+1}) + f(x_{2i+2})\right] = \frac{h}{3}\left[f(x_{2i}) + 4f(x_{2i+1}) + f(x_{2i+2})\right].$$

Celkový vzorec (pro m sudé, $b = x_m$):

$$\int_a^b f(x)\,\mathrm{d}x \approx \frac{h}{3}\left[f(a) + 4f(x_1) + 2f(x_2) + 4f(x_3) + \cdots + 2f(x_{m-2}) + 4f(x_{m-1}) + f(b)\right].$$

Koeficienty u funkčních hodnot se střídají: $1, 4, 2, 4, 2, \ldots, 4, 1$.

Poznámka 13.7 (Rychlost konvergence)**.** Složené Simpsonovo pravidlo má chybu $O(h^4)$. To znamená, že zdvojnásobení počtu intervalů ($2\times$ menší h) vede ke zhruba $2^4 = 16\times$ menší chybě. Složené lichoběžníkové pravidlo má chybu pouze $O(h^2)$ ($4\times$ zmenšení chyby).



Obrázek 52: Složené Simpsonovo pravidlo: aproximace pomocí parabol na dvojičkách intervalů.

13.5 Metoda poloviční kroku

Motivace: Dosud jsme viděli *apriorní* odhady chyby, tedy odhady získané bez znalosti napočteného výsledku. Existují ovšem i *apostriorní* odhady, které nám řeknou, jak přesný je konkrétní již napočtený výsledek. Nevýhodou apriorních odhadů je nutnost znát derivace funkce a přesné konstanty z odhadů, což v praxi často není k dispozici. U metody poloviční kroku stačí znát, jak se chyba škáluje při zjemňování dělení, zvyšování stupně polynomů, atd.

Uvažujme složenou kvadraturní formuli Q_h s krokem h. Předpokládejme, že chyba této metody se chová jako

$$I(f) - Q_h(f) \approx Ch^N, \tag{7}$$

kde C je neznámá konstanta nezávislá na h a N je známý řád chyby metody (např. $N = 2$ pro lichoběžníkové pravidlo, $N = 4$ pro Simpsonovo pravidlo). Takovýto vztah jsme měli doteď jen jako horní odhad, lze ale dokázat i silnější výsledek (7).

Věta 13.7: *Odhad chyby složené kvadratury metodou poloviční kroku*

Nechť Q_h je kvadraturní vzorec s krokem h a $Q_{h/2}$ je stejný kvadraturní vzorec s krokem $h/2$. Pak

$$I(f) - Q_{h/2}(f) \approx \frac{Q_{h/2}(f) - Q_h(f)}{2^N - 1}$$

Důkaz. Ze (7) plyne

$$I(f) - Q_{h/2}(f) \approx C\left(\frac{h}{2}\right)^N. \tag{8}$$

Toto odečtu od (7), tedy:

$$Q_{h/2}(f) - Q_h(f) \approx Ch^N - C\left(\frac{h}{2}\right)^N = C\left(\frac{h}{2}\right)^N \left(2^N - 1\right).$$

Odtud vyjádříme neznámý člen $C(h/2)^N$:

$$C\left(\frac{h}{2}\right)^N \approx \frac{Q_{h/2}(f) - Q_h(f)}{2^N - 1}.$$

Dosazením do pravé strany (8) dostaneme kýžený výsledek. □

Výsledek Věty 13.7 není striktní odhad zeshora, ale pravá strana v praxi velmi dobře aproximuje pravou. Symbolu '$\approx$' lze dát velmi přesný matematický význam a vše dokázat zcela rigorózně.

Poznámka 13.8 (Extrapolovaná kvadratura)**.** Člověka napadne, zda nelze výsledek Věty 13.7 použít jako lepší aproximaci integrálu:

$$I(f) \approx Q_{h/2}(f) + \frac{Q_{h/2}(f) - Q_h(f)}{2^N - 1}.$$

Ano, lze! Pokud např. za Q_h vezmu složené lichoběžníkové pravidlo, vzorec na pravé straně odpovídá složenému Simpsonovi. Pokud za Q_h vezmu složeného Simpsona, dostanu složené Booleovo pravidlo, atd.

14 Jednokrokové metody řešení ODR

Úloha: Aproximovat numerické řešení počáteční úlohy pro obyčejnou diferenciální rovnici. Hledám $y : [a, b] \to \mathbf{R}$ splňující $y'(x) = f(x, y(x))$, s počáteční podmínkou $y(a) = y_0$.

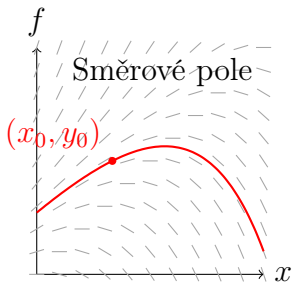
Příklad 14.1 (Praktické příklady diferenciálních rovnic)**.**

1. Růst populace: $y'(t) = ky(t)$. Počáteční úlohy: $y(0) = y_0 \in \mathbf{R} \implies y(t) = y_0 e^{kt}$.
2. Růst populace s omezenými zdroji (Verhulst): $y'(t) = ky(t)(1 - y(t))$. Růst úměrný velikosti populace a množství dostupných zdrojů.
3. Newtonovská mechanika: $F = ma$, tj. $mx''(t) = F$.
4. Hodgkin-Huxley: 3 nelineární ODR pro reakci neuronu na podnět (Nobelova cena).
5. $y'(x) = x^2 + y^2$: $\exists$ řešení, nelze zapsat konečnou kombinací elementárních funkcí.

Existence ajednoznačnost řešení Pro obecnou diferenciální rovnici

$$y'(x) = f(x, y(x)), \quad y(a) = y_0, \quad y : [a, b] \to \mathbf{R}$$

takovou, že f je spojitá a Lipschitzovsky spojitá vzhledem k y na intervalu $[a, b] \times \mathbf{R}$ existuje (z Picardovy věty) pro každé počáteční podmínky právě jedno řešení.



Obrázek 53: Integrální křivka jednoznačně určená směrovým polem a počáteční podmínkou

Diskretizace obyčejné diferenciální rovnice Dělení $a = x_0 < x_1 < \cdots < x_N = b$

- pro jednoduchost rovnoměrné dělení $x_n = a + nh$, $h =$ krok metody.
- $y(x_n) \approx y_n \in \mathbf{R}$, $n = 0, \ldots, N$.

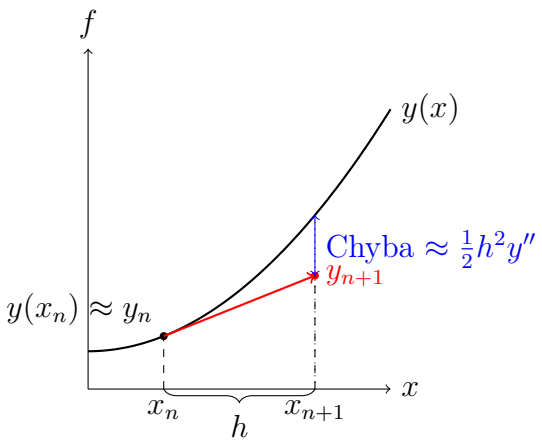
Eulerova metoda

$$\underbrace{y(x_{n+1})}_{\approx y_{n+1}} = y(x_n + h) \overset{\text{Taylor}}{=} \underbrace{y(x_n)}_{\approx y_n} + \underbrace{y'(x_n)}_{\substack{= f(x_n, y(x_n)) \\ \approx f(x_n, y_n)}} \cdot h + \underbrace{\frac{1}{2} h^2 y''(\xi_n)}_{\text{zanedbám}}$$

DEFINICE 14.1: *Eulerova metoda*

$$y_{n+1} = y_n + hf(x_n, y_n), \quad n = 0, \ldots, N - 1,$$

kde y_0 je dáno (počáteční podmínka).



Obrázek 54: Princip Eulerovy metody

Poznámka 14.1 (Explicitní a implicitní metody)**.** Rozlišujeme explicitní a implicitní metodu řešení počáteční úlohy. Pro **Explicitní metodu** Existuje přímý vzorec pro y_{n+1} (např. Euler, Heun). **Implicitní metoda** dává vztah mezi y_{n+1} a y_n, který musíme řešit numericky.

Implicitní Eulerova metoda

$$y_{n+1} = y_n + hf(x_{n+1}, y_{n+1}).$$

Odvození pomocí Taylorova rozvoje „dozadu“ v bodě x_{n+1}:

$$y(x_n) = y(x_{n+1} - h) = y(x_{n+1}) - hy'(x_{n+1}) + O(h^2)$$

$$y(x_{n+1}) \approx y(x_n) + hf(x_{n+1}, y(x_{n+1}))$$

Pokud je f nelineární, tak pro získání y_{n+1} musíme v každém kroku řešit nelineární rovnici (např. Newtonovou metodou). Odpovídá aproximaci derivace zpětnou diferencí: $\frac{y_{n+1}-y_n}{h} \approx y'(x_{n+1}) = f(x_{n+1}, y_{n+1})$. Metoda *1. řádu* (chyba je $O(h)$, zanedbáváme členy $O(h^2)$).

Heunova metoda je modifikovaná Eulerova metoda. Vychází z integrace diferenciální rovnice $y' = f(x, y)$:

$$\int_{x_n}^{x_{n+1}} y'(x)dx = y(x_{n+1}) - y(x_n) = \int_{x_n}^{x_{n+1}} f(x, y(x))dx.$$

Tedy $y(x_{n+1}) = y(x_n) + \int_{x_n}^{x_{n+1}} f(x, y(x))dx$. Tento integrál aproximujeme pomocí kvadratury:

- **Eulerova metoda:** Obdélníkové pravidlo (hodnota v levém kraji intervalu)

$$\int_{x_n}^{x_{n+1}} f(x, y(x))dx \approx hf(x_n, y_n)$$

- **Heunova metoda:** Lichoběžníkové pravidlo

$$\int_{x_n}^{x_{n+1}} f(x, y(x))dx \approx \frac{h}{2}[f(x_n, y_n) + f(x_{n+1}, y_{n+1})]$$

Dostáváme implicitní schéma (protože neznámé y_{n+1} vystupuje i na pravé straně):

$$y_{n+1} = y_n + \frac{h}{2}[f(x_n, y_n) + f(x_{n+1}, y_{n+1})]$$

Abychom získali explicitní metodu, aproximujeme y_{n+1} na pravé straně pomocí Eulerovy metody: $f(x_{n+1}, y_{n+1}) \approx f(x_{n+1}, y_n + hf(x_n, y_n))$, tedy dostaneme schéma

$$y_{n+1} = y_n + \frac{h}{2}\big[f(x_n, y_n) + f(x_{n+1}, y_n + hf(x_n, y_n))\big],$$

což můžeme lépe zapsat takto:

DEFINICE 14.2: *Heunova metoda*

Dáno y_0 (počáteční podmínka), pro $n = 0, \dots, N-1$ počítej

$$\begin{aligned} k_1 &:= f(x_n, y_n), \\ k_2 &:= f(x_{n+1}, y_n + hk_1), \\ y_{n+1} &= y_n + \frac{h}{2}(k_1 + k_2). \end{aligned}$$

Heunova metoda je metoda **2. řádu** (chyba je $O(h^2)$, zanedbáváme členy $O(h^3)$).

Obecné jednokrokové metody: Jednokrokové metody (Euler i Heun) počítají y_{n+1} pouze ze znalosti y_n (oproti vícekrokovým, které využívají historii $y_{n-1}, \dots$).

DEFINICE 14.3: *Jednokroková metoda (JKM)*

Obecný tvar jednokrokové metody:

$$y_{n+1} = y_n + h\Phi(x_n, y_n, h), \quad n = 0, 1, \dots$$

kde Φ se nazývá **přírůstková funkce**.

Příklady přírůstkové funkce Φ:

- **Euler:** $\Phi(x, y, h) = f(x, y)$.
- **Heun:** $\Phi(x, y, h) = \frac{1}{2}[f(x, y) + f(x + h, y + hf(x, y))]$.

Konzistence jednokrokových metod Předpoklad: Přírůstková funkce Φ je spojitá a Lipschitzovsky spojitá vzhledem k y (stejně jako f). Dále je rozumné předpokládat, že Φ odpovídá původní f v limitě $h \to 0>$

DEFINICE 14.4: *Konzistence metody*

Řekneme, že jednokroková metoda určená funkcí Φ je **konzistentní**, pokud

$$\Phi(x, y, 0) = f(x, y) \quad \forall x, y.$$

(Tedy $\lim_{h \to 0+} \Phi(x, y, h) = f(x, y)$).

Konzistence zaručuje, že v limitě $h \to 0+$ řešíme správnou diferenciální rovnici (konzistence s původní rovnicí).

Příklady:

- Eulerova metoda: $\Phi(x, y, h) = f(x, y) \implies \Phi(x, y, 0) = f(x, y)$. Je konzistentní.
- Heunova metoda: $\Phi(x, y, h) = \frac{1}{2}[f(x, y) + f(x + h, y + hf(x, y))]$.

$$\implies \Phi(x, y, 0) = \frac{1}{2}[f(x, y) + f(x, y)] = f(x, y).$$

Je konzistentní.

14.1 Odhad chyby a konvergence

Lokální diskretizační chyba Pro analýzu přesnosti metody zavádíme lokální chybu, která vznikne v jednom kroku metody, pokud vyjdeme z přesného řešení.

DEFINICE 14.5: *Lokální diskretizační chyba*

Nechť $y(x)$ je přesné řešení diferenciální rovnice. **Lokální diskretizační chyba** metody Φ v bodě x s krokem h je definována jako

$$\tau(x, h) = \frac{y(x + h) - y(x)}{h} - \Phi(x, y(x), h).$$

Recept: Dosaď přesné řešení do numerické metody zapsané ve tvaru jako má diferenciální rovnice ($y'(x) - f(x, y(x)) = 0$). Místo nuly dostanu napravo chybu $\tau(x, h)$. Ta typicky odpovídá členům zanedbaným při odvozování.

Příklad 14.2 (Lokální diskretizační chyba pro Eulerovu a Heunovu metodu)**.** Lokální diskretizační chyba pro Eulerovu metodu je

$$\tau(x, h) = \frac{1}{2}hy''(\xi) = O(h), \quad \xi \in (x, x + h).$$

Pro Heunovu metodu platí

$$\tau(x, h) = O(h^2).$$

DEFINICE 14.6: *Řád metody*

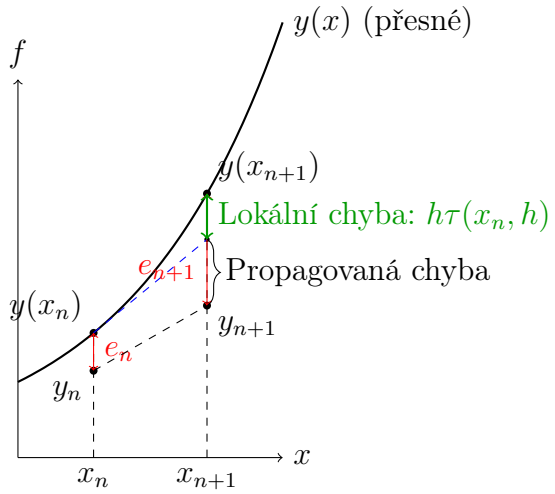
Řekneme, že jednokroková metoda je **řádu** p, jestliže existují konstanty $K, h_0 > 0$ takové, že pro všechna $x \in [a, b]$ a $h \in (0, h_0)$ platí

$$|\tau(x, h)| \leq Kh^p.$$

LEMMA 14.1: *Konzistence jednokrokové metody*

Nechť Φ je přírůstková funkce JKM. Pak JKM je konzistentní právě tehdy, když

$$\lim_{h \to 0+} \tau(x, h) = 0 \quad \forall x.$$



Obrázek 55: Lokální diskretizační chyba vs. globální chyba

Důkaz. ($\Longrightarrow$) Pro $h \to 0$

$$\tau(x,h) = \underbrace{\frac{y(x+h)-y(x)}{h}}_{\to y'(x)} - \underbrace{\Phi(x,y(x),h)}_{\to f(x,y(x))} \longrightarrow y'(x) - f(x,y(x)) = 0.$$

($\Longleftarrow$) Pro $h \to 0$

$$\tau(x,h) = \underbrace{\frac{y(x+h)-y(x)}{h}}_{\to y'(x)=f(x,y(x))} - \underbrace{\Phi(x,y(x),h)}_{\to \Phi(x,y(x),0)} \longrightarrow 0 \implies f(x,y(x)) = \Phi(x,y(x),0).$$

□

Globální chyba a konvergence Lokální diskretizační chyba je jen pomocný nástroj k odhadu toho, co nás ve skutečnosti zajímá: Jak se liší přesné a přibližné řešení, a jestli přibližné řešení konverguje k přesnému (a jak rychle), když $h \to 0$.

DEFINICE 14.7: *Globální chyba*

Globální chyba v bodě x_n je rozdíl mezi přesným řešením a vypočtenou aproximací:

$$e_n = y(x_n) - y_n.$$

Metoda je **konvergentní**, pokud

$$\lim_{h\to 0} \max_{a \le x_n \le b} |e_n| = 0$$

.

VĚTA 14.2: *Odhad chyby jednokrokové metody*

Nechť přírůstková funkce $\Phi(x, y, h)$ je Lipschitzovsky spojitá v proměnné y s konstantou L na oblasti $S = [a, b] \times \mathbf{R}$ a nechť lokální chyba metody splňuje $|\tau(x, h)| \leq Kh^p$ (tj. metoda je řádu p). Potom pro globální chybu platí odhad:

$$\max_{a \leq x_n \leq b} |y(x_n) - y_n| \leq Kh^p \, \frac{e^{L(b-a)} - 1}{L} = O(h^p).$$

Důkaz. Vycházíme z definice metody a definice $\tau(x_n, h)$:

$$\begin{aligned} y_{n+1} &= y_n + h\Phi(x_n, y_n, h) \\ y(x_{n+1}) &= y(x_n) + h\Phi(x_n, y(x_n), h) + h\tau(x_n, h). \end{aligned}$$

Odečtením rovnic (a s označením $e_n = y(x_n) - y_n$) dostaneme:

$$\underbrace{y(x_{n+1}) - y_{n+1}}_{e_{n+1}} = \underbrace{y(x_n) - y_n}_{e_n} + h \underbrace{\left[\Phi(x_n, y(x_n), h) - \Phi(x_n, y_n, h)\right]}_{|\cdot| \leq L|y(x_n) - y_n| = L|e_n|} + h \underbrace{\tau(x_n, h)}_{|\cdot| \leq Kh^p}$$

Převedením na absolutní hodnoty dostaneme pro všechna n:

$$|e_{n+1}| \leq (1 + hL)|e_n| + Kh^{p+1}.$$

Tuto nerovnost aplikujeme opakovaně (rekurentně):

$$\begin{aligned} |e_{n+1}| &\leq (1 + hL)\left[(1 + hL)|e_{n-1}| + Kh^{p+1}\right] + Kh^{p+1} && (9) \\ &= (1 + hL)^2|e_{n-1}| + Kh^{p+1}(1 + (1 + hL)) && (10) \\ &\vdots && (11) \\ &\leq (1 + hL)^{n+1}|e_0| + Kh^{p+1} \sum_{j=0}^{n} (1 + hL)^j. && (12) \end{aligned}$$

Jelikož $e_0 = y(x_0) - y_0 = 0$ (počáteční podmínka je splněna přesně), první člen odpadne. Součet geometrické posloupnosti s kvocientem $q = 1 + hL$ je:

$$\sum_{j=0}^{n} (1 + hL)^j = \frac{(1 + hL)^{n+1} - 1}{(1 + hL) - 1} = \frac{(1 + hL)^{n+1} - 1}{hL}$$

Dosadíme zpět do nerovnosti pro chybu (9):

$$|e_{n+1}| \leq Kh^{p+1} \frac{(1 + hL)^{n+1} - 1}{hL} = Kh^p \, \frac{(1 + hL)^{n+1} - 1}{L}.$$

Využijeme nerovnosti $1 + x \leq e^x$ (pro $x = hL$):

$$(1 + hL)^{n+1} \leq (e^{hL})^{n+1} = e^{hL(n+1)}$$

Protože $x_{n+1} = a + (n + 1)h$, platí $(n + 1)h = x_{n+1} - a \leq b - a$. Tedy:

$$|e_{n+1}| \leq Kh^p \, \frac{e^{L(x_{n+1}-a)} - 1}{L} \leq Kh^p \, \frac{e^{L(b-a)} - 1}{L}.$$

□

VĚTA 14.3: *Vztah konzistence a konvergence*

Nechť přírůstková funkce Φ je Lipschitzovská vzhledem k y. Pak je jednokroková metoda konzistentní právě tehdy, když je konvergentní.

Důkaz. " $\Longrightarrow$ " Jako v minulé větě platí odhad:

$$\max_n |e_n| \;\leq\; \underbrace{\max_n |\tau(x_n,h)|}_{\to 0 \text{ pro konzistentní metodu}} \cdot \frac{e^{L(b-a)}-1}{L} \to 0.$$

" $\Longleftarrow$ " Pokud by $\Phi(x,y,0) = g(x,y) \neq f(x,y)$, pak by řešení, ke kterému metoda konverguje řešilo rovnici $y' = g(x,y)$, což je spor. □

14.2 Absolutní stabilita

Pro jisté typy úloh, tzv. *stiff problémy*, vykazují jednokrokové (resp. explicitní) metody jistou nestabilitu. Aby bylo schéma stabilní, musí platit jisté omezení na krok h. Pro složité problémy z praxe je toto omezení někdy extrémně omezující (absurdně malý krok h) a metoda je tak v praxi téměř nepoužitelná.

Příklad 14.3 (Nestabilita Eulerovy metody)**.** Uvažujme diferenciální rovnici $y' = -10y$ s řešením $y(x) = y_0 e^{-10x}$. Toto řešení je stabilní (pro $y_0 = 0$ je $y \equiv 0$, pro $y_0 \neq 0$ jde k nule), protože derivace pravé strany podle y je záporná: $\frac{\partial f}{\partial y} = -10 < 0$.

Aplikujme Eulerovu metodu $y_{n+1} = y_n + hf(x_n, y_n)$. Zkoumejme šíření chyby $e_n = y(x_n) - y_n$. Přesné řešení splňuje:

$$y(x_{n+1}) = y(x_n) + hf(x_n, y(x_n)) + h\tau(x_n, h).$$

Odečtením rovnice metody dostaneme:

$$e_{n+1} = e_n + h[f(x_n, y(x_n)) - f(x_n, y_n)] + h\tau(x_n, h).$$

S využitím věty o střední hodnotě $f(x_n, y(x_n)) - f(x_n, y_n) = \frac{\partial f}{\partial y}(x_n, \xi_n)e_n$ máme

$$e_{n+1} = \underbrace{\left(1 + h\frac{\partial f}{\partial y}(x_n, \xi_n)\right)}_{A} e_n + h\tau(x_n, h).$$

Veličina A se nazývá **amplifikační faktor**. Aby chyba nerostla exponenciálně s rostoucím n (pro homogenní část rovnice), musí platit **podmínka stability**:

$$|A| < 1.$$

V našem případě $A = 1 - 10h$. Podmínka stability je:

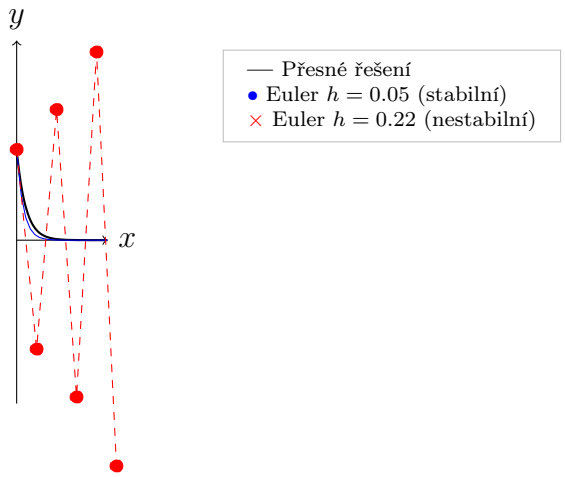
$$|1 - 10h| < 1 \iff -1 < 1 - 10h < 1 \iff -2 < -10h < 0 \iff h < 0.2$$

Pokud $h > 0.2$, metoda je nestabilní a chyba exponenciálně narůstá (osciluje).

DEFINICE 14.8: *A-stabilita*

Jednokroková metoda se nazývá **A-stabilní**, pokud podmínka $|A| < 1$ platí pro všechny problémy s $\frac{\partial f}{\partial y} < 0$ (tj. pro stabilní diferenciální rovnice).

Poznámka 14.2 (A-stabilita Eulerovy metody)**.** Eulerova metoda není A-stabilní, protože pro dané $\lambda = \frac{\partial f}{\partial y} < 0$ vyžaduje $h < \frac{2}{|\lambda|}$. Implicitní Eulerova metoda je A-stabilní (stabilní pro libovolné h).



Obrázek 56: Ukázka stability a nestability Eulerovy metody pro $y' = -10y$. Pro $h = 0.05$ (modrá) chyba klesá, pro $h = 0.22$ (červená) chyba exponenciálně roste (osciluje).

14.3 Konstrukce metod

Metody založené na Taylorově rozvoji. Mějme rozvoj přesného řešení:

$$y(x+h) = y(x) + hy'(x) + \frac{h^2}{2}y''(x) + \frac{h^3}{6}y'''(\xi).$$

Pro derivaci $y'(x)$ platí $y'(x) = f(x, y(x))$. Druhou derivaci získáme derivováním rovnice:

$$(y'(x))' = \frac{\mathrm{d}}{\mathrm{d}x}f(x, y(x)) = \frac{\partial f}{\partial x}(x, y(x)) + \frac{\partial f}{\partial y}(x, y(x)) \cdot y'(x).$$

Dosazením $y'(x) = f(x, y(x))$:

$$y''(x) = \frac{\partial f}{\partial x}(x, y(x)) + \frac{\partial f}{\partial y}(x, y(x)) \cdot f(x, y(x)).$$

Tím dostáváme metodu 2. řádu (Taylorova metoda):

$$y_{n+1} = y_n + hf(x_n, y_n) + \frac{h^2}{2}\left[\frac{\partial f}{\partial x}(x_n, y_n) + \frac{\partial f}{\partial y}(x_n, y_n) \cdot f(x_n, y_n)\right].$$

Přírůstková funkce pro tuto metodu je tedy:

$$\Phi(x, y, h) = f(x, y) + \frac{h}{2}\left[\frac{\partial f}{\partial x}(x, y) + \frac{\partial f}{\partial y}(x, y) \cdot f(x, y)\right].$$

Vlastnosti:

- Metoda je konzistentní.
- Lokální diskretizační chyba $\tau = O(h^2)$, tedy metoda je 2. řádu.
- **Nevýhoda (nepraktické):** Musíme analyticky počítat parciální derivace $\frac{\partial f}{\partial x}$ a $\frac{\partial f}{\partial y}$. Pro vyšší řády vycházejí velmi složité vzorce (musím ještě jednou zderivovat $(y''(x))'$, atd.).

Runge-Kuttovy metody Eulerova metoda využívá jedno vyhodnocení funkce f a dosahuje 1. řádu přesnosti. Heunova metoda má dvě vyhodnocení f a dosahuje 2. řádu. Myšlenka Runge-Kuttových metod: Více vyhodnocení funkce f v jednom kroku může vést k vyššímu řádu přesnosti, aniž bychom museli analyticky počítat derivace f (jako u Taylorovy metody).

Uvažujme obecnou metodu založenou na vyhodnocení f ve dvou různých bodech v blízkosti aktuálního bodu x_n, y_n:

DEFINICE 14.9: *Dvoustupňová Runge-Kuttova metoda*

Dvoustupňová explicitní Runge-Kuttovu metoda je definována přírůstkovou funkcí:

$$\Phi(x, y, h) = w_1 k_1 + w_2 k_2,$$

kde k_i (tzv. vnitřní stupně metody) jsou dány předpisem:

$$\begin{aligned} k_1 &= f(x, y), \\ k_2 &= f(x + \alpha h, y + h\beta k_1). \end{aligned}$$

Zde $w_1, w_2, \alpha, \beta \in \mathbf{R}$ jsou parametry určující konkrétní metodu.

Nyní hledáme koeficienty w_1, w_2, α, β tak, aby metoda měla co nejvyšší řád (v tomto případě 2. řád), tedy chceme, aby lokální diskretizační chyba byla $O(h^2)$:

$$\tau(x, h) = \frac{y(x+h) - y(x)}{h} - \Phi(x, y, h) = O(h^2)$$

Z předchozího odstavce (Taylorova metoda) znám jinou přírůstkovou funkci $\tilde{\Phi}$, o které vím, že je druhého řádu:

$$\tilde{\Phi}(x, y, h) = f(x, y) + \frac{h}{2}\left(\frac{\partial f}{\partial x}(x, y) + \frac{\partial f}{\partial y}(x, y) f(x, y)\right).$$

Pokud splníme podmínku $\Phi(x, y, h) - \tilde{\Phi}(x, y, h) = O(h^2)$, jsme hotovi, pak totiž bude

$$\frac{y(x+h) - y(x)}{h} - \Phi(x, y, h) = \underbrace{\frac{y(x+h) - y(x)}{h} - \tilde{\Phi}(x, y, h)}_{O(h^2)} + \underbrace{\Phi(x, y, h) - \tilde{\Phi}(x, y, h)}_{O(h^2)} = O(h^2),$$

což jsme chtěli dokázat. Pojďme tedy zkusit splnit podmínku splníme podmínku $\Phi - \tilde{\Phi} = O(h^2)$.

Rozvineme k_2 pomocí Taylorova rozvoje funkce dvou proměnných v bodě (x_n, y_n):

$$k_2 = f(x_n + \alpha h, y_n + \beta h k_1) = f(x_n, y_n) + \alpha h \frac{\partial f}{\partial x} + \beta h k_1 \frac{\partial f}{\partial y} + O(h^2).$$

Dosadíme $k_1 = f(x, y)$ a vyjádříme Φ:

$$\begin{aligned} \Phi(x, y, h) &= w_1 f + w_2 \left(f + h\alpha \frac{\partial f}{\partial x} + h\beta f \frac{\partial f}{\partial y}\right) + O(h^2) \\ &= (w_1 + w_2) f + h w_2 \left(\alpha \frac{\partial f}{\partial x} + \beta f \frac{\partial f}{\partial y}\right) + O(h^2) \end{aligned}$$

Porovnáním s $\tilde{\Phi}$ dostaneme soustavu rovnic pro koeficienty u jednotlivých členů:

$$\begin{aligned} w_1 + w_2 &= 1 \quad \text{(koeficient u } f\text{)}, \\ w_2\alpha &= \frac{1}{2} \quad \text{(koeficient u } h\frac{\partial f}{\partial x}\text{)}, \\ w_2\beta &= \frac{1}{2} \quad \text{(koeficient u } hf\frac{\partial f}{\partial y}\text{)}. \end{aligned}$$

Tato soustava má nekonečně mnoho řešení ve tvaru $w_1 = 1 - w_2, \alpha = \beta = 1/(2w_2)$, kde w_2 můžeme zvolit libovolně. Jednou možností je volba $w_2 = w_1 = 1/2, \alpha = 1, \beta = 1$. To přesně odpovídá Heunově metodě:

$$y_{n+1} = y_n + \frac{h}{2}\big(f(x_n, y_n) + f(x_n + h, y_n + hf(x_n, y_n))\big).$$

Celkově jsme tedy odvodili celou třídu metod parametrizovaných pomocí w_2, jednou z nichž je Heunova metoda. Také jsme mimochodem dokázali, že Heun je 2. řádu.

Obecné Runge-Kutta metody Heunova metoda je příkladem 2-stupňové (2-stage) metody. Můžeme konstruovat metody s více stupni pro dosažení vyššího řádu přesnosti.

DEFINICE 14.10: *Obecná explicitní Runge-Kutta metoda*

Pro s-stupňovou (s-stage) explicitní Runge-Kutta metodu je přírůstková funkce definována jako:

$$\Phi(x, y, h) = \sum_{i=1}^{s} w_i k_i,$$

kde koeficienty k_i jsou dány předpisem:

$$\begin{aligned} k_1 &= f(x, y), \\ k_2 &= f(x + \alpha_2 h, y + h\beta_{21} k_1), \\ &\vdots \\ k_i &= f(x + \alpha_i h, y + h\sum_{j=1}^{i-1} \beta_{ij} k_j), \\ &\vdots \\ k_s &= f(x + \alpha_s h, y + h\sum_{j=1}^{s-1} \beta_{sj} k_j). \end{aligned}$$

Koeficienty $\alpha_i, \beta_{ij}, w_i$ určují konkrétní metodu.

Poznámka 14.3 (Runge-Kutta v praxi)**.** V praxi se nejčastěji používají explicitní metody

se $s = 4$, které jsou 4. řádu, např. metoda známá jako RK4:

$$\begin{aligned} y_{n+1} &= y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4), \\ k_1 &= f(x, y), \\ k_2 &= f(x + \tfrac{1}{2}h, y + \tfrac{1}{2}hk_1), \\ k_3 &= f(x + \tfrac{1}{2}h, y + \tfrac{1}{2}hk_2), \\ k_4 &= f(x + h, y + hk_3). \end{aligned}$$

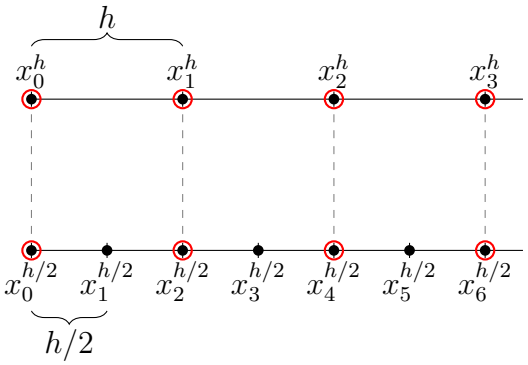
Runge-Kuttovy metody vyššího řádu se příliš nepoužívají, protože počet potřebných stupňů roste rychleji než řád: již pro 5. řád potřebuji 6 stupňů. Je "open problem", kolik nejméně potřebuji stupňů pro dosažení daného řádu ≥ 9.

Poznámka. Moderní konstrukce (a část teorie) Runge-Kuttových metod je překvapivě založena na teorii grafů, konkrétně tzv. kořenových stromů (Butcherovy stromy) a příslušné Butcherovy grupy.

14.4 Metoda poloviční kroku

Metoda poloviční kroku slouží k odhadu chyby již napočteného numerického řešení, obdobně jako u kvadratur v Kapitole 13.5. Opět napočítáme dvě různá řešení (hrubší a jemnější dělení) a z toho extrahujeme informaci o chybě.

Mám dělení s krokem h a uzly x_n^h a numerickým řešením y_n^h napočítaným oblíbenou numerickou metodou. Dále mám dvakrát jemnější dělení s krokem $h/2$ a uzly $x_n^{h/2}$ a numerickým řešením $y_n^{h/2}$. Body sítě se překrývají: $x_n^h = x_{2n}^{h/2}$.



Obrázek 57: Metoda poloviční kroku – zjemnění sítě

Předpokládejme, že globální chyba metody řádu p je úměrná h^p s nějakou neznámou hodnotou C (lze dokázat rigorózně, ne jen jako horní odhad):

$$y(x_n^h) - y_n^h \approx Ch^p. \tag{13}$$

VĚTA 14.4: *Odhad chyby řešení ODR metodou poloviční kroku*

Nechť y_n^h je numerické řešení ODR na dělení s krokem h a nechť $y_{2n}^{h/2}$ je numerické řešení s krokem $h/2$. Obě tato řešení odpovídají společnému bodu $x_n^h = x_{2n}^{h/2}$. Pak

$$y\big(x_{2n}^{h/2}\big) - y_{2n}^{h/2} \approx \frac{y_{2n}^{h/2} - y_n^h}{2^p - 1}.$$

Důkaz. Ze (13) plyne pro zjemněnou síť s krokem $h/2$ v odpovídajícím bodě $x_{2n}^{h/2} = x_n^h$, že

$$y\big(x_{2n}^{h/2}\big) - y_{2n}^{h/2} \approx C\left(\frac{h}{2}\right)^p. \tag{14}$$

Toto odečteme od (13) a s využitím $y(x_n^h) = y\big(x_{2n}^{h/2}\big)$ eliminujeme přesné řešení:

$$y_{2n}^{h/2} - y_n^h \approx Ch^p - C\left(\frac{h}{2}\right)^p = C\left(\frac{h}{2}\right)^p (2^p - 1).$$

Odtud vyjádříme neznámý člen $C(h/2)^N$:

$$C\left(\frac{h}{2}\right)^p \approx \frac{y_{2n}^{h/2} - y_n^h}{2^p - 1}.$$

Dosazením do pravé strany (14) dostaneme kýžený výsledek. □

15 Vícekrokové metody

- Runge-Kuttovy metody dosáhly vyšší přesnosti vyhodnocováním f ve více bodech.
- Zbytečná práce?
- Využít již pracně napočítané hodnoty y a f?

Konkrétně, k-kroková metoda využívá k předchozích hodnot y_i.

Definice 15.1: *k-kroková metoda*

Vícekroková metoda (VKM), resp. k-kroková metoda je dána vztahem

$$\sum_{i=0}^{k} \alpha_i y_{n+i} = h \sum_{i=0}^{k} \beta_i f_{n+i},$$

kde $\alpha_i, \beta_i \in \mathbf{R}$ a $f_i = f(x_{n+i}, y_{n+i})$. Dále předpokládáme, že koeficienty splňují $\alpha_k \neq 0$ a $|\alpha_0| + |\beta_0| > 0$ (metoda využívá skutečně k předchozích kroků).

Příklad 15.1 (Odvození pomocí Simpsonova pravidla)**.** Uvažujme diferenciální rovnici $y' = f(x, y)$. Integrujme přes dva kroky, tj. od x_n do x_{n+2}:

$$\int_{x_n}^{x_{n+2}} y'(x)\,dx = y(x_{n+2}) - y(x_n) = \int_{x_n}^{x_{n+2}} f(x, y(x))\,dx.$$

Pravou stranu aproximujeme pomocí Simpsonova pravidla (kvadraturní vzorec):

$$\int_{x_n}^{x_{n+2}} f(x, y(x))\,dx \approx \frac{1}{3}h\left[f(x_n, y_n) + 4f(x_{n+1}, y_{n+1}) + f(x_{n+2}, y_{n+2})\right].$$

Výsledná metoda:

$$y_{n+2} - y_n = \frac{h}{3}\left[f_n + 4f_{n+1} + f_{n+2}\right].$$

Vlastnosti této metody:

- 2. řád,
- implicitní (hledané nové řešení y_{n+2} je 'schované uvnitř' nelinearity f_{n+2}),
- 2-kroková metoda.
- Pokud začínám z x_0, y_0 a počítám y_2, tak potřebuji i y_1 k vyhodnocení f_1. Kdo mi ho dá? Např. Eulerova či jiná jednokroková metoda.

Vlastnosti a řešení k-krokových metod

Pro start metody je třeba znát k počátečních hodnot. Hodnota y_0 je dána počáteční podmínkou, hodnoty $y_1, \ldots, y_{k-1}$ musíme spočítat jinou metodou, např. vhodnou jednokrokovou metodou (Runge-Kutta).

Podle koeficientu β_k dělíme metody na:

- **Explicitní** ($\beta_k = 0$): Hodnotu y_{n+k} lze vyjádřit přímo z již známých hodnot.
- **Implicitní** ($\beta_k \neq 0$): Ve výrazu pro y_{n+k} se na pravé straně vyskytuje $f(x_{n+k}, y_{n+k})$. Musíme řešit nelineární rovnici pro y_{n+k} např. Newtonovou metodou, anebo...

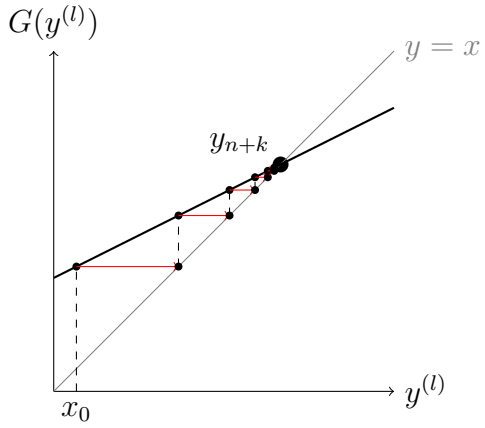
Řešení implicitního problému

Implicitní vztah přepíšeme do tvaru pro metodu prosté iterace:

$$y_{n+k} = \frac{\beta_k}{\alpha_k} h f(x_{n+k}, y_{n+k}) + \frac{1}{\alpha_k} \left[h \sum_{i=0}^{k-1} \beta_i f_{n+i} - \sum_{i=0}^{k-1} \alpha_i y_{n+i} \right] =: G(y_{n+k}).$$

Tento vztah řešíme iteračně. Zvolíme počáteční odhad $y_{n+k}^{(0)}$ (např. pomocí explicitní metody - prediktor) a počítáme posloupnost aproximací:

$$y_{n+k}^{(l+1)} = G(y_{n+k}^{(l)}), \quad l = 0, 1, \ldots.$$



Obrázek 58: Ilustrace iterace $y_{n+k}^{(l+1)} = G(y_{n+k}^{(l)})$ konvergující k pevnému bodu y_{n+k}.

Věta 15.1: *Konvergence k řešení implicitní k-krokové metody*

Nechť funkce f je lipschitzovská s konstantou L vzhledem k proměnné y. Je-li krok h dostatečně malý (konkrétně $h < |\frac{\alpha_k}{L\beta_k}|$), pak zobrazení G je kontrakce. To znamená, že pro libovolnou počáteční volbu $y_{n+k}^{(0)}$ posloupnost iterací $y_{n+k}^{(l)}$ konverguje k řešení y_{n+k} implicitní VKM pro $l \to \infty$.

Důkaz. Ověříme podmínku kontrakce pro zobrazení G:

$$|G(y) - G(\tilde{y})| = \left| \frac{\beta_k}{\alpha_k} h f(x_{n+k}, y) - \frac{\beta_k}{\alpha_k} h f(x_{n+k}, \tilde{y}) \right| \leq \left| \frac{\beta_k}{\alpha_k} \right| hL|y - \tilde{y}|.$$

Aby bylo zobrazení kontrakcí, musí být lipschitzovská konstanta menší než 1:

$$\left| \frac{\beta_k}{\alpha_k} \right| hL < 1 \iff h < \left| \frac{\alpha_k}{L\beta_k} \right|.$$

□

Poznámky:

- Čím menší h, tím rychlejší konvergence.
- Počáteční $y_{n+k}^{(0)}$ se často získává explicitní metodou (prediktor), implicitní metoda je pak ‘korektor’.

15.1 Konstrukce

Adams-Bashforthovy metody - explicitní

Uvažujme integrální tvar diferenciální rovnice $y' = f(x, y)$ na intervalu $[x_n, x_{n+1}]$:

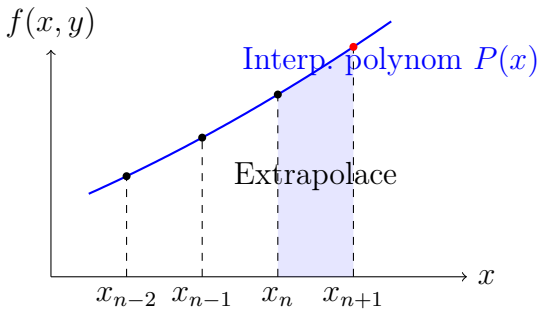
$$y(x_{n+1}) - y(x_n) = \int_{x_n}^{x_{n+1}} y'(x)\,dx = \int_{x_n}^{x_{n+1}} f(x, y(x))\,dx.$$

Neznámou funkci $f(x, y(x))$ nahradíme Lagrangeovým interpolačním polynomem $L_{k-1}(x)$, který interpoluje hodnoty f v k předchozích uzlech $x_n, x_{n-1}, \ldots, x_{n-k+1}$:

$$f(x, y(x)) \approx L_{k-1}(x).$$

Výsledná explicitní metoda má tvar (viz konstrukce Lagrangeovy interpolace):

$$y_{n+1} - y_n = \sum_{i=0}^{k-1} a_i f_{n-i}, \quad \text{kde} \quad a_i = \int_{x_n}^{x_{n+1}} \prod_{\substack{j=0 \\ j\neq i}}^{k-1} \frac{x - x_{n-j}}{x_{n-i} - x_{n-j}}\,dx.$$



Obrázek 59: Geometrický význam Adams-Bashforthovy metody: extrapolace f pomocí polynomu $L_{k-1}(x)$ a integrace pod ním.

Přehled metod pro nízká k

- $k = 1$ **(Eulerova metoda):**
$$y_{n+1} - y_n = h f_n.$$
- $k = 2$**:**
$$y_{n+1} - y_n = h\left(\frac{3}{2} f_n - \frac{1}{2} f_{n-1}\right).$$
- $k = 3$**:**
$$y_{n+1} - y_n = h\left(\frac{23}{12} f_n - \frac{16}{12} f_{n-1} + \frac{5}{12} f_{n-2}\right).$$

Řád k-krokové Adams-Bashforthovy metody je roven k.

Adams-Moultonovy metody - implicitní

Adams-Moultonovy metody jsou implicitní variantou předchozí konstrukce, kde interpolační polynom konstruujeme i s použitím bodu x_{n+1} a dosud neznámého y_{n+1}. Adams-Moultonovy metody mají řád $k+1$.

Příklad 15.2 (Adams-Moultonova metoda pro $k = 1$ (řád 2))**.**

$$y_{n+1} - y_n = h\left(\frac{1}{2}f_{n+1} + \frac{1}{2}f_n\right).$$

Backwards difference formula

Tyto metody jsou implicitní a vycházejí ze vztahu:

$$y'(x_n) \approx f(x_n, y_n).$$

Derivaci nahrazujeme derivací interpolačního polynomu:

$$y'(x_n) \approx L'(x_n),$$

kde L je Lagrangeův interpolační polynom funkce y na k předchozích uzlech. Příkladem je implicitní Euler, anebo metoda BDF2:

$$y_{n+1} - \frac{4}{3}y_n + \frac{1}{3}y_{n-1} = \frac{2}{3}hf_{n+1}.$$

15.2 Lokální diskretizační chyba a řád metody

DEFINICE 15.2: *Lokální diskretizační chyba*

Lokální diskretizační chyba $\tau(x,h)$ k-krokové metody je definována jako

$$\tau(x,h) := \frac{1}{h}\sum_{i=0}^{k}\alpha_i y(x+ih) - \sum_{i=0}^{k}\beta_i f(x+ih, y(x+ih)),$$

kde $y(x)$ je přesné řešení diferenciální rovnice. Tato chyba vyjadřuje, s jakou přesností splňuje přesné řešení diferenční schéma metody (po vydělení krokem h).

DEFINICE 15.3: *Řád vícekrokové metody*

Vícekroková metoda je řádu p, pokud existují konstanty $C, h_0 > 0$ takové, že

$$|\tau(x,h)| \leq Ch^p$$

pro všechna $x \in [a,b]$ a pro všechna $h \in (0,h_0)$.

15.3 Analýza řádu a konzistence

Zkoumejme nyní podmínky, za kterých je k-kroková metoda řádu p. Vyjdeme z Taylorova rozvoje přesného řešení $y(x+ih)$ a jeho derivace $y'(x+ih)$ v bodě x.

Věta 15.2: *Podmínky řádu VKM*

Vícekroková metoda je řádu p právě tehdy, když platí následující rovnosti pro její koeficienty:

$$\sum_{i=0}^{k} \alpha_i = 0,$$
$$\sum_{i=0}^{k} \alpha_i \frac{i^l}{l!} = \sum_{i=0}^{k} \beta_i \frac{i^{l-1}}{(l-1)!} \quad \text{pro } l = 1, 2, \ldots, p.$$

(Poznámka: zde formálně definujeme $0^0 = 0! = \frac{0}{0} = 1$.)

Důkaz. Dosadíme Taylorův rozvoj do definice lokální diskretizační chyby $\tau(x, h)$ (Definice 15.2). Využijeme vztah $f(x, y(x)) = y'(x)$.

$$\begin{aligned}
\tau(x,h) &= \frac{1}{h}\sum_{i=0}^{k} \alpha_i y(x+ih) - \sum_{i=0}^{k} \beta_i y'(x+ih) \\
&= \frac{1}{h}\sum_{i=0}^{k} \alpha_i \sum_{l=0}^{\infty} y^{(l)}(x)\frac{(ih)^l}{l!} - \sum_{i=0}^{k} \beta_i \sum_{j=0}^{\infty} y^{(j+1)}(x)\frac{(ih)^j}{j!} \\
&= \sum_{i=0}^{k} \alpha_i \sum_{l=0}^{\infty} y^{(l)}(x)\frac{i^l}{l!}h^{l-1} - \sum_{i=0}^{k} \beta_i \sum_{l=1}^{\infty} y^{(l)}(x)\frac{i^{l-1}}{(l-1)!}h^{l-1},
\end{aligned}$$

kde jsme v druhé sumě přeznačili index $l = j + 1$ (tedy pokud j jde od 0, pak l je od 1). Nyní seskupíme členy podle derivací $y^{(l)}(x)$.

Pro $l = 0$: Koeficient u $y(x)h^{-1}$ je $\sum_{i=0}^{k} \alpha_i$. Aby chyba neobsahovala člen h^{-1} (což by vedlo k nekonečné chybě pro $h \to 0$), musí platit $\sum \alpha_i = 0$.

Pro $l \geq 1$: Sloučíme členy obsahující $y^{(l)}(x)h^{l-1}$. Z první sumy máme člen pro l (s mocninou h^{l-1}): $\sum \alpha_i \frac{i^l}{l!} y^{(l)}(x)h^{l-1}$, druhé sumy máeme $\sum_{i=0}^{k} \beta_i \frac{i^{l-1}}{(l-1)!} y^{(l)}(x)h^{l-1}$. Celkový koeficient u $y^{(l)}(x)h^{l-1}$ je tedy:

$$C_l = \left[\frac{1}{l!}\sum_{i=0}^{k} \alpha_i i^l - \frac{1}{(l-1)!}\sum_{i=0}^{k} \beta_i i^{l-1}\right].$$

Aby metoda byla řádu p, musí být lokální chyba $\tau(x, h) = O(h^p)$, což znamená, že koeficienty u mocnin $h^{-1}, h^0, \ldots, h^{p-1}$ musí být nulové. To odpovídá podmínkám $C_l = 0$ pro $l = 0, 1, \ldots, p$. □

Definice 15.4: *Konzistentní VKM*

Vícekroková metoda se nazývá **konzistentní**, pokud

$$\lim_{h \to 0+} \tau(x, h) = 0, \quad \forall x \in [a, b].$$

LEMMA 15.3: *Podmínka konzistence*

Vícekroková metoda je konzistentní právě tehdy, když je alespoň 1. řádu, tj. platí:

$$\sum_{i=0}^{k} \alpha_i = 0 \quad \text{a} \quad \sum_{i=0}^{k} i\alpha_i = \sum_{i=0}^{k} \beta_i.$$

Příklad 15.3 (Simpsonovo pravidlo)**.** Uvažujme metodu danou vztahem (odvozenou v Příkladu 15.1):

$$y_{n+2} - y_n = \frac{h}{3}[f_n + 4f_{n+1} + f_{n+2}].$$

Jedná se o 2-krokovou implicitní metodu. Určíme její koeficienty a řád. Koeficienty metody jsou:

i	0	1	2
α_i	-1	0	1
β_i	1/3	4/3	1/3

Ověření konzistence a řádu pomocí Věty 15.2:

- $l = 0$: $\sum \alpha_i = -1 + 0 + 1 = 0$. (Splněno)
- $l = 1$: $\sum i\alpha_i = -1 \cdot 0 + 0 \cdot 1 + 1 \cdot 2 = 2$. Dále $\sum \beta_i = 1/3 + 4/3 + 1/3 = 6/3 = 2$. (Splněno, metoda je konzistentní, řád ≥ 1)
- $l = 2$:

$$\sum_{i=0}^{3} \alpha_i \frac{i^2}{2!} = -1 \cdot 0 + 0 + 1 \cdot \frac{4}{2} = 2.$$
$$\sum_{i=0}^{3} \beta_i \frac{i^1}{1!} = \frac{1}{3} \cdot 0 + \frac{4}{3} \cdot 1 + \frac{1}{3} \cdot 2 = \frac{6}{3} = 2.$$

- $l = 3$:

$$\sum_{i=0}^{3} \alpha_i \frac{i^3}{3!} = -1 \cdot 0 + 0 + 1 \cdot \frac{8}{6} = \frac{8}{6}.$$
$$\sum_{i=0}^{3} \beta_i \frac{i^2}{2!} = \frac{1}{3} \cdot \frac{0}{2} + \frac{4}{3} \cdot \frac{1}{2} + \frac{1}{3} \cdot \frac{4}{2} = \frac{8}{6}.$$

- $l = 4$:

$$\sum_{i=0}^{3} \alpha_i \frac{i^4}{4!} = -1 \cdot 0 + 0 + 1 \cdot \frac{16}{24} = \frac{2}{3}.$$
$$\sum_{i=0}^{3} \beta_i \frac{i^3}{3!} = \frac{1}{3} \cdot \frac{0}{6} + \frac{4}{3} \cdot \frac{1}{6} + \frac{1}{3} \cdot \frac{8}{6} = \frac{2}{3}.$$

- $l = 5$:

$$\sum_{i=0}^{3} \alpha_i \frac{i^5}{5!} = -1 \cdot 0 + 0 + 1 \cdot \frac{32}{120} = \frac{4}{15}.$$
$$\sum_{i=0}^{3} \beta_i \frac{i^4}{4!} = \frac{1}{3} \cdot \frac{0}{6} + \frac{4}{3} \cdot \frac{1}{24} + \frac{1}{3} \cdot \frac{16}{24} = \frac{5}{18}.$$

Pro $l = 5$ již nemáme rovnost, tedy metoda 4. řádu. Podle tzv. Dahlquistovy bariéry je toto maximální možný řád pro 2-krokovou metodu.

15.4 Stabilita

Příklad 15.4 (Nestabilní metoda)**.** Uvažujme 2-krokovou metodu 2. řádu danou vztahem

$$y_{n+2} - 4y_{n+1} + 3y_n = -2hf_n.$$

Zkoumejme její chování na modelové rovnici $y' = 0$ s počáteční podmínkou $y(0) = y_0$. V tomto případě je $f_n = 0$, takže VKM přejde na lineární rekurenci (diferenční rovnici):

$$y_{n+2} - 4y_{n+1} + 3y_n = 0.$$

Hledáme posloupnost $\{y_n\}_{n=0}^{\infty}$, která splňuje tuto rovnici. Přesné řešení diferenciální rovnice je $y(x) \equiv$ konst, takže bychom chtěli, aby numerické řešení bylo přinejmenším omezené a "nešlo do nekonečna".

Nicméně, jak uvidíme, řešení této diferenční rovnice obsahuje složku, která exponenciálně roste. I libovolně malá počáteční chyba (např. zaokrouhlovací) se s každým krokem zvětšuje. Pro rostoucí n chyba narůstá exponenciálně (jev zvaný nestabilita). Pro $h \to 0$ se navíc toto chování jen zhoršuje.

Řešení lineárních diferenčních rovnic

Uvažujme modelovou rovnici $y' = 0$. Obecná VKM $\sum_{i=0}^{k} \alpha_i y_{n+i} = h \sum_{i=0}^{k} \beta_i f_{n+i}$ má pak tvar homogenní lineární diferenční rovnice s konstantními koeficienty:

$$\sum_{i=0}^{k} \alpha_i y_{n+i} = 0.$$

Hledáme řešení ve tvaru $y_n = \xi^n$ (kde $\xi \in \mathbf{C}$ je konstanta). Dosazením do diferenční rovnice dostáváme:

$$0 = \sum_{i=0}^{k} \alpha_i \xi^{n+i} = \xi^n \sum_{i=0}^{k} \alpha_i \xi^i.$$

Pokud $\xi \neq 0$, můžeme krátit ξ^n a dostáváme podmínku pro ξ:

$$0 = \sum_{i=0}^{k} \alpha_i \xi^i.$$

DEFINICE 15.5: *Charakteristický polynom*

Polynom

$$\rho(x) = \sum_{i=0}^{k} \alpha_i x^i$$

nazýváme **charakteristickým polynomem** diferenční rovnice (resp. dané VKM).

Vlastně jsme dokázali lemma:

LEMMA 15.4: *Kořen charakteristického polynomu*

Posloupnost $\{\xi^n\}_{n=0}^{\infty}$ je řešením příslušné diferenční rovnice právě tehdy, když číslo ξ je kořenem charakteristického polynomu $\rho(x)$.

Poznámka 15.1 (Princip superpozice)**.** Protože je diferenční rovnice lineární, platí princip superpozice: Jsou-li posloupnosti $\{y_n\}_{n\in\mathbf{Z}}$ a $\{z_n\}_{n\in\mathbf{Z}}$ řešeními diferenční rovnice, pak i jejich lineární kombinace $\{ay_n + bz_n\}_{n\in\mathbf{Z}}$ je řešením pro libovolná $a, b \in \mathbf{R}$.

VĚTA 15.5: *Obecné řešení pro jednoduché kořeny*

Má-li charakteristický polynom $\rho(x)$ jednoduché kořeny (násobnost 1) $\xi_1, \ldots, \xi_k$, pak obecné řešení diferenční rovnice je tvaru

$$y_n = a_1\xi_1^n + a_2\xi_2^n + \cdots + a_k\xi_k^n = \sum_{j=1}^{k} a_j\xi_j^n, \quad n \in \mathbf{Z},$$

kde $a_1, \ldots, a_k \in \mathbf{R}$ jsou konstanty. Žádná jiná řešení neexistují. Konstanty $a_1, \ldots, a_k$ jsou jednoznačně určeny počátečními hodnotami $y_0, \ldots, y_{k-1}$.

Příklad 15.5 (Fibonacciho posloupnost)**.** Fibonacciho posloupnost je dána rekurencí $F_{n+2} - F_{n+1} - F_n = 0$. Charakteristický polynom je $\rho(x) = x^2 - x - 1$. Jeho kořeny jsou $\xi_{1,2} = \frac{1\pm\sqrt{5}}{2}$. Obecné řešení je tvaru $F_n = c_1\left(\frac{1+\sqrt{5}}{2}\right)^n + c_2\left(\frac{1-\sqrt{5}}{2}\right)^n$. Protože $|\xi_1| > 1$, posloupnost roste nade všechny meze (pokud $c_1 \neq 0$). Požadujeme-li navíc $F_0 = 0, F_1 = 1$, vyjde $c_1 = 1/\sqrt{5}, c_2 = -1/\sqrt{5}$ (tzv. Binetova formule pro n-té Fibonacciho číslo).

Příklad 15.6 (Detailní rozbor nestabilní metody)**.** Vraťme se k metodě z Příkladu 15.4:

$$y_{n+2} - 4y_{n+1} + 3y_n = 0.$$

Charakteristický polynom je $\rho(x) = x^2 - 4x + 3 = 0$. Kořeny jsou:

$$\xi_{1,2} = \frac{4 \pm \sqrt{16-12}}{2} = \frac{4\pm 2}{2} \implies \xi_1 = 1, \xi_2 = 3.$$

Obecné řešení homogenní diferenční rovnice je tedy:

$$y_n = a \cdot 1^n + b \cdot 3^n = a + b \cdot 3^n.$$

Uvažujme počáteční podmínky s malou chybou: $y_0 = 0$ a $y_1 = \varepsilon$ (kde ε představuje např.

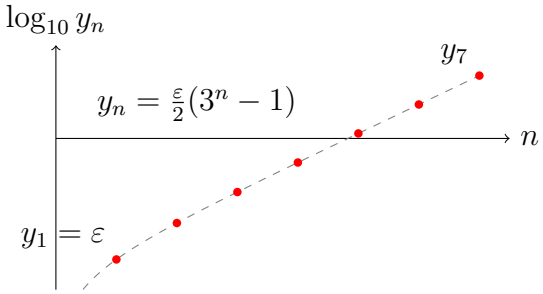
zaokrouhlovací chybu v prvním kroku). Dosadíme do obecného řešení:

$$n = 0: \quad a + b = 0 \implies a = -b,$$
$$n = 1: \quad a + 3b = \varepsilon \implies -b + 3b = 2b = \varepsilon \implies b = \frac{\varepsilon}{2}, a = -\frac{\varepsilon}{2}.$$

Partikulární řešení pro tyto počáteční podmínky je tedy:

$$y_n = -\frac{\varepsilon}{2} + \frac{\varepsilon}{2} \cdot 3^n = \frac{\varepsilon}{2}(3^n - 1).$$

Vidíme, že i pro velmi malé ε (např. 10^{-10}) hodnota y_n pro rostoucí n velmi rychle roste nade všechny meze jako 3^n (pro $\varepsilon = 10^{-10}$ bude např. $y_{40} \sim 10^9$). Tato metoda je tedy **nestabilní**.



Obrázek 60: Ukázka nestability metody $y_{n+2} - 4y_{n+1} + 3y_n = 0$ s počáteční poruchou $y_1 = \varepsilon$ (logaritmická škála).

Podmínka stability

Věta 15.6: *Podmínka stability VKM*

Nutnou podmínkou stability vícekrokové metody je, aby pro všechny kořeny ξ jejího charakteristického polynomu $\rho(x)$ platilo

$$|\xi| \leq 1.$$

Poznámka 15.2 (Univerzální kořen konzistentních metod)**.** Pro konzistentní metody platí $\sum_{i=0}^{k} \alpha_i = 0$, což znamená $\rho(1) = 0$. Tedy $\xi = 1$ je vždy kořenem charakteristického polynomu jakékoli konzistentní metody.

Vícenásobné kořeny charakteristického polynomu

Doteď jsme uvažovali jen jednonásobné kořeny charakteristického polynomu. Pro vícenásobné kořeny je situace o něco složitější:

LEMMA 15.7: *Řešení pro vícenásobné kořeny*

Nechť ξ je l-násobný kořen charakteristického polynomu $\rho(x)$. Pak posloupnosti

$$\{\xi^n\}_{n\in\mathbf{Z}}, \quad \{n\xi^{n-1}\}_{n\in\mathbf{Z}}, \quad \ldots, \quad \{n(n-1)\ldots(n-l+2)\xi^{n-l+1}\}_{n\in\mathbf{Z}}$$

(celkem l posloupností) řeší příslušnou homogenní diferenční rovnici. Obecné řešení je lineární kombinací těchto posloupností (a posloupností příslušných ostatním kořenům).

Důkaz. Ukážeme tvrzení pro dvojnásobný kořen ξ charakteristického polynomu ρ. Nechť ξ je dvojnásobný kořen $\rho(x)$, tj. $\rho(\xi) = 0$ a $\rho'(\xi) = 0$. Definujme pomocný polynom

$$\varphi_n(x) = x^n\rho(x) = x^n \sum_{i=0}^{k} \alpha_i x^i = \sum_{i=0}^{k} \alpha_i x^{n+i}.$$

Jelikož ξ je dvojnásobný kořen $\rho(x)$, je i dvojnásobným kořenem $\varphi_n(x)$ (pro $\xi \neq 0$). Platí tedy:

1. $\varphi_n(\xi) = 0$:
$$\sum_{i=0}^{k} \alpha_i \xi^{n+i} = 0.$$
To znamená, že posloupnost $\{\xi^n\}_{n\in\mathbb{N}}$ řeší danou diferenční rovnici.

2. $\varphi_n'(\xi) = 0$: Derivací $\varphi_n(x)$ dostaneme
$$\varphi_n'(x) = \sum_{i=0}^{k} \alpha_i (n+i) x^{n+i-1}.$$
Dosazením ξ máme
$$0 = \sum_{i=0}^{k} \alpha_i \underbrace{(n+i)\xi^{n+i-1}}_{y_{n+i}}.$$
Označme $y_n = n\xi^{n-1}$. Pak posloupnost $\{y_n\} = \{n\xi^{n-1}\}$ je rovněž řešením diferenční rovnice.

Pro kořeny vyšší násobnosti l bychom postupovali analogicky, jen s vyššími derivacemi. □

Posloupnosti z Lemmatu 15.7 rostou i jinak než exponenciálně. Např. pokud $\xi = 1$ je dvojnásobný kořen, pak posloupnost $y_n = n$ řeší příslušnou diferenční rovnici. To sice není taková tragédie jako exponenciální růst, ale také tomu chceme zamezit:

DEFINICE 15.6: *Stabilita VKM*

Vícekroková metoda je **stabilní**, pokud pro každý kořen ξ charakteristického polynomu $\rho(x) = \sum_{i=0}^{k} \alpha_i x^i$ platí:

1. $|\xi| \leq 1$,
2. $|\xi| = 1 \implies \xi$ je jednoduchý kořen.

VĚTA 15.8: *Dahlquist*

Vícekroková metoda je konvergentní právě tehdy, když je stabilní a konzistentní.

Analogií k Větě 14.3 je u VKM následující věta.

Poznámka 15.3 (Jednokrokové metody)**.** U jednokrokových metod (JKM) máme vztah $y_{n+1} - y_n = \dots$. Charakteristický polynom je $\rho(x) = x - 1$, který má kořen $\xi = 1$. Tento kořen je jednoduchý a leží na jednotkové kružnici, takže JKM jsou z hlediska této teorie stabilní. POZOR! A-stabilita souvisela s nevhodnými vlastnostmi ODR, nikoli metody jako takové!

Soustavy diferenciálních rovnic Uvažujme vektorovou funkci $\vec{y}(x) = (y_1, \dots, y_N)^T$ a soustavu

$$\vec{y}\,'(x) = \vec{f}(x, \vec{y}(x)).$$

Metody aplikujeme po složkách. Například Eulerova metoda:

$$\vec{y}_{n+1} = \vec{y}_n + h\vec{f}(x_n, \vec{y}_n).$$